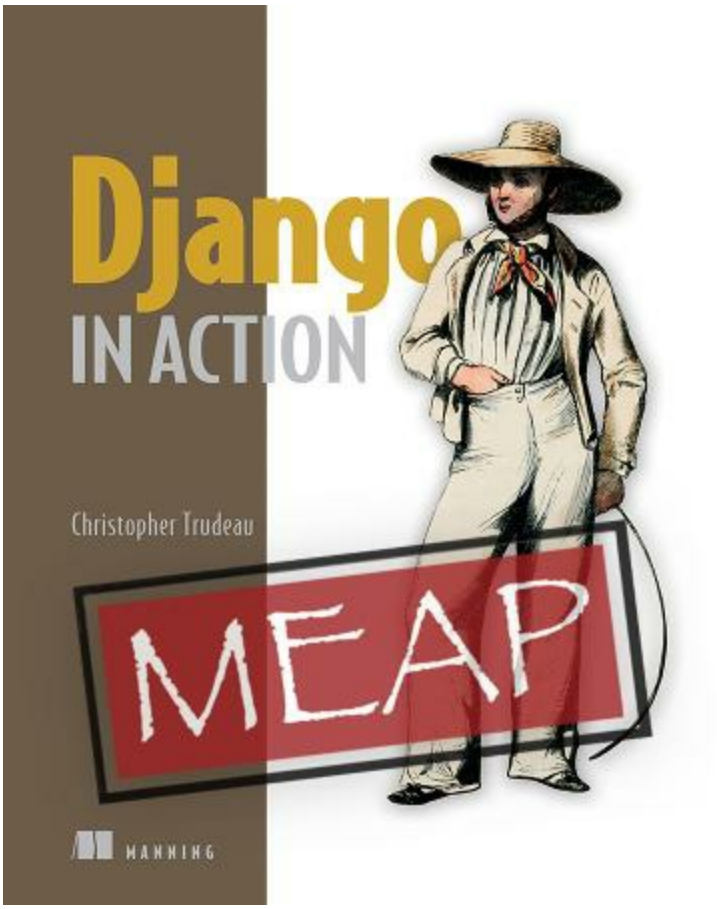
Django IN ACTION

Christopher Trudeau

MEAP

MANNING

Django in Action MEAP V01



MEAP VERSION 1

MANNING PUBLICATIONS

Welcome

Dear reader,

As a first time author, it is hard to express just how fun it is to write that greeting! Thank you for purchasing the MEAP of *Django In Action*. This book starts you on your journey with Django, showing you what you need to get a project going, and proceeds all the way to using third-party libraries to enhance your website's offerings. This book is for developers who are already familiar with Python. A basic understanding of HTML tags will also be beneficial.

Django has been around for over 20 years, and it has been quite the journey. In that time, a ton of features have been added, making Django one of the most comprehensive web framework toolkits available. The book is divided into three parts. The first part gives you a grounding in Django fundamentals and will get you to a place where you can write your own websites. Part 2 is about the many features built into Django meaning you have less code to write. It includes topics such as creating multi-user sites, dealing with user-uploaded content, and writing automated tests. Part 3 introduces you to a variety of third-party libraries that will enable you to write more complex websites. Here you'll learn about REST APIs, HTMX, custom template tags, and more.

There is a lot of content out there on Django already, but it can be frustrating trying to piece it all together. This book builds a single project, "RiffMates", a classified-ads site for musicians seeking bands and bands seeking musicians. As you add features to the project in each chapter, you'll be learning something new and gaining handson experience building a Django site. Each chapter includes exercises, so you can test your learning as you go along. Sample code and answers to the exercises are available at:

https://github.com/cltrudeau/django-in-action/

Each directory contains a snapshot of the project's progress within the

chapters.

Working on this book has been fun, but the true test is whether you find it useful. I greatly appreciate any feedback. Please post any questions, comments, suggestions, or bugs (never, never, that's inconceivable!), in theliveBook Discussion forum. Your early feedback on the book will be invaluable in ensuring a better final product.

— Christopher Trudeau

In this book

1 Django unfolds

This chapter covers

- What the Django Framework is and why to use it
- The actions Django performs when you enter a URL in your browser
- Server-side rendering vs single page applications
- The kinds of projects you can do with Django

You've written a brilliant Python script that runs on your local machine and now want other people be able to use it. You can use magical packaging tools and send your program to your users, but then you're still stuck with the challenge of whether they have Python installed. Python does not come by default on many computing platforms, and you have the added problem of making sure the Python version your script requires is installed on your user's

Alternatively, you can turn your Python program into a web application. In this case, your user's interface becomes a web browser, and those are installed everywhere. Your users no longer even need to know what Python is, they simply point their browser at the right URL and can use your software. To do this, your program needs to be adapted to run on a web server. This requires a bit of work, but also has the advantage of a single environment: you are in full control over what version of Python gets run.

If you've already got some experience with Python, a great answer to building web applications is Django. Django is a third-party Python framework that lets you write code that runs on a web server. Using Django, you write Python code called a *view* that is tied to a URL. When your users visit the URL, the Django view runs and returns results to the user's browser. Django does a lot more than that though, it includes tools for doing the following:

- Routing and managing URLs
- Encapsulating what code gets run per page visit

- Reading and writing to databases
- Composing HTML output based on reusable chunks
- Writing multi-user enabled sites
- Managing user authentication and authorization to your site
- Web-based administrative tools that manage all these features

Django started out as a tool for writing newspaper articles at the Lawrence Journal-World in Lawrence, Kansas. Rather than having each article be a single file on a web server, the newspaper wanted a way of re-using pieces of HTML. You don't want to write the newspaper's banner in every file, you want to compose it like you would with code. You also probably don't want your reporters writing HTML; it isn't their expertise.

Django evolved as both a coding framework and a series of tools to manage web content. Among other things, the framework provides the ability to compose HTML out of reusable pieces solving the banner problem, and one of the tools it includes is a web-based interface for the creation of content. Reporters can write articles without worrying about HTML. Django's tools are built on top of the framework, giving you the power to customize them to suit your needs.

The robustness of the Django framework has lead to widespread industry adoption. Reddit, YouTube, The Onion, Pinterest, Netflix, Dropbox, and Spotify are just a few of the organizations that use Django. Sites like these need scale, and Django has proven itself. Whether you want to build something small, or hope to become a massive site, Django can help you. It is a mature platform with a vibrant community focused on a maintaining and evolving a well-tested, secure framework. To top it all off, the original developers at the Journal-World open-sourced it, meaning it is freely available for you to both use and modify for your needs.

1.1 Django's Parts

Django is built on three core parts:

1. A mapper between URLs and view code
2. An abstraction for interacting with a database

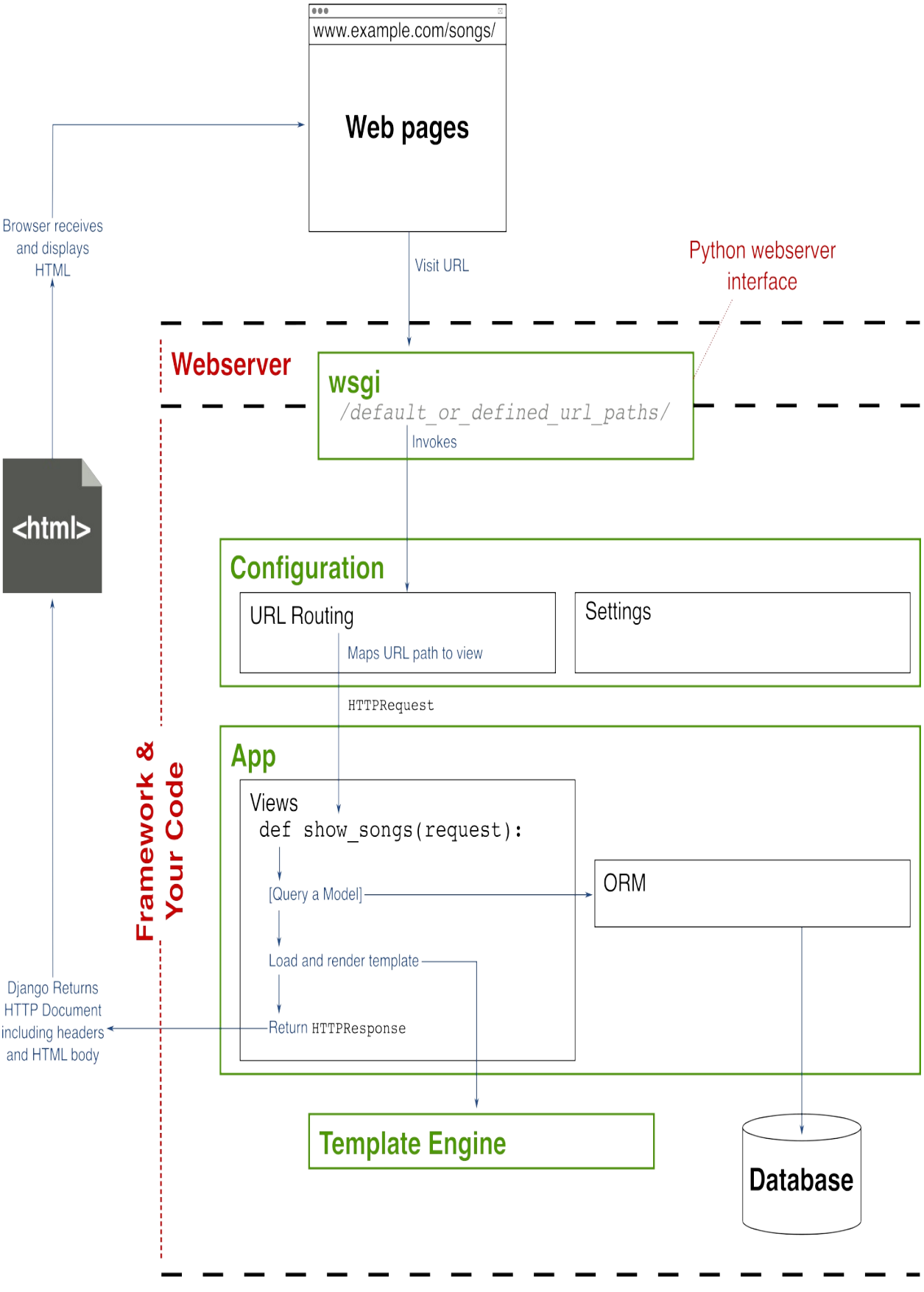
3. A templating system to manage your HTML like code

On top of these three parts is a large selection of tools. The tools include things like a web interface for modifying content in the database and the code needed to write multi-user sites. Each of these is built using the core parts. This means you can incorporate them into your site and modify them the same way you write your web application.

When a user visits a web page, the web server determines how to respond based on the URL. In the case of frameworks like Django, the server gets configured so some, or all of the URLs get handed to the framework. This hand-off gets done through a generic Python web interface called the *Web Server Gateway Interace* (*WSGI*).

When you visit a Django URL, the web server calls the framework through WSGI at which point Django is responsible for generating the page response. The first core part of Django determines what code to call based on the URL. This is done through a mapping in your Django project. The mapping is between URLs and response handlers known as *views*. A view is a function or class that generates content which Django then parcels as a response to the web browser. The complete flow starting with the browser request, web server hand-off via WSGI, Django mapping the URL, to the view formulating the response object that Django turns into the response for the browser, is shown in figure 1.1.

Figure 1.1. Django handling a page visit


www.example.com/songs/
Web pages
Browser receives and displays HTML
Visit URL
Python webserver interface
Webserver
wsgi
/default_or_defined_url_paths/
Invokes
<html>
Configuration
URL Routing
Maps URL path to view
Settings
HTTPRequest
App
Views
def show_songs(request):
[Query a Model]
ORM
Load and render template
Return HTTPResponse
Framework & Your Code
Django Returns HTTP Document including headers and HTML body
Template Engine
Database

The mapping between URLs and views gets done through a `list` data structure inside a Python file. Each mapping gets defined through the construction of a `path` object which contains the URL to map, and a reference to the handling view. Mappings can also be named to reference them elsewhere in your code.

Visiting a URL causes Django to loop through all the mappings and look for a match. If there isn't a match, Django shows a 404 page. If there is a match, the corresponding view gets invoked.

The vast majority of the code you write for a Django project is the actual view code. Views return a response object that encapsulates the information to send back to the browser. The most common of these objects is `HTTPResponse`, which returns text over HTTP, the text typically being HTML.

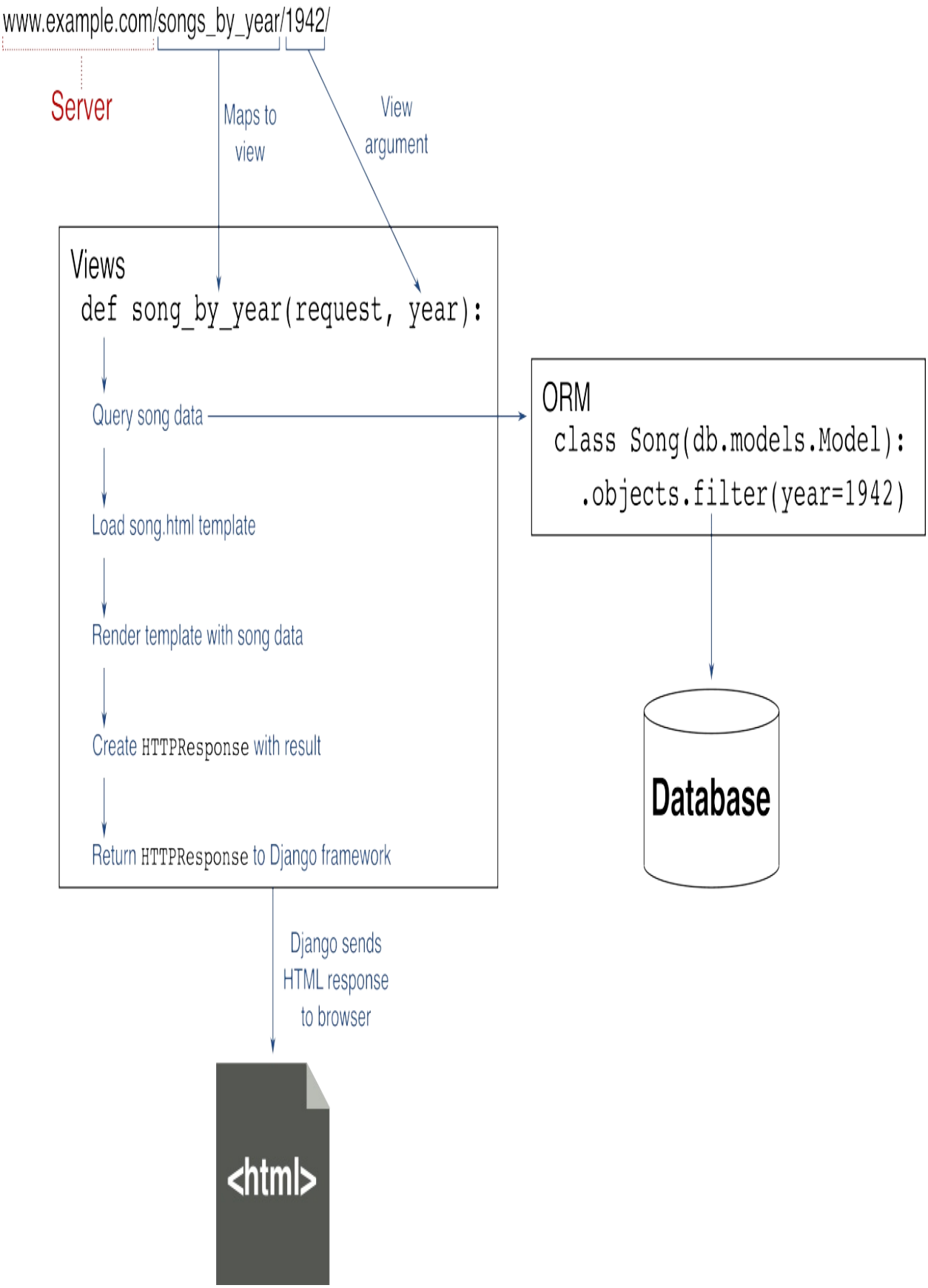
Inside your view, you determine what the user sees. Part of that may involve interacting with a database. Your data may be what parts are in your warehouse, what books in your library, or which users can access which pages. To facilitate communication with a database, Django includes an *Object Relational Mapping (ORM)*, the second of the three core parts. This is an abstraction that allows you to manage content in a database through Python classes and objects. If your web page requires information from the database, the ORM is used to look up a `QuerySet` object that contains database results.

The third core part is the templating engine, which helps you create and manage the HTML within the page response. HTML is a fairly verbose text format often containing a lot of repeated content. Using the templating engine, you define HTML templates that describe parts of a web page. The parts get assembled, composed, and reused to form the response to the user.

For example, a navigation bar, or a footer, that appears on every page, only needs to be defined once and the template engine helps you include it everywhere. Templates are not HTML specific and are also used for parameterizing email or anywhere else you wish to manage text like reusable code.

Consider a web page that shows the songs released in a given year. The URL for all the songs in 1942 might be: `http://example.com/song_by_year/1942/`. The code for displaying this page starts with a mapping between the `/song_by_year/` portion of the URL and a function to handle the response. The mapping is capable of parsing the `/1942/` portion of the URL as a parameter, and passing it into the view function that generates the page. Figure 1.2 shows the parts of the URL handled by the view and how the view interacts with the ORM.

Figure 1.2. A song-by-year view rendering a web page based on an argument


www.example.com/songs_by_year/1942/
Server
Maps to view
View argument
Views
def song_by_year(request, year):
Query song data
Load song.html template
Render template with song data
Create HTTPResponse with result
Return HTTPResponse to Django framework
ORM
class Song(db.models.Model):
.objects.filter(year=1942)
Database
Django sends HTML response to browser
<html>

The `song_by_year` view does a look-up in the database for all the songs in 1942. The database query gets abstracted through a `Song` class in the ORM, and a query gets performed by invoking a function on that class. The result from the database gets encapsulated in a `QuerySet` response object containing all the corresponding songs.

With the data in hand, the view creates the HTML response. It loads the songs.html template, which inherits from a common template that includes the look-and-feel for the website. This means songs.html only contains the parts that are unique to that page. The template engine has a tag language allowing you to loop over content, generating the rows in an HTML `<table>`, populating each row with the information from a song from 1942.

Once the HTML has been rendered, it is packaged in an `HTTPResponse` object and sent back to the Django framework. The loading and rendering of a template is so common that Django even provides shortcuts to do this.

Django uses the response object to construct the data the web server needs for the browser. This includes all the HTTP headers and the body containing the HTML. The content gets returned to the web server through WSGI, and the server sends it down to the browser. That’s how you see your page full of songs.

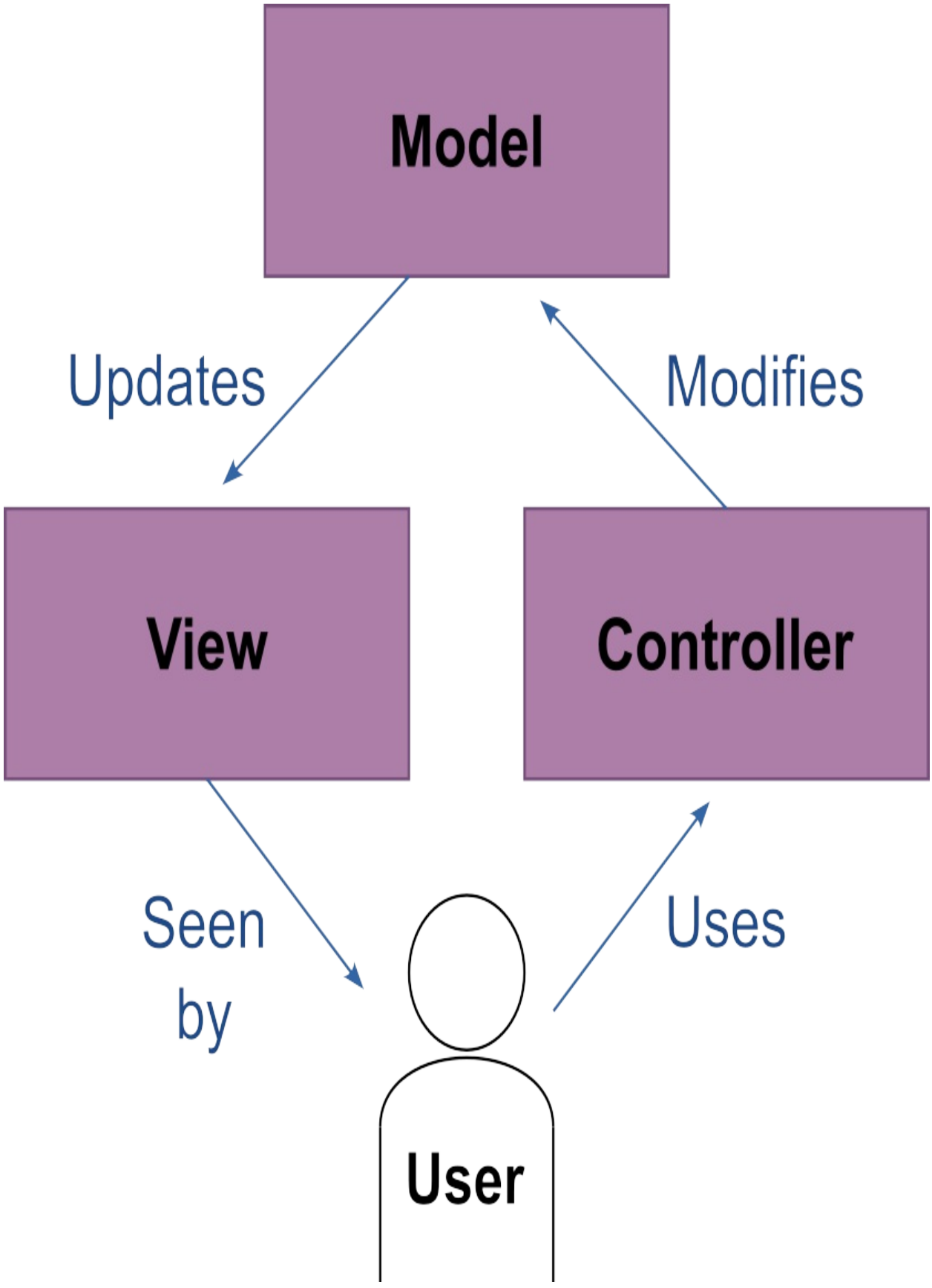
1.1.1 Mapping URLs, Django views, and the MVC model

The simplest form of a website is a mapping between a file and a URL. A web page that shows a picture of a kitten is at least two files: one for the HTML document and one for the image of the kitten. Simple file-to-URL mapped sites like this are called *static sites*. If you want your users to be able to upload their own images or add comments to your page, your site can no longer be just static files. A *dynamic site* is one that determines a page’s content when it gets visited.

Dynamic sites are more complicated than static ones, they need an interface for the users to interact with, they need somewhere to store data like uploaded pictures and comments, and they need logic for knowing how to combine these into content.

Django’s primary purpose is building dynamic websites. In doing so, it uses a common architectural pattern for user interfaces known as the *Model-View-Controller (MVC)* method. This method breaks software into three pieces as shown in figure 1.3:

Figure 1.3. Model-View-Controller architectural pattern


Model
Updates
Modifies
View
Controller
Seen by
Uses
User

1. **Model:** represents the logic and data of an application independent of its user interface
2. **View:** presents data to the user, possibly with multiple views for the same data
3. **Controller:** an input and control mechanism the user uses to interact with models and views

The MVC is a good way of thinking about how software interacts with users but Django is more inspired by it than strictly governed by it. The line between what is a View and a Controller is a little fuzzy. This fuzziness provides the added benefit of creating entertaining debate on the internet.

When a web server passes a URL request to Django through WSGI, Django looks that URL up in its master mapping file named `urls.py`, and as the name implies it is just Python. The file can be renamed, but there really isn't any reason to do so. The `urls.py` file contains a variable called `url_patterns` which is a `list` containing Django `path` objects. Each `path` object either maps a URL to a Django view or loads another file containing more mappings.

A Django view is just Python code. It can be a function that returns a response document, or a class with methods that do the same. Each view takes at least one argument called the `request`. This variable contains information about the HTTP Request that was made. The request object includes the URL that was hit, any query parameters in the URL, HTTP method information, HTTP headers, and in the case of pages with forms, the data the user filled in. Not every view needs all this information, but it is there when you require it.

The Django view is really a hybrid of both the View and Controller parts of the MVC, hence the debate on the internet. The mapping part is very much a Controller. The Django *view*, confusingly, is also mostly a Controller. The view is responsible for returning the response to the browser, which makes it also a View.

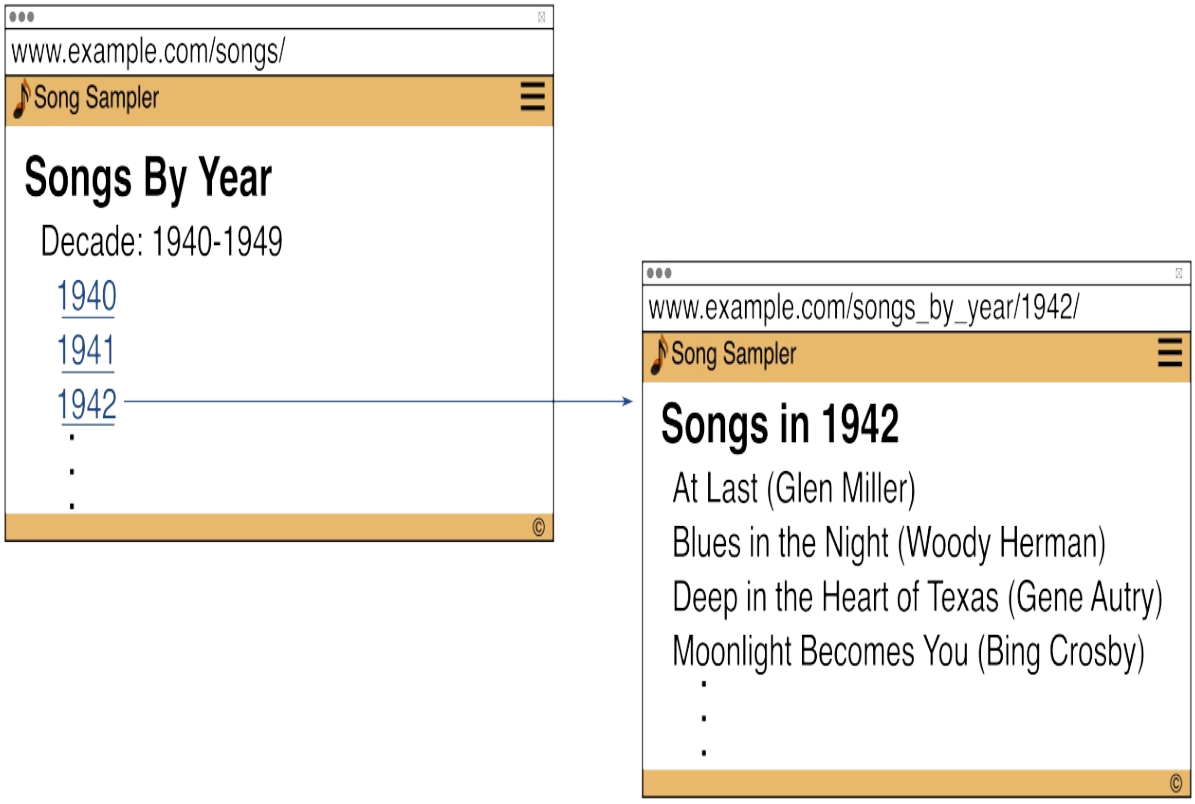
Consider the prior song-by-year page example. The same Django-view outputs different results depending on the year passed in. The HTML template used in the Django-view is an MVC-View: it is responsible for

formatting the look-and-feel of the response. The database lookup in the Djang-view is an MVC-Controller, changing what data gets rendered by the MVC-View based on the "year" argument.

Each page in your web application is generated by a view. Most are views that you write, but some are views you configure, taking advantage of tools built into Django. Figure 1.4 shows four web pages, two of which are views you would write, one is a Django tool view, and the other halfway in between.

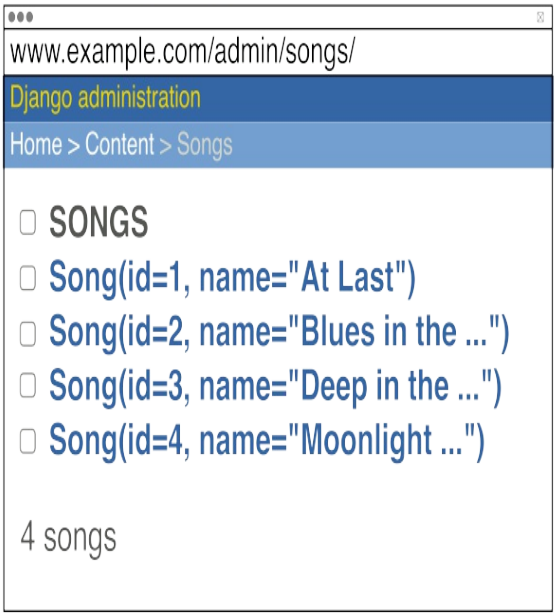
Figure 1.4. Most pages will be your views with your templates, but Django provides tools to reduce your work

Your views using your templates



Your templates, Django's views

Built-in Django Admin



A typical view is responsible for querying data out of a database, making logic decisions about output, and then loading, rendering, and returning the result of a template. In the "Songs By Year" page, the view reads all the years from the song database and groups them by decade. The grouped data is sent to the template engine that renders the page, including links to each corresponding detail page, "Songs in 1942" being just one example. The same view generates all "Songs in" pages, with the year being passed as an argument. The argument changes how the MVC-Controller works, but uses the same MVC-view for the output.

1.1.2 Databases: Dynamic sites need data

Views are key to the `"dynamic"` part of a dynamic web-site. They can create what is on a page based on data or input from the user. That data might be products in a catalog, scores to last night’s game, or your latest recipe for gnocchi. In all these cases, your data has to live somewhere. Enter the database.

Databases are a field in computing in their own right. There are different kinds, each with its advantages and disadvantages. There are many implementations of each kind and specialty languages written just for communicating with them. All of this can be a little overwhelming. Django implements an *Object Relational Mapping (ORM)* to abstract this away.

Figure 1.5. Tables store either item information or relationship information

Item Tables

Book Table

ID	Title	Year
10	Martian Chronicles, The	1950
11	Farenheit 451	1953
12	Carrie	1974
13	Firestarter	1980
14	Talisman, The	1984
15	Mystery	1993

Unique key for each item

Author Table

ID	Last_name	First_name
1	Bradbury	Ray
2	King	Stephen
3	Straub	Peter

Publisher Table

ID	Name
23	Viking Press
24	Doubleday
25	Ballantine
26	Dutton

Relationship Tables

Book-Publisher Table

ID	Book_ID	Publisher_ID
40	10	24
41	11	25
42	12	24
43	13	23
44	14	23
45	15	26

Doubleday published both Martian Chronicles and Carrie

Keys relate Books to Publishers

Book-Author Table

ID	Book_ID	Author_ID
80	10	1
81	11	1
82	12	2
83	13	2
84	14	2
85	14	3
86	15	3

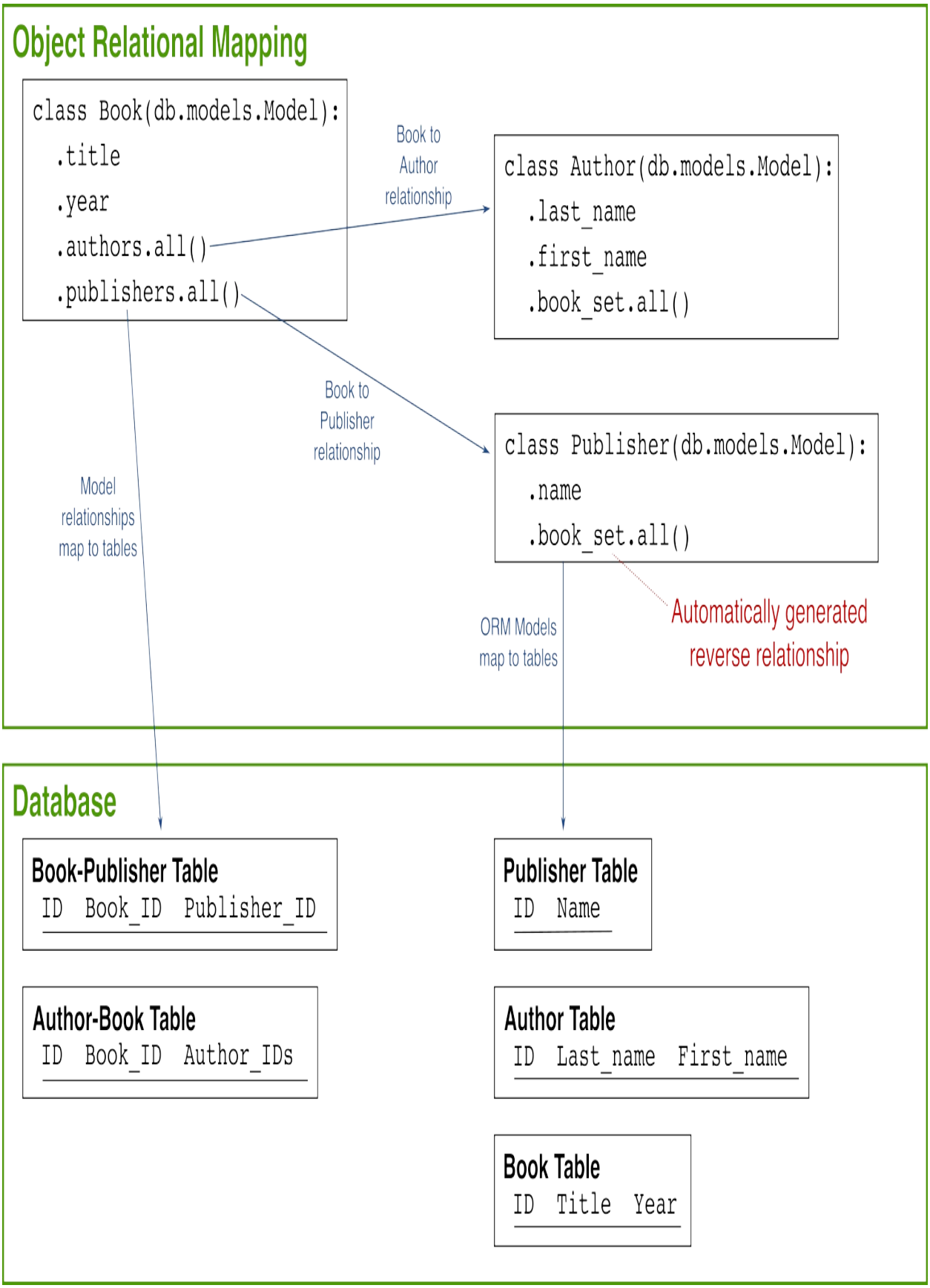
Two authors, one book

Keys relate Books to Authors

Consider a database containing information about books. You are going to have different kinds of data: an author (that's me), a publisher (that's Manning), and a book (that's what you're holding). There are many authors, many books, and many publishers. Some books have multiple authors, and some authors have written multiple books. A relational database stores this information in tables which are similar to a sheet in an Excel spreadsheet. It then expresses relationships between these items with keys. The Author table might reference the Book table using a key that is a unique identifier for a Book.

An Object Relational Mapping is an object-oriented way of representing data and the relationships between them. Each row in a database table gets expressed as an object. Instead of thinking about the Book table you think about the Book class. An ORM lets you query a specific book as well as linking to the book's associated author and publisher, which themselves are objects. Django automatically creates reverse relationships, so if you have an Author object you can query all the author's books without having to write any additional code. The relationships between Book, Author, and Publisher objects is shown in figure 1.6, along with their corresponding database table representations.

Figure 1.6. ORM Models are data objects that map to underlying item and relationship tables


Object Relational Mapping
class Book(db.models.Model):
.title
.year
.authors.all()
.publishers.all()
Book to Author relationship
class Author(db.models.Model):
.last_name
.first_name
.book_set.all()
Book to Publisher relationship
class Publisher(db.models.Model):
.name
.book_set.all()
Model relationships map to tables
ORM Models map to tables
Automatically generated reverse relationship
Database
Book-Publisher Table
ID Book_ID Publisher_ID
Publisher Table
ID Name
Author-Book Table
ID Book_ID Author_IDs
Author Table
ID Last_name First_name
Book Table
ID Title Year

Django's ORM allows you to define data models that map to tables in a database. You create rows in those tables by creating and saving objects. You can create, read, update, and delete data using the ORM, never having to touch the database itself.

Django's ORM comes with support for multiple databases, including PostgreSQL, MariaDB, MySQL, Oracle, and SQLite. There are also many third party plugins for other databases. There are even plugins for non-relational databases like the MongoDB No-SQL server.

When a Django view requires information out of the database it uses one or more ORM classes which have been defined by the programmer. Each class inherits from an ORM base class called `Model`. The base class provides methods for querying the database. For example, calling `.objects.filter(last_name__startswith="A")` on `Author` returns all the authors with a last name beginning with `"A"`. Query results get represented by a common Django object called a `QuerySet`. The `QuerySet` can be passed as part of a data context to the template engine. The template engine generates HTML using the results. A list of authors might generate a bullet-point list in the output. Django also provides a way to deal with the database directly if the abstraction is insufficient in specific cases.

1.1.3 Structuring HTML for reuse

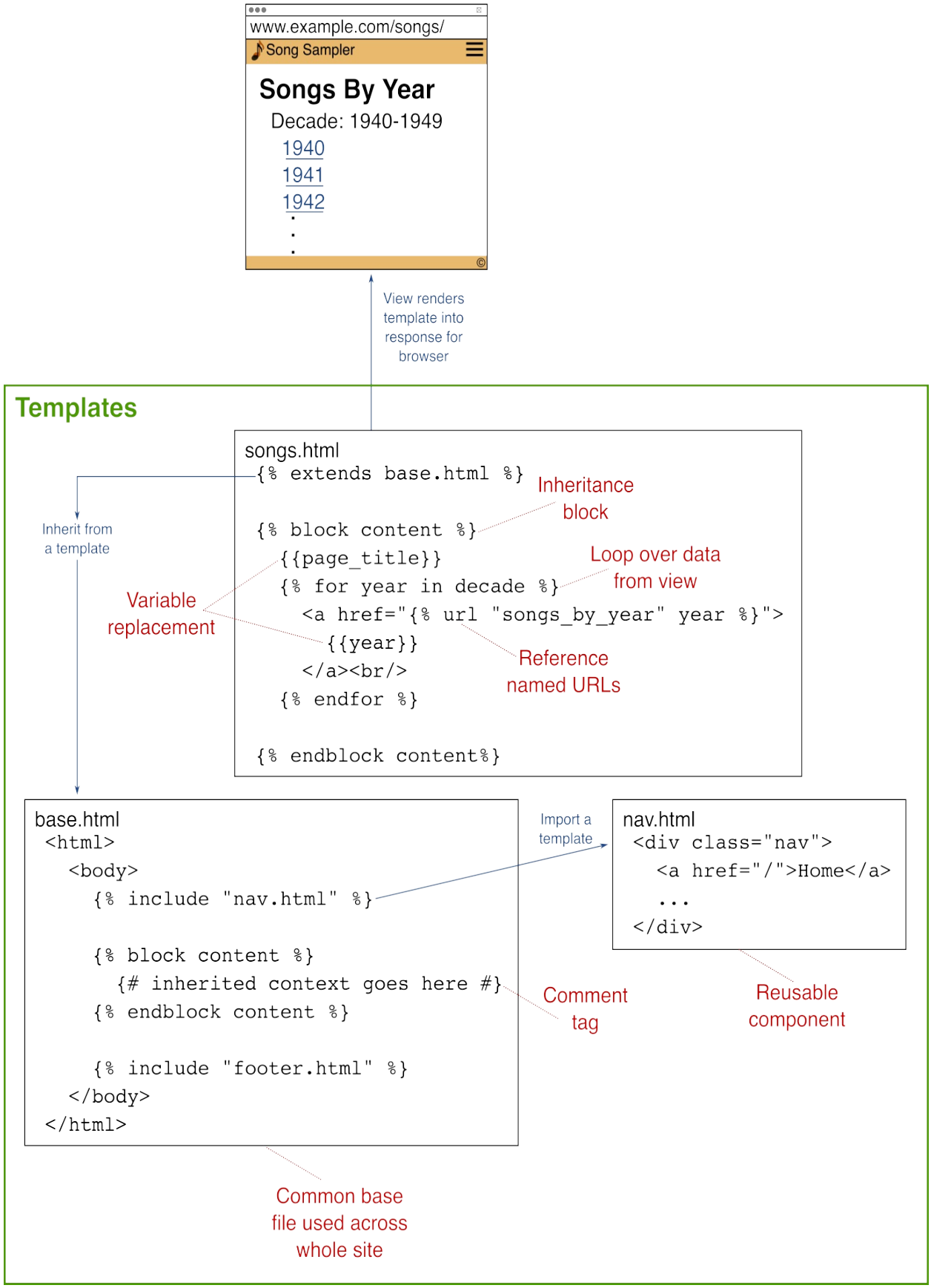
Your Django view is responsible for returning content to the browser, which most of the time is HTML. HTML was originally designed for physicists to write easily accessible articles for other physicists. Modern web pages have more in common with marketing brochures than physics articles. Each page in a website tends to include a lot of repetition, for example the navigation header that appears on every page.

Django includes a templating engine. The engine's job is to compose pieces of content together into a page. Using it, you only have to write the navigation header once, and then it can be included in every page of your site. To change the navigation header, you only have to do it in one place.

Your view likely wants to show some of that previously mentioned data on

your page. Lists of things can get quite repetitive. In Python if you want to print a long list you don't hard-code it. Instead, you put a `print` statement inside a `for` loop that iterates on the list. The templating engine lets you do the same using *tags* and *filters*. Tags are instructions to the engine for conditional rendering, while filters change how data is displayed. Using these, you can create loops, conditionally render blocks, print a date in your user's format, and much more. Figure 1.7 shows how templates are composed and how to use tags, denoted by `{% %}`. All of this is covered in detail in the chapter on the template engine.

Figure 1.7. Composing HTML templates into a rendered page


www.example.com/songs/
Song Sampler
Songs By Year
Decade: 1940-1949
1940
1941
1942
View renders template into response for browser
Templates
songs.html
{% extends base.html %}
{% block content %}
{{page_title}}
{% for year in decade %}

{{year}}

{% endfor %}
{% endblock content%}
Inheritance block
Loop over data from view
Variable replacement
Reference named URLs
Inherit from a template
base.html
<html>
<body>
{% include "nav.html" %}
{% block content %}
{# inherited context goes here #}
{% endblock content %}
{% include "footer.html" %}
</body>
</html>
Import a template
nav.html

Home
...

Comment tag
Reusable component
Common base file used across whole site

Django is able to interact with different template engines. The original templating language that ships with Django, understandably, is called the *Django Templating Language* (*DTL*). Django also ships with the popular third party templating language Jinja2. This isn't maintained by the Django core team, and won't be covered in this book, but is included in the framework for the convenience of those familiar with it.

The DTL interface has three concepts: variable rendering, tags, and filters. Variable rendering replaces values in the template with data sent into the engine. Tags are like functions and provide looping, conditional blocks, and a number of operations on your templated text. Filters are variable modifiers changing values before they are displayed. For example, one filter takes a date object and formats it to a desired output. The DTL is also pluggable, allowing you to write your own tags and filters.

The output from a Django view is a response object, a web page uses one called `HTTPResponse`. Django provides shortcut functions that wrap the process of loading and rendering a template and embedding the result in an `HTTPResponse` object. The view returns object to Django, and Django then uses the object to formulate the content that is sent back to the browser. This includes any HTTP Headers along with the body containing the HTML.

1.1.4 Multi-user sites

From the very beginning Django was meant for multiple users. The Lawrence Journal-World's web application consists of views for the readers, showing the newspaper pages, as well as views for the journalists. The journalists' views are the tools they use to write articles.

Both the readers and the journalists are users, each using a different part of the web application. A regular reader should not be able to edit articles, they should only be able to read them. Controlling who can do what in a website can be broken into two parts: authentication and authorization. Authentication is the user proving who they are, and authorization is the permissions associated with that user. Django provides tools for managing all of this.

User identities really are just another kind of data, so a multi-user Django site uses the ORM to track user accounts. Authentication compares a username and password from a login page with a known set of values in the database. Because users tend to be forgetful, you need more than just a login page, you also need a password reset screen. To manage users, you want tools for listing and editing them. You need pages for creating new users, either by an administrator or through self sign-up. As users are people, you likely need to occasionally block one of them from logging in. Temporarily of course, I'm sure they'll see the error of their ways. Django comes with tools to help you do all of these things.

In addition to providing user management tools, Django includes a general tool called the Django Admin, which is a web interface to the database. Anything that you construct with the ORM can be managed through the Django Admin, including all of your user accounts. The Django Admin is a huge advantage and can save a large amount of time when building sites. The Django Admin is built on the same coding concepts as your own site: Django views, the ORM, and a series of templates. The tools that come with Django are built using the core parts of Django.

1.2 What can you do with Django?

Django's original purpose at the Lawrence Journal-World was a tool where journalists upload articles to the web. The generic term for this kind of software is a *content management system*. Django is a general web framework, but it has a heavy leaning towards content management. For the journalists, the content in question is newspaper articles. For a blog site, the content is a blog posting. In both of these cases the content is text that makes up the page, but that isn't the only choice.

An e-commerce website has pages of items for sale, a shopping cart, and payment processing. This isn't really considered a content management system, but it does have some things in common. The catalog pages query the database for available items and display them, the "content" here is the merchandise. A shopping cart is a special case of this, where the contents of the cart is specific to the user and this visit.

Consider what blog and e-commerce sites have in common: there are things stored in the database (blog posts and merchandise) and users visit pages that display one or more of those things. The Django ORM provides an object-oriented abstraction for database storage, making it easier to code and manage the "things", and the Django Admin provides a web interface for interacting with instances of those "things". Django views use the ORM to show the user things appropriate to the page, and construct the page through the template engine.

The general approach of objects plus views rendering templates means Django is a good fit for any multi-page web-site.

1.2.1 Server Side, Single Page, and Mixed Page Applications

As web applications have evolved, the architectural choices in building them has grown. Inside a Django view, the template engine gets called to render HTML to be sent to the browser. This is called *Server-Side Rendering*. By contrast, a *Single Page Application (SPA)* moves this rendering and some application logic into the browser itself. An SPA's approach to the MVC model is for the View and Controller parts to be entirely in the web browser.

Server-Side Rendering vs Single Page Applications

The phrase "Server Side Rendering" can be a little confusing. Of course, in all situations, your browser displays the HTML. The distinction is between where the HTML is composed. In Server Side Rendering, the server is responsible for assembling all of the pieces of the HTML and sending it down to the browser. The template engine runs on the server side, composing the parts of the template into a result, like in figure 1.7.

By contrast, Single Page Applications create the HTML in the browser. All the logic for page creation happens in the browser: you can think of this as a browser based template engine. The server sends the code for building the templates to the browser as well as the data necessary, but all page composition happens on the client side.

This doesn't mean Django can't be used to build SPAs. When a user visits an

SPA-based site, a JavaScript bundle gets downloaded. The bundle contains the complete user interface and a lot of the business logic for the application. The JavaScript presents a dynamic interface hosted in a single web-page, hence the name. The application still needs the web server as a data store and to execute certain kinds of business logic, and this part of the architecture can be done with Django.

In a Django-based SPA, a view doesn't render HTML, it instead serves data to the SPA. This is typically done using JSON. Plug-in libraries like the *Django REST Framework* and *Django Ninja* provide features for building APIs based on views as well as tools for serializing your ORM data objects into a form the JavaScript application can consume.

The world isn't binary. There are degrees between a traditional server-side application and a full single-page application. Using libraries like Vue.js to trigger API calls or HTMX which uses pieces of HTML, you can augment server side rendering with client side flexibility. Django fits this naturally, with views rendering full pages, supporting APIs, and HTML snippets as required.

1.2.2 When and where you should use it

There are lots of choices in the Python world for writing web applications. A popular alternative to Django is Flask. Flask is smaller than Django and only directly offers the routing and view portion. Flask tends to require other libraries to build out a full system. It is often paired with SQLAlchemy for an ORM, and Jinja2 for templating. By contrast, Django comes with an ORM and templating engine already. You may be able to get going with Flask a little faster than Django, but if your application grows you're going to need the other pieces as well. With the routing, views, template rendering, and ORM all coming in one package, you don't have to worry how and whether things will work together.

The flip side of this argument is that Django isn't really meant to be used in pieces. Although the ORM is fantastic, if you're not building a web application and only want the ORM, Django isn't the best fit. Django is intended to build web projects. You can use it as a tool for other things, but

you'll find you're fighting it.

Single-page applications are designed with a back-end server that provides data only, while the GUI component gets handled by the browser. The FastAPI library specializes in providing REST APIs and is a popular choice if you are using Python for the back-end of an SPA.

If you are writing Single-page applications, there is a strong argument to using Node.js on the server side. Having both the client and server language be JavaScript has advantages. If you write the server in Python you are going to have to duplicate some amount of code, usually around object to data conversion. Personally, my preference for writing Python code far outweighs this extra work. Django is sometimes viewed as a server-side only technology because that was what it was originally written for. The addition of libraries like the Django Rest Framework and Django Ninja (heavily inspired by FastAPI) make this a thing of the past. Django can be a solid framework for your back-end server in an SPA.

Django has been around for a long time, its first release at the Lawrence Journal-World was in 2003. It no longer qualifies as the new kid on the block. The programming world has a tendency towards rejecting the old, but a distinction should be made between dinosaurs and sharks. Dinosaurs are extinct, sharks while around with the dinosaurs are still mean eating machines today. Django is actively being developed and still releases major version yearly. Recent releases have focused on re-designing the underlying technology to better support asynchronous coding, making Django still competitive with newer web development stacks.

Django isn't just a shark, but a shark with accessories: it's modular in design and supports plugins. There are thousands of plugins available, one of which may already do what you were going to code for yourself. The Django Packages site (https://djangopackages.org/) has over 4000 installable libraries that cover topics such as: analytics, data tools, e-commerce, feed aggregation, forums, payment processing, polls, reporting, and social media. Whatever web application you want to build, there is likely a package that can help you.

1.2.3 Potential projects

There are many types of websites out there, and depending on what you're building, you may end up using different parts of the Django framework. The following is an incomplete list of the types of projects you might be interested in:

- **Blog:** This is a natural fit for Django, with the ORM storing article content, and views responsible for showing articles and table-of-content pages. Wiki sites and newspapers are a variation on this, where instead of blog posts you have encyclopedic information or newspaper articles as content.
- **E-Commerce:** These sites are all about the catalog. The ORM stores information about your merchandise while the views present them to the users. In complex sites you may be interacting with an inventory back-end instead, or exporting shopping patterns for data analysis. The Django Admin provides an out-of-the-box inventory management system by working in conjunction with the ORM.
- **Content Sites:** A lot of websites are repositories of specialty information. Consider the Internet Movie Database (IMDB), each is really a front-end to a database entry describing information about a movie. Django's ORM and the Django Admin make it a good tool for these kinds of sites.
- **Document Management:** This is a broad class of sites where some content gets served from the file system instead of in the ORM. Typically, the ORM contains a lightweight object that points to the file on the server or CDN. Users then interact with pages to see or download the documents. Video sites are a version of this where the document is a movie file. Data scientists use this kind of site for presenting reports or results from their analysis.
- **Utility Sites:** It is quite common for developers and system administrators to create utility scripts solving specific problems in an organization. Often these scripts are difficult to share as they're environment specific. Creating a web front-end for these kinds of utilities makes them useful to a wider audience within an organization. Similarly, most companies are share folders full of Excel files that are ad hoc solutions to a problem. Excel can be a fragile beast, there is seldom any input validation, and a small change in the wrong cell can produce the wrong result. Processes like this that are used by many people call

out for a web version, converting the logic in the Excel into a web application providing input validation and robustness.

Orthogonal to the type of site, is your architectural approach. Your project may use one or more of the following design concepts:

- **Single Page Applications:** Not so much a type of project, but a design approach, SPAs can be built with Django as their API back-end. Third party libraries like Django Ninja simplify the creation of these kinds of projects.
- **Mutli-User Sites:** Django includes user account management out-of-the-box. The Django Admin can be used to administer user accounts, and Django provides views to do login and password management.
- **Multi-Lingual Sites:** Django includes mechanisms for handling multiple written languages and localization such as date and currency format. This can be especially powerful when mixed in with multi-user capabilities, users see the site in their native language.
- **Static Page Sites:** For speed of interaction, nothing beats a static website, where the server feeds the user straight HTML. Of course, maintaining these kinds of sites can be painful unless you use tools that allow you to compose the page. The third-party library django-distill is a site generator: you build a Django site like you normally would then use distill to output it as a static site. This gives you all the advantages of the Django template engine while keeping the performance gain of a static site.

Books are great (you should buy multiple copies for your friends), but practice is where you really learn things. This book presents a project called *RiffMates*, where you help your cousin build a musicians-seeking-musicians personals site. This project is primarily the content site type, but it mixes in some document management aspects, all in a multi-user context. Coding along will help you better understand how to build your own projects. Each chapter includes exercises to test your progress, and sample code and answers are available online.

This book is divided into three sections. The first section covers the three core parts of Django, the second delves into Django's built-in tools, and the third covers how to use third-party libraries to build different kinds of sites.

In the first section you start to build *RiffMates*. You will write views and register them against URLs. You'll learn about the template engine and how to load and render templates in your views to output output HTML to the browser. You will also learn how to interact with the database doing queries on musician and venue data, while using the Django Admin to manage it all.

The second section shows you the power of Django and all the nifty stuff built into it. You'll turn *RiffMates* into a multi-user site and learn how to customize the provided views dealing with logins and password management. Once you've got user accounts happening, you add the ability to get input from your users, including uploaded files.

The third section deals with how to use some of Django deeper features as well as some third-party libraries. In addition to topics like localization and custom template tags, you'll see how to write other kinds of websites. Django was originally designed as a content management system framework, but the community has created all sorts of pluggable tools to do other things. You can build websites with REST APIs, deal with data import and export, and interact with popular JavaScript frameworks like HTMX.

Happy web coding!

1.3 Summary

- Django maps a URL to a view which is either a function or a class that returns a document to the browser.
- A Django view can render a document by composing pieces of HTML through a template engine.
- The Model-View-Controller architectural pattern divides the parts of a program into three pieces. The Model contains data and business logic, the View is for visualizing the data, and the Controller is for interacting with it.
- An Object Relational Mapping (ORM) is a way of abstracting database tables with programming classes.
- Django includes tools for managing multi-user web-sites.
- Single Page Applications (SPA) move the View and Controller parts of the MVC into the browser.

- Django can be used as the back-end for an SPA

2 Your first Django site

This chapter covers

- Creating a Django project
- Django projects and apps
- Creating a Django app
- Running the development server
- Writing a Django view

In this chapter, you'll start your Django coding journey by taking the first steps to building a website for musicians and bands. You'll create your first web page using the Django framework: a credits page showing who built the site.

2.1 The *RiffMates* project

Your cousin Nicky is a guitarist and she has a great idea for a website: classifieds for musicians. The site will cater to musicians looking for bands, bands looking for musicians, venues looking for bands, and bands looking for gigs. Nicky even has a name picked out, *RiffMates*. Over coffee, you and Nicky have talked at great length about what the site needs, including the following:

- Profile pages for musicians, bands, and venues
- Classified ad listings
- Announcement listings for try-outs
- Search capabilities based on instruments played and style of music
- Messaging between potential matches

Nicky has confidence in your coding skills because you've talked excitedly about the Python you've learned in the past. As you think about the problem, you realize that those features are just the business requirements. To build a working website that can do all those things, you'll also need features that are

common to most websites, such as:

- Public-facing and private-only web pages
- Logins
- Ability for users to submit both text and image content
- User sign-up
- Password management
- A database for storage
- Admin pages for managing the site

A lot goes into what seems like a simple website. Asking around you've heard that Django provides all those capabilities for you, and so you've decided to learn Django and build *RiffMates*.

Django is a *framework*. Programming with frameworks is different from programming with a library. In the case of a library, your code determines the order of execution. By contrast, when you're building inside a framework, execution control belongs to the framework. If you were building a house, a library would contain the pieces you needed for the house, whereas a framework is the rough outline of the house already. Working within a framework means your application designs are more constrained, but what you lose in flexibility you gain in efficiency: there is less code for you to write.

The partially completed building that you get with Django is the structure of a website. Django calls this a *project*. A Django project is what interacts with a web server to respond when a user visits a URL. Your code lives inside the project and gets invoked by Django. Each time you want to create a new site, you create a new project and you need to have the project in place before you can start adding pages to it. The rest of this chapter is about creating a project and writing your first web page inside it.

2.2 Creating a Django project

The Django framework includes a command-line tool that is used for creating new projects, called `django-admin`. (Not to be confused with the Django Admin, the web tool for managing your data). Django is published on PyPI

(https://pypi.org/) and can be installed in a Python virtual environment using the command `python -m pip install django`. Once you've got the package installed, you use the command `django-admin` to create a skeleton framework for your project.

Using virtual environments

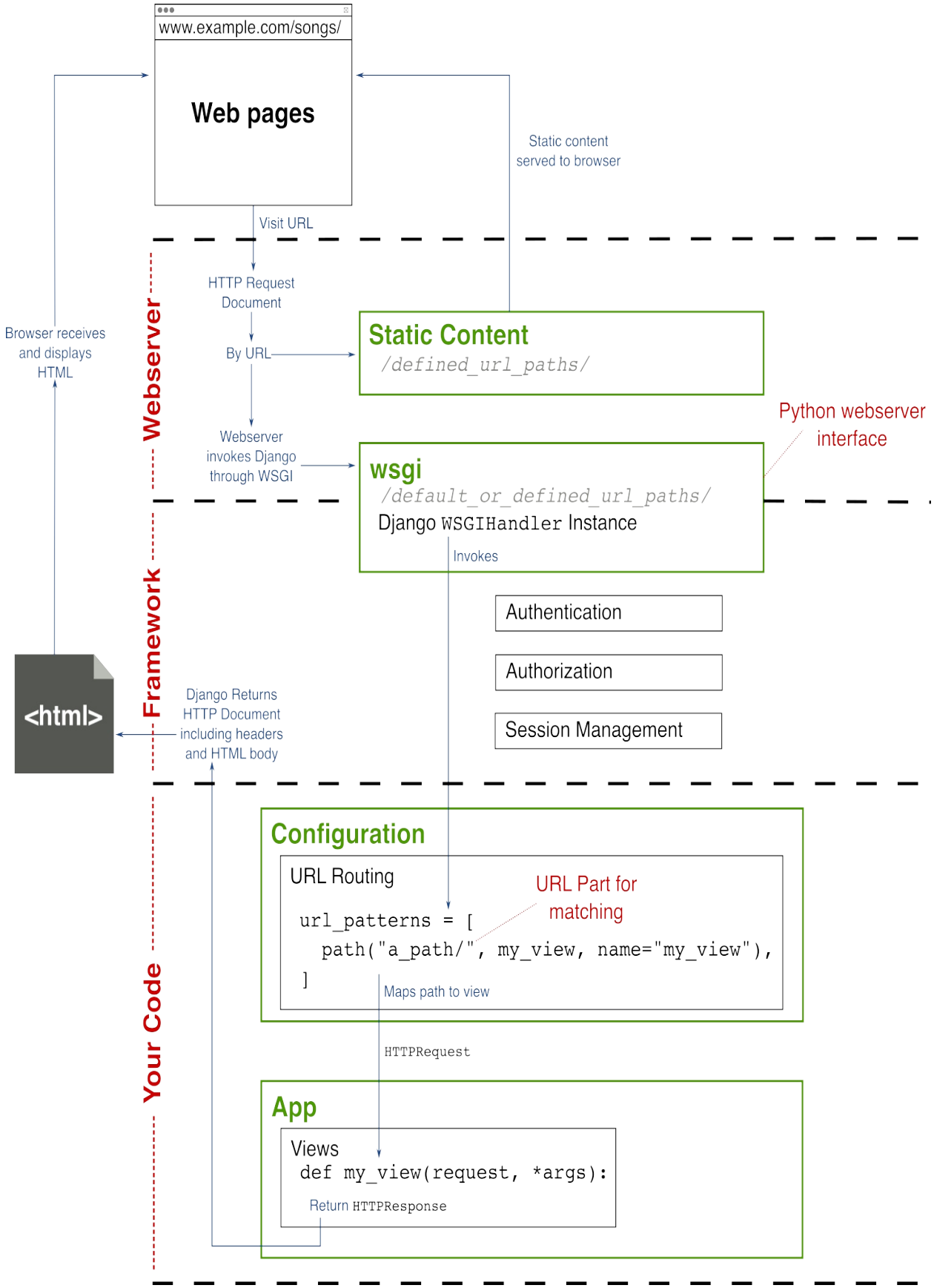
Each time you install a package in Python you're making it available to the entire system. As different projects have different needs, this can cause conflicts. To deal with this, Python uses *virtual environments*, which are self-contained Python setups which are isolated from each other. It is important to use a virtual environment when writing Python to avoid packaging conflicts.

Appendix A contains a brief overview of installing packages in Python virtual environments. For a full explanation on the technology, see: https://realpython.com/python-virtual-environments-a-primer/.

Each Django project is a Python program, but because it is built using a framework, the project provides the execution paths. When you create a new project, Django installs the bare minimum you need inside a new directory with the name that you give it.

At the heart of a Django project is a file named wsgi.py. WSGI stands for Web Server Gateway Interface, and it is a Python standard that specifies how code interacts with a web server. You really don't need to know much about this, especially when you're working in your development environment; the file gets generated for you. Inside wsgi.py is a short program responsible for interfacing between your code and the web server. When you're ready to put your project in production, you'll configure your web server to point at this application in your project. This is why Django is considered a framework rather than a library: wsgi.py is the entry point, and most of the code you write will be called through it.

Figure 2.1. A web server invokes Django through WSGI, Django calls your code, then formats the response for a browser


www.example.com/songs/
Web pages
Static content
served to browser
Visit URL
Webserver
HTTP Request
Document
By URL
Static Content
/defined_url_paths/
Browser receives
and displays
HTML
Python webserver
interface
Webserver
invokes Django
through WSGI
wsgi
/default_or_defined_url_paths/
Django WSGIHandler Instance
Framework
Invokes
Authentication
Authorization
Session Management
<html>
Django Returns
HTTP Document
including headers
and HTML body
Configuration
URL Routing
URL Part for
matching
url_patterns = [
path("a_path/", my_view, name="my_view"),
]
Maps path to view
HTTPRequest
Your Code
App
Views
def my_view(request, *args):
Return HTTPResponse

When you build *RiffMates*, or any other website, what you're really trying to do is make web pages. Each page lives at a URL and each time a user visits a URL you're going to want some code called to generate the page. In Django, this code is called a *view*.

When you load a page, or in some cases interact with one, your web browser is communicating with a web server. In the case of a Django project, the web server communicates with the WSGI application defined in wsgi.py. The WSGI application triggers the Django framework code. Django runs a view that is responsible for returning content that Django turns into an HTTP response for the browser.

Django needs to know what view to call for each URL on the website. This is done by mapping URLs to views. Within the framework a URL sometimes is known as a *route*. Each Django project has a main urls.py file that contains a list of URL mappings called `urlpatterns`. The list contains `django.urls.path` objects that specify a pattern that matches one or more URLs and what view to call when that URL gets visited.

The wsgi.py and urls.py files are key to your Django project. Every new website you build with Django requires its own copy of these files. To make your life easier, the `django-admin` command that comes with the package creates these files for you, customizing them based on your project name.

The `django-admin` program is actually the entry point for a number of configuration activities. Each activity gets invoked using a sub-command. To create a new Django project you'll be using the `startproject` sub-command and giving it the name of your project. By default, the `startproject` sub-command creates a new directory for your project, copies a script called `manage.py` inside, and creates a configuration directory. The `manage.py` script is similar to `django-admin`, running control activities for your project. The configuration directory contains wsgi.py, urls.py, and other files.

The configuration directory is confusingly named: it has the same name as your project. When you create *RiffMates*, you're going to get both a RiffMates directory and a RiffMates/RiffMates directory. It is unclear to me why the Django folks didn't name this something like "config". This causes

confusion when you're talking to someone else about the code. Are you in the RiffMates directory? "Yes." Which one?

Because your Django project is a directory itself, you have some flexibility about where you put your Python virtual environment. Some virtual environment tools prefer keeping all the environments together in a common directory, whereas others keep the environment with the project. In this latter case, you can put your virtual environment inside your project folder, which is what I'll be doing in the example here.

To create your first Django project, you're going to do the following:

- Create a virtual environment
- Install Django into a virtual environment
- Use the Django `django-admin` tool to create your *RiffMates* project

Once you've done all of this, you will be able to start the development server and point your browser at your new site. Open a command-line terminal and change to the directory where you normally keep your code. Inside the terminal: create a directory for your project and create a virtual environment inside of it:

```
$ mkdir RiffMates #1
$ cd RiffMates
RiffMates$ python3.11 -m venv ./venv  #2
```

 **Note**

Most of this book will be spent in Python, but to get going you need to run some command-line scripts. All command-line examples use Unix syntax with dollar-sign ($) denoting the prompt. The value before the dollar-sign indicates the current directory and whether a virtual environment has been activated. For details on how to install Unix compatible tools on Windows, see appendix A.

With the project directory and virtual environment in place the next step is to install Django. Activate the virtual environment and run `pip install`:

```
RiffMates$ source venv/bin/activate  #1
(venv) RiffMates$ python -m pip install django  #2
```

Django is dependent on a couple of other packages. The `pip install` command will install the dependencies. Your results will look something like this:

```
Collecting django
  Using cached Django-4.2.2-py3-none-any.whl (8.0 MB)
Collecting asgiref<4,>=3.6.0 (from django)
  Using cached asgiref-3.7.2-py3-none-any.whl (24 kB)
Collecting sqlparse>=0.3.1 (from django)
  Using cached sqlparse-0.4.4-py3-none-any.whl (41 kB)
Installing collected packages: sqlparse, asgiref, django
Successfully installed asgiref-3.7.2 django-4.2.2 sqlparse-0.4.4
```

Now that you have Django installed, the last step is actually creating your project. Run the `startproject` sub-command:

```
(venv) RiffMates$ django-admin startproject RiffMates . #1
```

Virtual environment placement and Django projects

The previous steps created the RiffMates project directory manually, it was done this way so that the virtual environment directory could be in the project directory.

By default, the `django-admin startproject` command creates your project directory for you. If you're using centralized virtual environments, you can skip the step creating the project directory and use `django-admin startproject RiffMates` without the period ('.') at the end, instead:

```
code$ source ~/wherever-your-RiffMates-venv-is/bin/activate
(RIFF) code$ pip install -m django
Collecting django
  Using cached Django-4.2.2-py3-none-any.whl (8.0 MB)
Collecting asgiref<4,>=3.6.0 (from django)
  Using cached asgiref-3.7.2-py3-none-any.whl (24 kB)
Collecting sqlparse>=0.3.1 (from django)
  Using cached sqlparse-0.4.4-py3-none-any.whl (41 kB)
Installing collected packages: sqlparse, asgiref, django
Successfully installed asgiref-3.7.2 django-4.2.2 sqlparse-0.4.4
(RIFF) code$ django-admin startproject RiffMates #1
```

```
(RIFF) code$ ls
RiffMates/
```

Running the `django-admin startproject` command creates a skeleton of your project. The new RiffMates project directory looks like this:

```
RiffMates/
├── RiffMates
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── manage.py
```

The project directory (RiffMates) contains the confusingly named configuration directory (RiffMates/RiffMates), the virtual environment, and the `manage.py` command. The `manage.py` command does pretty much the same things as `django-admin`, except it is connected to, and aware of, your project.

The RiffMates/RiffMates configuration directory contains the previously mentioned wsgi.py and urls.py files, along with a few others. Your project is a Python program, so the configuration directory is a module, meaning it contains an __init__.py file.

The asgi.py file is an asynchronous version of WSGI. Depending on the kind of web server you're using you might point to asgi.py instead of wsgi.py. More details on when these two files get used is covered in appendix B.

The final file in the configuration directory is settings.py, it contains configuration values for changing the behavior of Django. (More on that in a bit.)

You've created your first Django project, you actually have a website. A very small, rather boring website, but it exists. You haven't built any pages yet, but Django provides a default one for you, and it has a fancy rocket ship on it! To see the website and bask in the glory of the rocket ship, you need a web server. Django ships with one and when you're developing your site, this is the server to use.

You start the Django development server using the `manage.py` script and the `runserver` sub-command. Note that you do this inside the RiffMates project directory and not the RiffMates/RiffMates configuration directory! Start the Django development server by running the following command:

```
(venv) RiffMates$ python manage.py runserver
```

 **Note**

If you're on a Unix based system, the `manage.py` script is executable and you can shorten the command to `./manage.py runserver` to avoid some typing.

Once going, the development server shows you some debug info and then tells you its base URL. By default, the server listens on port 8000 of your local machine. The output from the server command looks like this:

```
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work pro
until you apply the migrations for app(s): admin, auth, contentty
sessions. Run 'python manage.py migrate' to apply them.

Django version 4.2.2, using settings 'RiffMates.settings'
Starting development server at http://127.0.0.1:8000/  #2
Quit the server with CONTROL-C.
```

Don't worry about the `18 unapplied migration(s)` message, you'll deal with it shortly. Remember that `localhost` is an alias to the IP address `127.0.0.1`, so you can also use the English alias `http://localhost:8000/` to visit your site. Enter the base URL into your browser and see the rocket ship on the default Django project page (figure 2.2).

Figure 2.2. Django Rocket Page

http://localhost:8000/

django

View release notes for Django 4.1

The install worked successfully! Congratulations!

You are seeing this page because DEBUG=True is in your settings file and you have not configured any URLs.

Django Documentation
Topics, references, & how-to's

Tutorial: A Polling App
Get started with Django

Django Community
Connect, get help, or contribute

Dev servers are for development

For convenience, Django includes a web server. This means you don't need to install and manage one when you're creating and maintaining your website. This is not a hardened web server though, and it should never be put into an untrusted environment.

Do not expose the Django development server directly to the internet, and if you're using it internally at a company, speak with your IT people about whether it should be hosted behind your firewall. Furthermore, the default configuration for a Django project is in debug mode, which also should not be used in production. Appendix B discusses what you need to know to put your Django project into a hardened production environment.

It is right there in the name: the **development server** should only be used for **development**!

At the moment, your website only responds to the base URL (/) because you haven't defined any views. Without views, Django shows you its version of "hello world": the Django rocket ship. It is a pretty small site, but maybe boring was under-selling it, rocket ships are fun!

As fun as rocket ships are, they have nothing to do with musicians. Nicky might be happy to see you've built something, but you're going to want to replace that ship with your own content. To do that, you'll have to write your own view. In Django, views go inside modules called *apps*.

Exit the Django development server with CTRL-C, and let's go create an app.

2.3 Projects versus apps

Each page in your website is likely to have its own view. I've mentioned the main urls.py file where routes get registered, mapping a URL pattern to code. In theory, you can write a view, or use a built-in one, right inside the urls.py file. Your entire website could be inside the files that the `startproject` sub-command created for you. In practice, this is a horrible way of organizing

things, and it would the file would quickly become unwieldy.

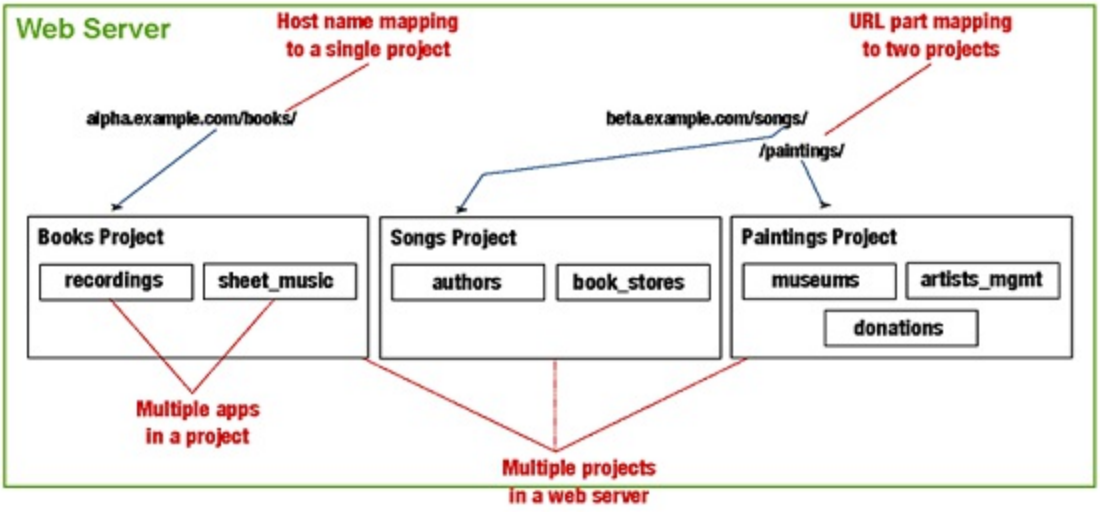
To help organize your code, Django has a concept built on top of Python modules called *apps*. This term got used by Django before it became the common name of programs on your phone. In Django's terminology, your site is a *project* and your modules are *apps*.

You need at least one app in your project. For smaller projects containing a single app, the name of that app isn't all that important. I often call mine "core." With larger projects that have multiple apps, you want to name your apps something that is descriptive of what they do. This is no different from a larger Python program containing multiple module directories.

Apps in projects in web servers

Even small sites can benefit by the organization principle of having several apps within a project. And although the most common configuration is for a single project in a web server, this isn't a requirement. A web server can host multiple Django projects, as long as it is clear which URLs goes to which project. This decision might be based on a host name in web servers configured to support multiple hosts, or based on part of the URL, with the first segment of the URL path pointing to different projects.

Figure 2.3. Multiple apps per project, multiple projects per server



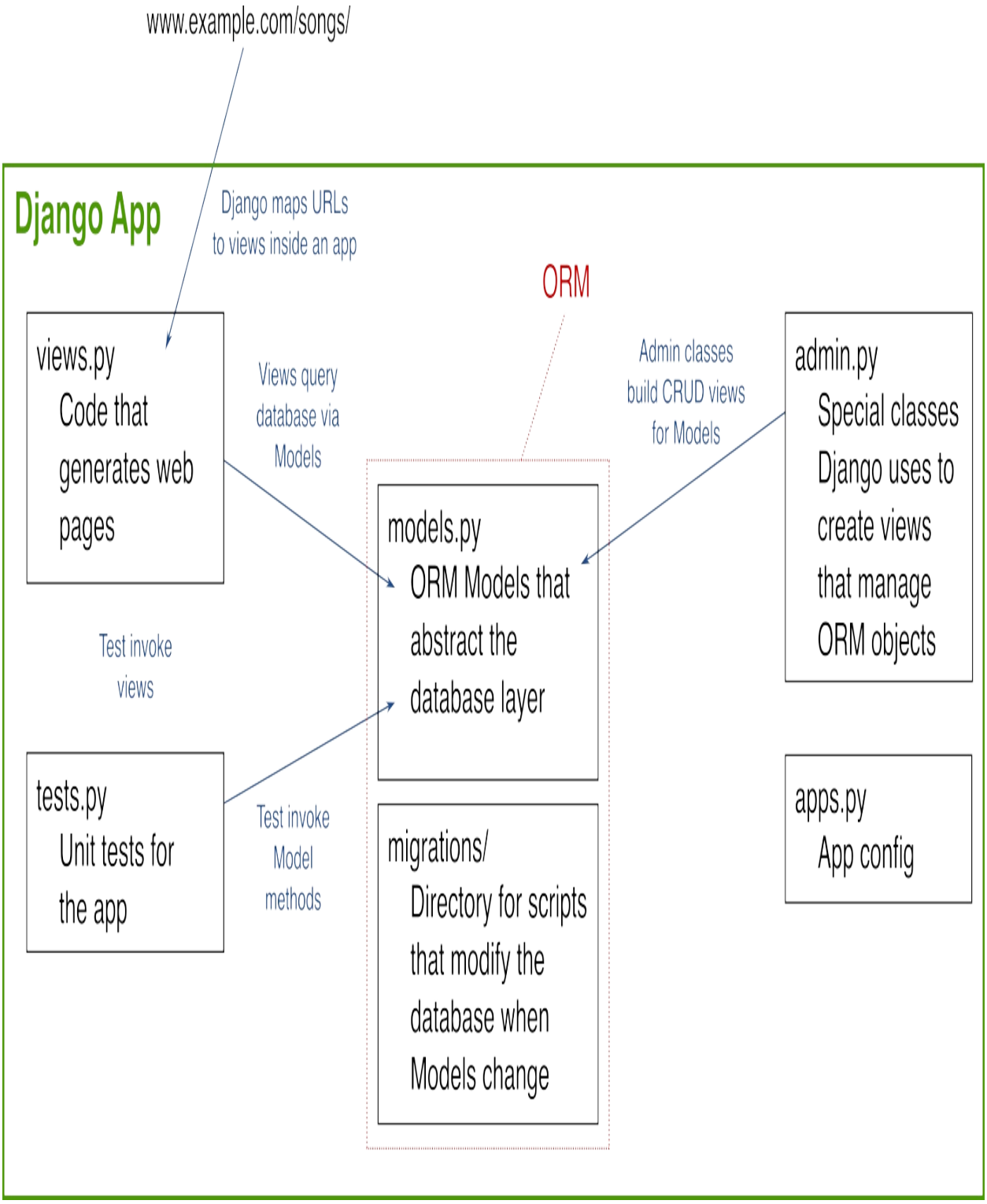
You should design your Django apps to be self-contained. The idea of an app even extends to third-party libraries. You can install apps written by other people and there are thousands of them available to you out there. How you organize your apps and how much they reference each other is up to you, but the structure is in place to help you stay organized.

Similar to creating a project, there is a sub-command for creating an app. When you create a new app, Django automatically creates skeleton files inside the app directory. The files have specific names that are expected by Django, they are:

- `apps.py`: app specific configuration
- `views.py`: code that outputs web page content
- `models.py`: code that defines database objects
- `tests.py`: unit tests for your app
- `admin.py`: code for managing your database objects
- `migrations/`: a folder containing scripts to manage the database

The apps.py file contains default configuration for the app which helps Django identify your directory as an app, and you'll likely never need to touch it. The other four Python files are stubs that contain an `import` statement for the Django module most commonly used in that type of file.

Figure 2.4. Anatomy of a Django app



Every time you create a new app, you get these files, but not every app needs all of them. If your app doesn't interact with the database, you don't need models.py, admin.py, or the migrations directory. If you're not a fan of tests,

you can remove tests.py, but if you do, somewhere a kitten will die.

You should always have tests. For the sake of the kittens, writing good tests for your Django project gets covered in a later chapter.

Your first app, containing your first view, will be named home and it will contain your project's Home page, About page, and other similar content.

2.4 Your first Django app

Normally when building software it is a good idea to build a Minimally Viable Product (MVP). You want feedback on your code as quickly as possible and the sooner you get in front of users the sooner you can adapt to their needs. Instead of taking an MVP approach, here you'll be taking an MUT approach. What's MUT? Besides something I just made up? It is *Minimal Understanding of Technology*. Instead of approaching your site with the best bit of customer functionality, here you'll be approaching based on the minimal amount of understanding you need to get things going. As such, the first web page you're going to build for *RiffMates* is the Credits page. Not terribly exciting, but it requires the least amount of work to build.

You've already got the RiffMates project in place. To write your first view, you'll need to add a Django app. The `manage.py` script in your project folder is the entry point to Django *management commands*. A management command is something you'd like to do with Django on the command-line. Django comes with many commands. You can even write your own—there's an entire chapter on how to do this!

To create an app in your project you use the `startapp` management sub-command. This sub-command creates a directory for your app and the needed stub files, including views.py where you will be writing your first view. Inside your project directory create the home app by running:

```
(venv) RiffMates$ python manage.py startapp home
```

The `startapp` sub-command is the strong silent type and returns without any response, it did do the work though. Your project directory tree now contains the home directory and the app skeleton files:

```
RiffMates/
├── RiffMates
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── home
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── migrations
│   │   └── __init__.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
└── manage.py
```

Your home app is a Python module, hence the __init__.py file. The other skeleton files for the app got created as well. Unfortunately, creating the app is not sufficient. You need to Django tell about it. The settings.py file in your RiffMates/RiffMates configuration directory is a Python script containing variable declarations that define Django's behavior.

The settings.py file comes with dozens of configuration values and you can add your own and access them through the `django.config.settings` module. To register your app, modify the `INSTALLED_APPS` configuration value which is a `list` of strings that are the names of all the apps in your project.

The developers of Django have consciously made a choice here: instead of auto-discovering all the modules and treating them like apps, you have to explicitly register an app. Although this is an extra step, it means Django doesn't have to differentiate between regular Python modules and Django apps, which are also modules. By doing it this way, your project can contain modules that aren't apps if you wish.

Django has its own apps that come with the framework. A new project has several of them are already included in `INSTALLED_APPS`. To register your newly created home app, open RiffMates/RiffMates/settings.py and add your app's name to the `list`, as in listing 2.1.

Listing 2.1. Tell Django about your `home` app

```
# RiffMates/RiffMates/settings.py
...
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'home', #1
]
...
```

Django is now aware of your app. You're ready to add a view to show your Credits page.

2.5 Your first Django view

Each view is responsible for creating content that Django returns when the view's corresponding URL gets visited. Django provides some built-in views, but most of the time you're going to want your own. You can write views as either functions or classes. I prefer function-based views as I find it clearer to understand a URL calling a function, rather than it corresponding to one or more methods in a class. I'll be sticking with function based views throughout the book.

Recall that when you visit a URL, your web browser is opening up a network connection to a web server. It know which web server based on the host portion of the URL (known as the domain). The language that the browser speaks to the web server using is the HTTP protocol, essentially a set of rules for how the browser and web server communicate. The HTTP protocol is split into two parts: the request from the browser to the server, and the response from the server to the browser. Django encapsulates both of these in objects helpfully named `django.http.HTTPRequest` and `django.http.HTTPResponse`.

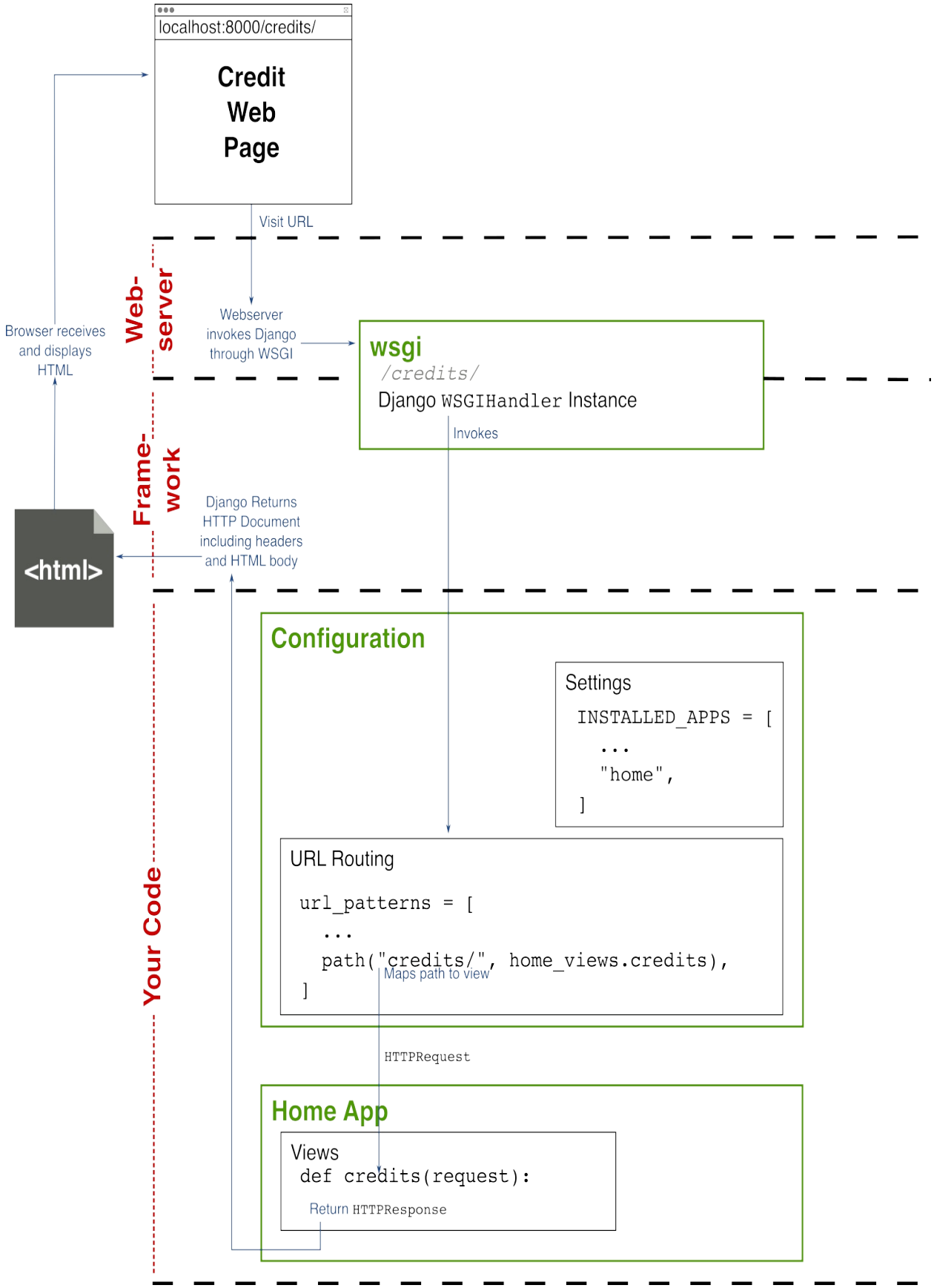
A function based view has two requirements: it takes at least one argument, an `HTTPRequst` object, and returns an `HTTPResponse` data structure (or

something like it). Django uses the `HTTPResponse` to produce the response document including headers and HTML body for your browser.

By convention, the `HTTPRequest` argument gets called `request` and contains many of the headers your browser sent the web server, including the URL. Your view function can take more arguments than just the request. Django supports a way of specifying multiple parameters inside a URL and mapping these to your function arguments. It also supports query arguments—that's everything after the question-mark (?) in a URL. Query arguments get embedded in the request object. You'll learn more about both these techniques in future chapters.

To keep it simple, your credits web page will contain your name and Nicky's, and not much else. To keep it really simple, it will be plain text rather than HTML. Your view is only a few of lines of code: a function that takes a request object, declares a string, and returns an `HTTPResponse` object.

Figure 2.5. The `credits` view within your *RiffMates* Django project


localhost:8000/credits/
Credit Web Page
Visit URL
Web-server
Webserver invokes Django through WSGI
Browser receives and displays HTML
wsgi
/credits/
Django WSGIHandler Instance
Invokes
Frame-work
Django Returns HTTP Document including headers and HTML body
<html>
Configuration
Settings
INSTALLED_APPS = [
...
"home",
]
URL Routing
url_patterns = [
...
path("credits/", home_views.credits),
]
Maps path to view
HTTPRequest
Home App
Views
def credits(request):
Return HTTPResponse
Your Code

The `HTTPResponse` you return needs two arguments: the content to return, and the type of that content. The content in this case is a string containing your credits. The second argument is `content_type` and it changes the MIME-type of the response.

Web browsers are capable of displaying all kinds of different content: HTML, images, CSS, and more. An HTTP response can contain a MIME-type header that indicates just what the content is. MIME stands for *Multipurpose Internet Mail Extensions*, and as the name implies, originally came about to define how to attach different kinds of content to an email message. By default, the `HTTPResponse` object uses a MIME type that indicates HTML, as the credits page returns plain text, the `HTTPResponse` object needs to be told this by setting `content_type="text/plain"`.

To write your first view, open `RiffMates/home/views.py` and replace the stub code with the content of listing 2.2.

Listing 2.2. Credits view

```
# RiffMates/home/views.py
from django.http import HttpResponse #1


def credits(request): #2
    content = "Nicky\nYour Name" #3

    return HttpResponse(content, content_type="text/plain") #4
```

You're not quite done yet. Although you have a view, you haven't mapped it to a URL. You do that by modifying urls.py.

2.6 Registering a route

When a user visits a URL controlled by Django, the web server calls the WSGI application defined in wsgi.py with the HTTP request information. Django uses its list of URL patterns defined in urls.py and tries to find a match. If a match gets found, the view associated with the URL pattern gets called. You have defined the home app with the `credits()` view in its

views.py file, now you need to register a URL route with that view.

The URL mappings in urls.py are contained in a list called `urlpatterns`. You define a mapping using a `path` object. When you used the `startproject` sub-command to create your project, a bare-bones urls.py file got created. That generated file includes the import for `path` objects. Each `path` object requires at least two arguments: a URL pattern and a mapping. The mapping in this case is a reference to your `credits()` function. Mappings can also be references to other mapping files giving the ability to create hierarchies of maps.

Your path object requires an actual reference to the `credits()` view, yes, the callable view function itself. Importing every view in your project would take up a lot of space, so it is common practice to import the view module and then dot-reference the function inside.

RiffMates/RiffMates/urls.py is the central place for all URL definitions. All apps register routes in this file, and because every app names its view file views.py, this can get confusing. To make your urls.py clearer, use the `import … as` pattern to rename your imported module. For example, instead of using `from home import views`, you rename the imported module as `from home import views as home_views`.

Django includes a management tool called the Django Admin—not to be confused with the `django-admin` command that is completely unrelated. By default, the `urlpatterns` list includes a mapping for the Django Admin. When you register your credits view, you'll be adding this along with the existing mapping. To do this, open RiffMates/RiffMates/urls.py and change it to look like listing 2.3. A future chapter covers the Django Admin in detail.

Listing 2.3. Register your view

```
# RiffMates/RiffMates/urls.py
from django.contrib import admin

from django.urls import path

from home import views as home_views
```

```
urlpatterns = [
    path("admin/", admin.site.urls),
    path('credits/', home_views.credits),
]
```

Note the lack of a leading slash (/) in the `path` pattern. Django prefixes the leading slash for you. The URL `/credits/` is registered using `path("credits/")`. To register that URL sub-path against your view, you pass a reference to the `credits` function to the `path` object. Remember, as it is a reference to a function you aren't calling it; don't use parenthesis, it is `credits`, not `credits()`.

In this code, I import the module and use a dotted reference to the view: `home_views.credits`. Instead, you could import the view from the module, but as you add more views to home/views.py that would mean a lot of importing.

Other ways of specifying paths

Older versions of Django provided a different mechanism for specifying the structure of a URL to register against that looked like a regular expression. Some programmers still prefer the power of regular expressions when registering routes, so `re_path` exists if you prefer that style, or have a particularly fancy route that isn't covered by `path`. I recommend switching the to newer style, it is generally easier to read and use.

For more information on `re_path` see:
https://docs.djangoproject.com/en/4.2/ref/urls/#django.urls.re_path

2.7 Viewing your view

You're almost ready to go. The last time you ran the development server you got a warning message: `18 unapplied migrations`. Django tracks when you make changes to database models and creates special scripts called migrations for making database changes. The `unapplied migrations` are scripts for built-in apps like the Django Admin. In later chapters you'll write your own database objects, but for now, you can run a migration to get rid of the warning.

The management sub-command to run database migrations is `migrate`. Before running the development server, run `python manage.py migrate` to get rid of the `unapplied migrations` warning:

```
(venv) RiffMates$ python manage.py migrate
```

Running `migrate` produces a lot of output, you will see a list of the apps that required migration (`Apply all migrations: admin, auth, contenttypes, sessions`), along with the output from each migration app. The output will include the full name of the app and migration script. For example, `Applying admin.0001_initial… OK` indicates that script `0001_initial` for the `admin` app was run successfully.

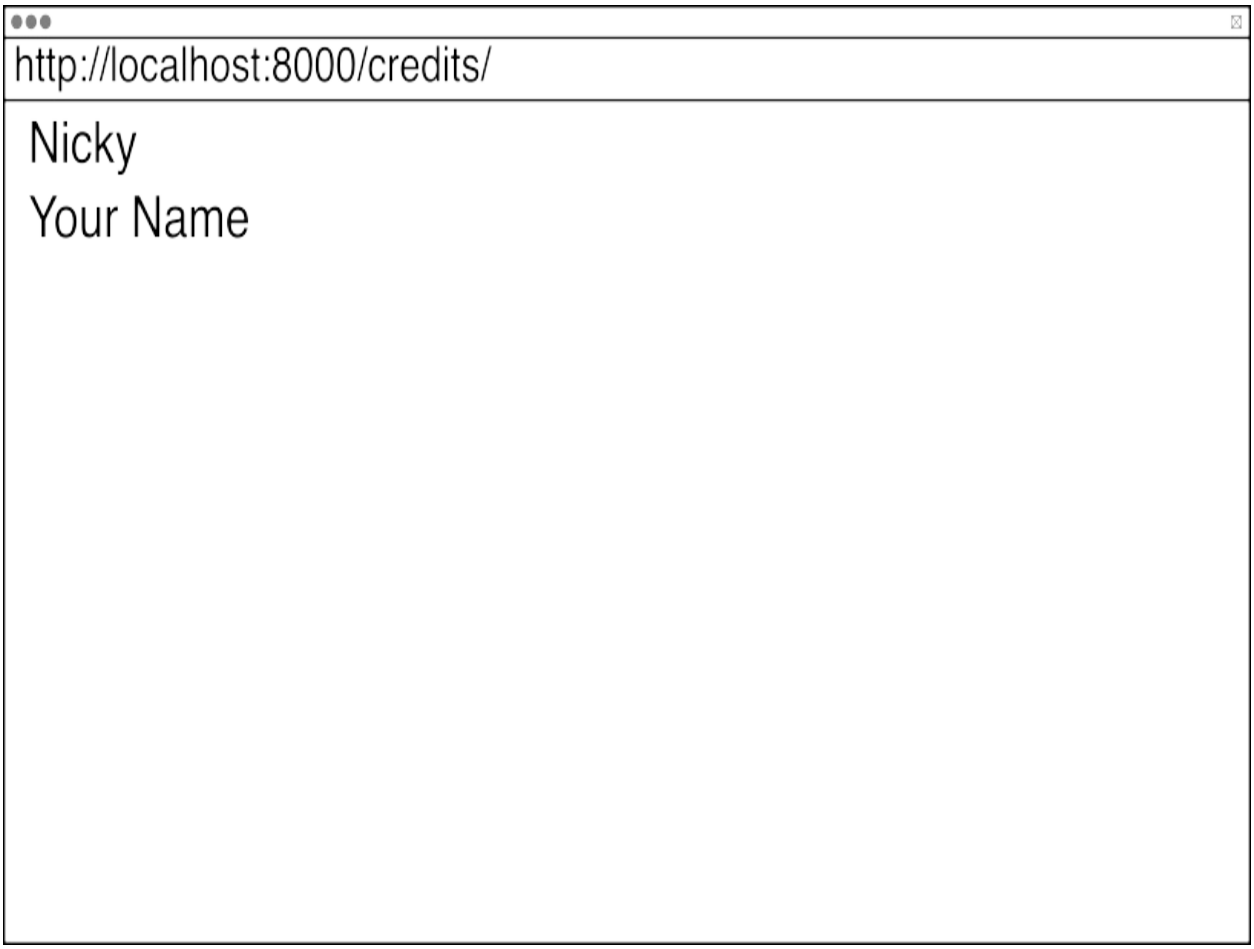
The `migrate` command applies migration scripts across several modules. You'll learn all about this in detail in a future chapter.

Re-start the development server, this time you won't get the warning:

```
(venv) RiffMates$ python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

Django version 4.2.2, using settings 'RiffMates.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

With the development server running, you can see the results of your hard work by visiting: `http:localhost:8000/credits/`. If you've wired everything together correctly, it should look like figure 2.6.

Figure 2.6. Your first Django view



When you're done admiring your accomplishment, press CTRL-C to exit the development server.

You've got your first project built and your first view written. You have enough information to build an entire website. Everything else is about how to make fancier and fancier views, but the corner stone has been laid. Each page you add to your site will be another view. Instead of your view returning plain text, you can return HTML. HTML is verbose. Because of this, Django includes tools to manage it more like code. You can write re-usable parts and compose them together. Doing that is the next step.

2.8 Exercises

For a bit of practice, try to adapt your code to do the following:

1. Add an About page to the site using the same techniques as you did for

the credits page. Instead of using plain text, have the content block contain valid HTML content.

2. Add a Version Info page that contains the version number of your site. Instead of using plain text return valid JSON. You can do this by adapting the MIME type, or using a `django.http.JsonResponse` object instead. Details on the `JsonResponse` object can be found in the documentation: https://docs.djangoproject.com/en/4.2/ref/request-response/#jsonresponse-objects

2.9 Summary

- Django is a framework rather than a library meaning your code gets called by Django rather than the other way around.
- Django calls a website a *project*.
- You create a Django project using the `django-admin startproject` command. This creates skeleton files for your website.
- Django projects contain a configuration directory that has the same name as the project directory. This directory includes the URL mapping file (urls.py) and a script that specifies configuration settings (settings.py).
- The code for your website goes in modules that Django calls *apps*.
- You create a new app using the `python manage.py startapp` command. This creates skeleton files for your app.
- Your app must be registered by adding it to the `INSTALLED_APPS` value in settings.py.
- URLs get mapped to functions, known as *views*, responsible for returning web content.
- To write a view: **Add a function to the views.py file in an app.** The function takes at least one argument, a `HTTPRequest` object named `request` by convention. **The function returns an `HTTPResponse` object specifying the content returned to the browser.** Your view gets registered in urls.py using a `path` object to map a URL to a reference to the view function.

3 Templates

This chapter covers

- Managing HTML like code through templates
- Rendering templates based on context data
- Template tags and filters
- Using the `render()` shortcut inside of views
- Composing and inheriting templates for HTML reuse

This chapter introduces you to adding HTML content to your website. Django includes a templating engine that can compose and control text content, allowing you to create reusable pieces of HTML, managing it like you would manage your code.

3.1 Where to use templates

In the previous chapter you learned how to start a simple website by creating a Django project. Inside the project you created your first app and your first view. To keep things simple an important thing got left out: HTML. A website isn't much of a website without it. The purpose of a Django view is to return content, typically for a single web page. Your first view was the Credits page, showing the website's authors. This view used plain text. Instead of returning plain text most of your views will return HTML instead.

The Django Template Engine is a collection of tools that allow you to compose and control text in way that is similar to object-oriented programming. A header or footer can be declared in a separate file and then included in each page that is rendered. This isolates the text and any changes automatically get propagated throughout your site.

On a web page with a navigation menu you might want to highlight the current page. Instead of different header blocks for each page, Django includes conditional rendering. It supports the equivalent of Python `if` and

`else` statements, allowing you to change the rendering of the navigation bar based on Python objects passed as arguments to the template engine.

 **Note**

The template engine is not HTML specific, it operates on any text. The vast majority of the time you will be using it to manage HTML, but you can also templatize email messages, personalize pre-filled text in forms, or any other text manipulation that you come up with.

The interface to the template engine is two key objects: `Template` and `Context`. Both are found in the `django.template` module. The `Template` object represents a piece of text written in the template language, while the `Context` object provides data to the template engine for rendering the result.

3.2 `Context` objects and the parts of a `Template`

Using the Django Template Engine breaks down into three steps:

1. Create or load a template
2. Define context
3. Render a template using the context

When you create or load a template you get a `django.template.Template` object as a result. The contents of this object are actually a compiled version of the text you want in the template. To get the output from the template you need to render it. Rendering requires a `django.template.Context` object that contains any data used in the template. For example, if the template is for a navigation menu and you want to highlight the current page, the context might contain a variable indicating the name of the current page. Inside the template a conditional block checks the value of the current page variable and changes the rendering of the menu as needed. Pieces of templates can be composed together, so often a template is part of a page—it doesn’t have to be the whole thing.

`Context` objects get built using dictionaries. Anything you can put inside a

Python dictionary can be used as context. If the template references something that is not found within the context, it is rendered as an empty string. The effect of this is that errors are handled silently. This is both a feature and a frustration. Blank spots on the page are less problematic for your users, but it may be a bit trickier to find bugs as the evidence is sneakily silent.

The simplest possible template is text containing no special characters. When rendered, you will get exactly the text you gave the template. What you put in is what you get out. This is of limited value. To have your template change based on the context, Django provides four constructs:

1. Variable rendering through matched double braces: `{{ name }}`
2. Script actions like conditionals and loops using tags: `{% now "Y" %}`
3. Comments: `{# a comment #}`
4. Output modification through filtering data: `{{ name | upper }}`

A set of matched braces, sometimes known as mustache braces, gets used to display a variable. The name given inside the braces gets used as a key in the context dictionary and replaced with the corresponding value.

A variable can be referenced using dot-notation to get at the contents of an object. Consider a `Person` class with a `last_name` attribute. If the context contained an object named `person`, then `{{ person.last_name }}` uses the person object and dereferences the `last_name` attribute on that object.

Tags in the template language get used to perform logic actions inside the template. Some tags get used on their own, and some are paired. The `{% now %}` tag renders as the current date. This tag takes a parameter that indicates the format of the date. Rendering `{% now "Y" %}` results in the current year.

Paired tags typically control the rendering of the block between the tags. One such pair is `{% if %}` and `{% endif %}` which surround a block that is conditionally rendered. The `{% if %}` tag takes a condition to be evaluated as an argument and behaves similarly to the `if` statement in Python. For example, `{% if num > 3 %} Lots {% endif %}` renders `Lots` if the key `num` exists in the context dictionary, is a number, and has a value larger than three. A selection of common tags get covered later in this chapter.

Filters are a way of modifying data in a variable. Remember, variables get displayed through paired double-braces. You apply a filter to a variable by using the pipe (|) operator. For example, `{{ name | upper }}` applies the `| upper` filter to the value contained within `name`. The `| upper` filter is similar to `str.upper()`, returning the contents of `name` in capital letters. Several filters and their use get covered later in this chapter.

There are over seventy tags and filters built into the Django template language. Appendix C is a glossary of the tags and filters built into Django and points you at the online documentation. The template language is also extensible: you can write your own tags and filters. Writing your own tags and filters gets covered in a later chapter. Many people have written their own and shared them as third-party libraries that you can install as Django apps. The Django Packages website has a listing dedicated to just template tag packages: https://djangopackages.org/grids/g/templatetags/.

3.3 Django `shell`

As templates are typically rendered through a view, learning and experimenting with them might mean having to write a lot of throw-away code. Instead of creating a dummy view, you can use the Python REPL. If you haven't come across the REPL yet, it's built into Python and allows you to immediately see the results of a line of code. REPL is short for *Read, Evaluate, Print, Loop*. In the REPL you type some code (the tool *Reads* it), the tool *Evaluates* it, then *Prints* the results, and *Loops* to do it all over again.

For regular Python you enter the REPL by running `python` without any arguments. With Django being a framework, you can't just fire up the Python REPL within a Django context. There are several steps to be performed to get you in Django-land, including importing and activating the framework. Luckily, Django provides a sub-command so you don't have to do this by hand. Instead of calling `python`, you call the `manage.py` sub-command named `shell`:

```
(venv) RiffMates$ python manage.py shell
```

When you run the command you get a regular Python REPL, except it is

Django-aware. Because Django is a framework, if you used the default Python REPL on its own, you'd have to run some commands to get it to load and understand the Django environment, instead the `shell` sub-command takes care of this for you. When started, the command reports version information and then sends you to the `>>>` prompt, awaiting your instructions:

```
(venv) RiffMates$ python manage.py shell
Python 3.11.1 [Clang 13.0.0 (clang-1300.0.29.30)] on darwin
Type "help", "copyright", "credits" or "license" for more informa
(InteractiveConsole)
>>>
```

If you need to experiment with some Django code, using the `shell` sub-command to enter the REPL is less work than trying to implement your idea in a view. It is also less error-prone, you don't have to remember to remove your experiment afterwards. Now, let's use the `shell` to play with some templates.

 **Note**

The REPL supports the use of up-arrow to retrieve previous commands. This can save a lot of typing if you're doing repetitive work. If you are using the REPL a lot, you may want to search for third party replacements such as `bpython`, `IPython`, and `ptpython` that include advanced code editing tools and macro capabilities.

3.4 Rendering a `Template`

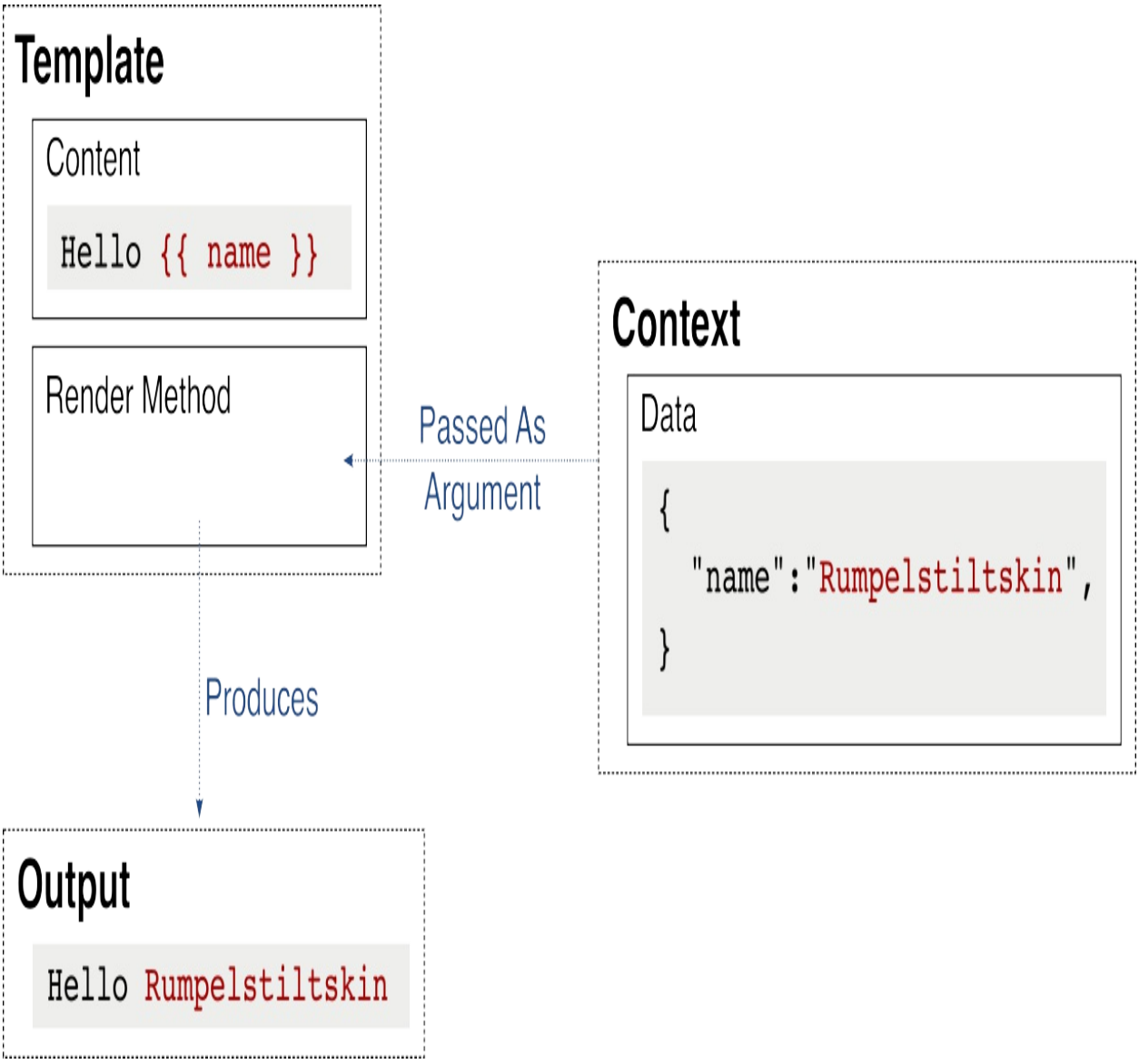
`Template` objects are, well, objects. You can instantiate them like any other object. In practice, you're more likely to use a shortcut that loads a file containing the template content, but let's deal with the objects directly first to better understand how they work. You'll learn about using shortcuts to load and render template content from file a little later in the chapter.

`Context` objects contain the data that is used by the template when it is rendered. They take a dictionary containing the data you want in the context as an argument. To render a template you call `Template.render()` and it

returns the resulting string.

In keeping with a grand programming tradition, your first template is "hello world". Figure 3.1 illustrates the concept. A `Template` object is instantiated with a template string: `Hello {{ name }}`, a `Context` object is created with a dictionary with the key `name`, and the `Template` object's `.render()` method is called, with the `Context` as an argument. The result is the rendered template.

Figure 3.1. The `render()` shortcut saves you code



Double-braces indicate variable replacement in a template. In this example,

the `{{ name }}` causes the template engine to look for a key called `name` in the `Context`. The contents of the corresponding value are used to replace the double-brace content in the template.

Let's play with this in the Django REPL. Use the `shell` management sub-command like you did earlier to start a new session. Once in the REPL, import the `Template` and `Context` from `django.template`. Instantiate a `Template` object, passing in `"Hello {{ name }}"` as the template to be rendered. This is the same as in figure 3.1. Now, instantiate a `Context` object passing in a `dict` with key `name` and a corresponding value for your name. Call the `.render()` method on the `Template` object, passing in your `Context` object. This will render the template, returning the output string. Listing 3.1 shows just that.

Listing 3.1. Rendering a template in the Django REPL

```
>>> from django.template import Template, Context
>>> t = Template("Hello {{name}}")  #1
>>> c = Context({"name":"Rumpelstiltskin"}) #2
>>> t.render(c) #3
'Hello Rumpelstiltskin' #3
```

The template language allows you to access the attributes on objects, items in lists and tuples, and the values in dictionaries. All of these are done using a dot-notation, even those parts that would require square-brackets to access in Python. For example: `{{ person.name }}` works for both a `person` object and a `person` dictionary, accessing the `name` attribute or key, respectively. The third item in a list of fruit is accessed using `{{ fruit.2 }}`.

Dot-notation is also used for calleables without arguments. For example: `{{ person.show_name }}` will render as the result of the `person.show_name()` method.

In addition to doing variable replacement with double-braces, you can use template tags to control the content of your rendered result. The most common tags are conditional blocks and loops, but some are for straight replacement as well. The `{% lorem %}` tag gets rendered as *lorem ipsum* text, a chunk of Latin often used in the printing industry to indicate placeholder text. In your web site, you might use the `{% lorem %}` tag when you're

waiting on someone else to write some copy, drop the tag in so the page renders with some content taking up space on the page instead of just leaving the area blank. This may not be the most popular tag, but it is a good example to learn how tags work.

In the REPL, create a new `Template` object that uses the `{% lorem %}` tag. As your template doesn't have any data, use an empty dictionary to instantiate the `Context`. Listing 3.2 contains the code and the resulting Latin.

Listing 3.2. Using the `{% lorem %}` tag in the Django REPL

```
>>> t = Template("Cicero said: '{% lorem %}', amongst other thing
>>> t.render(Context({}))
"Cicero said: 'Lorem ipsum dolor sit amet, consectetur
adipisicing elit, sed do eiusmod tempor incididunt ut labore et
dolore magna aliqua. Ut enim ad minim veniam, quis nostrud
exercitation ullamco laboris nisi ut aliquip ex ea commodo
consequat. Duis aute irure dolor in reprehenderit in voluptate
velit esse cillum dolore eu fugiat nulla pariatur. Excepteur
sint occaecat cupidatat non proident, sunt in culpa qui officia
deserunt mollit anim id est laborum.', amongst other things"
```

Like many tags, `{% lorem %}` takes optional arguments allowing you to control how much text results. The first two arguments are a count and a method. The method can be `w`, `p`, or `b`, for "words", "paragraphs", or "blocks". The value of count tells you how many of the words, paragraphs, or blocks to include. Render another template in the REPL, this time telling `{% lorem %}` to display five words. Listing 3.3 shows the code and resulting, much shorter, text.

Listing 3.3. The `{% lorem %}` tag with arguments in the Django REPL

```
>>> t = Template("{% lorem 5 w %}")
>>> t.render(Context({}))
'lorem ipsum dolor sit amet'
```

Tags are kind of like functions and like functions the choice of arguments are specific to the tag. Some tags interpret their arguments as the names of variables, while others, like `{% lorem %}`, take them verbatim. You'll see the difference as you learn a few of the more common tags in the next section.

3.5 Common tags and filters

Although Django comes with many tags and filters, there are a few that you will find you use over and over again. Let's cover a few that you're likely to use while building *RiffMates*.

3.5.1 Conditional blocks

It was probably early on in your programming journey that you first came across the need for if-statements. You'll find the same need in templates. The basic conditional statement is a block, which means it uses paired tags: `{% if %}` and `{% endif %}`.

The `{% if %}` tag takes a condition, which if true means the block gets rendered. The simplest condition is the name of a variable. In this case the variable gets evaluated for "truthiness" following the same rules as if-statements in Python. The following renders the `<h1>` content if `first_page` evaluates to `True`:

```
{% if first_page %}
    <h1> Welcome to <i>RiffMates</i> </h1>
{% endif %}
```

If `first_page` contains zero, `False`, an empty container (`list`, `set`, `dict`, etc.), or doesn't exist in the context, the block gets excluded from the render. All other values of `first_page` result in the enclosing block getting shown.

Block tags generally don't care about newlines or spacing. The `{% if %}` and `{% endif %}` tags can appear on the same line. I frequently have a conditional block in my `<title>` tag, optionally appending a page name to a default shorter title:

```
<title>RiffMates{% if title_suffix %} &mdash; {{title_suffix}} {%
```

The `{% if %}` tag supports the same comparison operators as Python. You can use `==`, `!=`, `<`, `>`, `<=`, `>=`, `in`, `not in`, `is`, and `is not` to compare variables and/or values. Comparing the value of `num_drummers` to a constant is similar to Python:

```
{% if num_drummers > 1 %}
    <b>You only need one drummer!</b>
{% endif %}
```

When the variable in a comparison is not in the context, the condition gets treated as `False`. If `num_drummers` is not in the context in the preceding example, the comparison fails, the block does not get shown.

Conditional clauses can be combined using boolean operators like those in Python. You can use `and`, `or`, and `not` in an `{% if %}` tag. The same order of precedence as Python gets applied. If `and` and `or` are in the same clause, `and` gets higher precedence. The following checks for the "truthiness" of `musicians` and compares `num_venues` to three:

```
{% if musicians and num_venues > 3 %}
    You have lots of choice!
{% endif %}
```

More complex conditional blocks can be constructed by adding `{% elif %}` and `{% else %}` tags to your conditional. These tags behave like their Python counterparts. As you might expect, the comparison clauses that work in `{% if %}` work in `{% elif %}` as well. The following checks for `num_musicians` being greater than zero and less than six, and then if that condition fails it checks if `num_musicians` is greater than or equal to six, and if that condition fails you are told that you have zero musicians:

```
{% if num_musicians > 0 and num_musicians < 6 %}
    Band sized
{% elif num_musicians >= 6 %}
    Big-band sized
{% else %}
    Your band needs people
{% endif %}
```

3.5.2 Looping blocks

Consider the case where you have a list of instruments that you want to output in an HTML bullet list. To do this cleanly, you need to loop over your data. The `{% for %}` tag is similar to a `for` statement in Python, allowing you to iterate over a container. Just like Python, the tag takes the name of a

variable used inside the looping context and an object to iterate over. The value in the looping context gets used like any other variable, often displayed using double-braces.

One use of the `{% for %}` tag is to construct `<ul>`, `<ol>`, or `<table>` structures in HTML. In all these cases the `{% for %}` tag goes inside the containing structure, with the contents of the `{% for %}` block being what you want to be repeated. The following code outputs a `<ul>` tag with its insides constructed by looping over the `instruments` value, rendering `<li>` tags:

```
<ul>
    {% for instrument in instruments %}
        <li> {{instrument.name}}: {{instrument.cost}}<li>
    {% endfor %}
</ul>
```

Remember that variable rendering supports dot-notation. For the preceding to work, `instruments` must be iterable and contain objects that have `name` and `cost` attributes.

If the iterable given to `{% for %}` is empty, nothing in the block will be displayed. Depending on your situation this might result in an empty list or table. You can either wrap the enclosing HTML in an `{% if %}` block, or you can use the optional `{% empty %}` feature of the loop. Anything within the `{% empty %}` section of a `{% for %}` tag only gets rendered if there was nothing in the iterable being looped over:

```
<ul>
    {% for instrument in instruments %}
        <li> {{instrument.name}}: {{instrument.cost}}<li>
    {% empty %}
        <li> <i>No instruments found</i> </li>
    {% endfor %}
</ul>
```

In addition to the loop variable that you define in `{% for %}`, there is an implicit variable as well: `forloop`. This is an object that contains attributes with information about the state of the loop as it is being iterated. The `forloop.counter` attribute is a 1-based count of the loop iteration. The

boolean values `forloop.first` and `forloop.last` indicate whether it is the first or last iteration of the loop, respectively. You can turn the previous example's `<ul>` into sentence instead:

```
My favorite instruments are:
{% for instrument in instruments %}

    {% if forloop.last %} and {% endif %}

    {{ forloop.counter }}. {{instrument.name}}

    {% if forloop.last %}. {% else %}, {% endif %}
{% endfor %}
```

I've put extra newlines and indentation in this example to make the template readable, for HTML this wouldn't matter. If `instruments` contained two objects named "Bassoon" and "Oboe", the result (stripped of newlines and extra spaces), would be:

```
My favorite instruments are: 1. Bassoon, and 2. Oboe.
```

All the `forloop` attributes

Here is a complete list of the attributes available in the internal `forloop` object:

- `forloop.counter`: the iteration count of the loop, starting at 1
- `forloop.counter0`: the iteration count of the loop, starting at 0
- `forloop.revcounter`: the number of iterations remaining in the loop, with the last being 1
- `forloop.revcounter0`: the number of iterations remaining in the loop, with the last being 0
- `forloop.first`: a boolean that is `True` for the first iteration
- `forloop.last`: a boolean that is `True` for the last iteration
- `forloop.parentloop`: in the case of nested loops, this is the `forloop` value for the surrounding loop

3.5.3 Comment blocks

There are two ways of writing a comment in the Django template language:

inline and a multi-line tag. Inline comments use matching {# and #} braces. Multi-line comments use a {% comment %} block tag. The multi-line form is useful for temporarily commenting out parts of a template while you are debugging. This code has an inline comment beside "Guitar" and has commented out the "Oboe" and "Bassoon" tags:

```
<ul>
    <li> Guitar {# cool instrument #} </li>
    <li> Drums <li>
    {% comment "For orchestras only" %}
    <li> Oboe </li>
    <li> Bassoon </li>
    {% endcomment %}
</ul>
```

When the preceding HTML gets rendered, the opinion next to “Guitar” is not shown. Likewise, neither of the symphony instruments appear in the list. The {% comment %} tag takes an optional argument of a string that is used as a note about the comment. Yes, you can comment your comment. It might seem silly, but if you are commenting out a large block of a template it is sometimes helpful to leave yourself a note reminding you why.

3.5.4 Verbatim blocks

One of the things coders like to write about is coding. A lot of characters that are meaningful to HTML are also characters you will find in your code. This can make for some messy HTML. Writing a Python if-statement with comparison operators in a web page requires entities like > and < to display properly.

This problem is even worse if you’re writing a website about Django or JavaScript templating. Both of these cases use characters that are special to HTML or special to the Django Template Engine itself. There is a tag that solves this: {% verbatim %}. This block tells the template engine to render everything contained within exactly how it is. You can even name a {% verbatim %} block so that you can wrap {% verbatim %} blocks. The following code contains {{ }} operators and a {% verbatim %} tag that are ignored by the Django Template Engine as they’re all inside of a {% verbatim %} block:

```
{% verbatim myblock %}
    Many JavaScript templates use {{ }} operators which overlap
    with the Django Template Engine. A similar problem exists if
    you wish to show a tag like {% verbatim %}.
{% endverbatim myblock %}
```

3.5.4 Referencing URLs

There are many situations where URLs appear inside of HTML. You may be linking to another page, referencing a CSS file, or showing an image. In most of these cases your URL will be pointing to another Django view, or an asset under Django's control. You can hard-code the URL in your HTML, but if you decide to refactor your URL mapping, you'll end up with a lot of work, or worse, some broken links.

To help solve this problem, Django provides a way of naming URL mappings. Recall the `django.urls.path` objects inside urls.py that map a URL to a view. These objects have an optional argument allowing you to name the `path`. You can reference a URL in your code using this name, or in a template through the `{% url %}` tag.

To see this in action, you first will need to name one of your URLs. Open RiffMates/RiffMates/urls.py and modify the `path` object for the Credits page, adding the `name` attribute like in listing 3.4.

Listing 3.4. Naming a URL mapping

```
# RiffMates/RiffMates/urls.py
...
from home import views as home_views
...
urlspatterns = [
    ...
    path('credits/', home_views.credits, name="credits"), #1
]
```

By adding the `name="credits"` argument to the `path` object, this URL mapping can now be referenced with the string `"credits"` elsewhere in your code and templates. To access the resulting URL in a template, use the `{% url %}` tag, giving it `'credits'` as an argument. Note the surrounding quotes,

the `{% url %}` can take either a variable or a hard coded string. The most common use for the `{% url %}` tag is inside a link in your HTML:

```
<a href="{% url 'credits' %}">Credits Page</a>
```

The preceding anchor tag's `href` parameter gets rendered using the URL to your Credits page. If you decide to change the URL for the `credits()` view, you only do this in the urls.py file, the `{% url %}` tag takes care of the rest.

Tip

You should always use named URLs in your code, doing otherwise is like not using constants in your code.

Take careful note of the use of the single and double quotes in this example. As HTML uses double quotes to demark a tag's attributes, you need to use single quotes inside the `{% url %}` tag.

3.5.6 Common filters

Tags are sometimes for rendering a specific value, like with `{% now %}`, or `{% url %}`, but are mostly used for control flow, like `{% if %}`, and `{% for %}`. Often, you want to perform an action on some data, modifying how it will be displayed. Consider a `datetime` object in Python, it stores date and time information, but when you want to print it out, you have a large number of choices for formatting the display. By changing the arguments to `.strftime()` method, you change how the date is shown. Filters in Django templates are a generic way of doing this kind of action on all kinds of data.

Filters apply inside of double-brace surrounded variables, applying the filter's change effect to the variable before it is rendered. You use a filter through the pipe (|) operator within the double-braces. The input to a filter is the contents of the variable, while its output is an action applied to the data.

Two commonly used filters are `| upper` and `| lower`. These behave like the `str.upper()` and `str.lower()` calls, changing the case of the corresponding string. The template `{{ word | upper }}` results in rendering the contents of

`word` after changing all its letters to upper-case.

Some other common filters are:

- `{{ num | pluralize }}`: Returns “s” if the value of `num` is greater than 1.
- `{{ words | first }}`: Returns the first item in a `list`
- `{{ words | last }}`: Returns the last item in a `list`

The `{{ | pluralize }}` filter is useful for composing counting sentences. For example, the template text `You have {{ num }} message{{ num | pluralize }}` gets rendered as "You have 3 messages." when `num` contains three. The `{{ | first }}` and `{{ | last }}` filters are handy when you need items out of a list. You can always use `{{ words.0 }}` to also get the first, but as the template language doesn’t support negative indexing, the only way to get the last item is with this filter.

Like tags, filters can take arguments. You pass arguments to a filter using the colon (:) character. The `| join` filter works like Python’s `str.join()` method, taking a `list` and constructing a string from the result. This filter takes an argument specifying the separator between the joined items, most commonly a comma (,). The template `{{ words | join:"," }}` is equivalent to the Python `",".join(words)`.

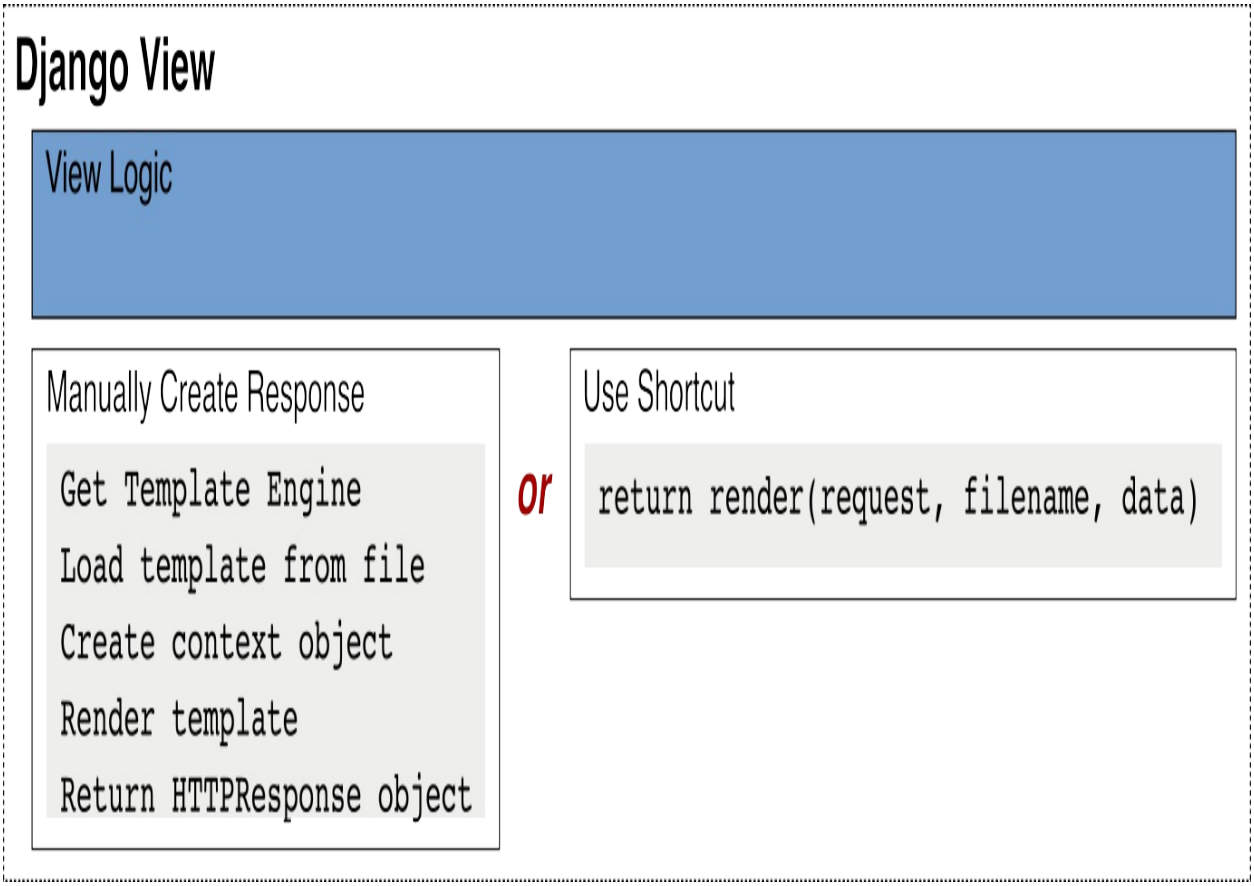
You’ve seen how to create `Template` objects, how to render them using a `Context`, and an overview of some key tags and filters. Doing all of this in a view would make your Python code rather unwieldy. Almost every view you write is going to want to render a template to return some HTML to your user. Because this is such a frequent activity, Django provides a shortcut method called `render()`. This shortcut loads a template from a file, creates a `Context` object based on a dictionary, and returns an `HTTPResponse` object, all in a single line of code.

3.6 Using `render()` in a view

Each time someone visits a page on *RiffMates* your view is going to return some HTML to their browser. The vast majority of your views will do a bit

of logic processing and then load and render a template containing the HTML for their page. The `django.shortcuts.render()` function encapsulates most of what you need to render some HTML in a single call.

Figure 3.2. The `render()` shortcut saves you code



The `render()` function has two required arguments: the `HTTPRequest` object passed into the view, and the name of a template to load from disk. You tell Django where to look for templates by changing configuration in settings.py, more on that in a moment. The `render()` function also has optional arguments for providing a context dictionary, specifying the MIME type of the response, and changing the HTTP status code.

The view for your Credits page consisted of a hard-coded string using plain text. You want *RiffMates* to contain more complicated pages than that. Let's add a view that shows your users the latest happenings on the site with a News page, this time using HTML.

You'll need a view for your News page. Call it `news()` and put it in RiffMates/home/views.py along with your views for the Credits and About pages (if you did the exercises at the end of chapter 2). The `news()` view uses the `render()` shortcut which needs to be imported at the top of the file.

The signature for `news()` is the same as `credits()`, taking a `request` object as an argument. Inside the view a dictionary holds a list of containing the news shown on the page. This dictionary gets passed to `render()` as context for the template. The result from `render()` is an `HTTPResponse` object that the view then returns. The `render()` shortcut takes three arguments:

1. The `request` object
2. The name of an HTML page, `news.html` in this case
3. A reference to the data dictionary to be used as the template context

Modify RiffMates/home/views.py, adding the import for `render()` and the new code for `news()` as in listing 3.5.

Listing 3.5. Adding a view for the news page

```
# RiffMates/home/views.py
from django.shortcuts import render
...
def news(request):
    data = {
        'news':[
            "RiffMates now has a news page!",
            "RiffMates has its first web page",
        ],
    }

    return render(request, "news.html", data)
```

Like all views, `news()` must be registered as a route. You need a `path` object inside RiffMates/RiffMates/urls.py for the `news()` view, similar to how you did for `credits()`. Add the new `path` object to `urlpatterns` like in 3.6.

Listing 3.6. Registering the 'news' view as a route

```
# RiffMates/RiffMates/urls.py
...
```

```
urlpatterns = [
    ...
    path('news/', home_views.news), #1
]
```

The `render()` shortcut in the `news()` view looks for an HTML template named news.html. Before creating that template, you must tell Django where to look for templates. You do this by modifying the `TEMPLATES` configuration in RffMates/RiffMates/settings.py. The `TEMPLATES` value is a dictionary containing four keys that control how templates work in Django.

1. `BACKEND` specifies which template engine to use in your project. Django ships with the original Django Template Engine as well as a popular third-party template engine named Jinja2. You can also install other third-party engines if you prefer. I'll be sticking with the default throughout this book.
2. `DIRS` is a list of directories where Django looks for templates.
3. `APP_DIRS` is a boolean value. When `True` (the default), Django will look for template directories within the apps directories.
4. `OPTIONS` is a nested dictionary to pass configuration options to the template engine. The default values for this are sufficient.

By default, the `DIRS` property is an empty `list` and needs to be changed to wherever you decide to put your templates. The most common place to put your templates is in your project folder in a directory named "templates". Use your file explorer or the command line to add such a directory. Note this goes in your project folder, it is a sibling of your home app directory, it does **not** go in the RiffMates/RiffMates configuration folder. Listing 3.7 shows the complete folder structure so far.

Listing 3.7. RiffMates project structure including the new templates directory and the soon to be created news.html

```
RiffMates/
├── RiffMates
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
```

```
├── db.sqlite3
├── home
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
├── manage.py
├── requirements.txt
└── templates
    └── news.html
```

Template locations: a common location versus app directories

The `APP_DIRS` option in the `TEMPLATES` configuration allows you to tell Django to look inside your apps for templates. When `APP_DIRS` is `True`, Django will look for a directory named templates in each of your app directories. Whether you keep all of your templates with their corresponding apps or put them in a common location is a matter of style.

When I first started writing Django, I tended to keep my templates with their corresponding apps. I only used the common folder for reusable template blocks. Over time, I have drifted in the other direction. For most projects I have found it simpler to keep all the HTML in one place.

The one exception is if the app itself is reusable. Reusable apps are beyond the scope of this book, but you can write Django apps that can be installed like other Python projects. In this case, any templates the app needs must be shipped with the app and thus go in the app's templates directory.

The `DIRS` property list expects a fully qualified path to your template directory. Paths like this are common in the settings.py file. As most paths in your project are going to be relative to the project directory, Django declares a configuration option near the top of settings.py called `BASE_DIR`. By default, the value of `BASE_DIR` is the fully qualified path of your project directory inside a `pathlib.Path` object. Older versions of Django used `os.join()` and strings to specify a path, so be careful if you are dealing with older code. Modify the `DIRS` option of the `TEMPLATES` configuration in settings.py to point your newly created RiffMates/templates directory. The resulting

configuration for `TEMPLATES` is shown in listing 3.8.

Listing 3.8. Modified `TEMPLATES` value in the RiffMates/RiffMates/settings.py file pointing to your project template directory.

```
# RiffMates/RiffMates/settings.py
...
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplat
        'DIRS': [ BASE_DIR / 'templates', ],  #1
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messa
            ],
        },
    },
]
```

Your view is in place, and you have told Django where to look for news.html. The only thing left is to create `news.html`. This template is relatively small, containing a header and a `{% for %}` loop inside a `<ul>` tag. The `{% for %}` loop iterates over the `news list` in the context dictionary and creates an `<li>` item for each thing in the list.

Create RiffMates/templates/news.html according to listing 3.9.

Listing 3.9. HTML template to be rendered by the `news` view

```
<!-- RiffMates/templates/news.html -->
<!doctype html>
<html lang="en">
<head>
  <title>RiffMates News</title>
</head>
<body>
  <h1>RiffMates News</h1>

  <ul>
    {% for item in news %}
```

```
      <li>{{ item }}</li>
    {% endfor %}
  </ul>
</body>
</html>
```

With all these changes in place, it is time to see the results. Run the development server using `python manage.py runserver` and visit `http://localhost:8000/news/`. It should look something like figure 3.3.

Figure 3.3. Your `news()` view

You're much closer to a real website now. Each web page you wish to host gets its own view. Each view gets a corresponding URL as a registered route, and an HTML page. The HTML gets rendered from a template in your project's RiffMates/RiffMates/templates folder. There two more things though that will make your HTML writing easier: escaping special

characters, and composing templates. Let’s start by dealing with those pesky special characters.

3.7 Escaping special characters

HTML uses angle brackets to make tags. Unfortunately, angle brackets are also known as the greater than and less than signs. Your text may need to include these symbols, so HTML provides entities to work around this. If you want to write a greater than symbol in HTML, you use `>`.

Template variables are data. Data comes from all sorts of places, and may contain characters special to HTML. To display your data on the page as it is meant to appear, a transformation has to happen. Django has your back and automatically does the transformation. This is known as escaping characters.

To play with escaping, you’re going to need another template. Earlier you used the Django REPL to directly instantiate a `Template` object, for historical reasons this version of the object does not do auto-escaping. When you load a template file into the engine through the `render()` shortcut, you are actually using a different version of a `Template` object that does auto-escape. To build a realistic experiment you need a little snippet of a template file in your RiffMates/RiffMates/templates directory, I named mine experiment.html. Create your own experimental file like in listing 3.10. My version renders a single variable.

Listing 3.10. Mini-template experiment.html for playing with auto-escape

```
I love the sound a {{ instrument }} makes.
```

With this template in place, you can load and render it in the REPL. You’re going to do that a bunch using different values of `instrument` to see the results of auto-escaping.

Remember, you can get to Django’s version of the REPL using the `manage.py shell` command. This time, instead of creating a `Template` object, load one using the template engine based on your experiment.html file.

Django supports multiple template engines. You retrieve the one currently configured in your settings using `Engine.get_default()`. `Engine` objects have a `get_template()` method that loads a template similar to the `render()` shortcut.

Once you have a `Template` object, call `.render()` passing in a `Context` object like you did earlier in the chapter. Do this with different values for `instrument` to see the results of auto-escaping. Listing 3.11 renders the template twice, once with `trombone` and once with `tuba > baritone`.

Listing 3.11. Rendering experiment.html with and without special HTML characters in the Django REPL

```
>>> from django.template import Engine, Context
>>> engine = Engine.get_default()
>>> t = engine.get_template("experiment.html")
>>> data = {"instrument":"trombone"}
>>> t.render(Context(data))
'I love the sound a trombone makes.\n'
>>> data = {"instrument":"tuba > baritone"}
>>> t.render(Context(data))
'I love the sound a tuba > baritone makes.\n'
```

Note how changing the value of `instrument` from `trombone` to `tuba > baritone` causes auto-escaping to execute. The greater-than symbol (>) is rendered as `>`, making it safe to display in HTML.

Django gives you several ways to control auto-escaping. The `| safe` filter indicates that a value should not be escaped. You do this if what you're rendering has HTML in it and you want it to display as HTML rather than be escaped. The `| escape` filter does the opposite, causing the filtered value to be auto-escaped. The `{% autoescape %}` tag allows you to turn auto-escaping on or off for a block. To see these in action, add a few more lines to experiment.html:

```
I love the sound a {{ instrument }} makes.
A shiny {{ instrument | safe }} is good.

{% autoescape off %}
Don't put a {{ instrument }} too close to your ear.
{% endautoescape %}
```

The template engine compiles and caches templates; if you modify experiment.html you may or may not get the latest copy in the REPL. The Django development server watches your files and will reload edited templates, but that isn't guaranteed in the REPL. Exit the REPL after making the previous changes, then run it again to see the effect of your new tags. Your results should look like listing 3.12. When you're done, don't close your REPL session, you're going to play around some more using the same template in a moment.

Listing 3.12. Rendering 'experiment.html' after your changes in the Django REPL

```
>>> from django.template import Engine, Context
>>> engine = Engine.get_default()
>>> t = engine.get_template("experiment.html")
>>> data = {"instrument":"flute"}
>>> t.render(Context(data))
"I love the sound a flute makes.\nA shiny flute is good.\n\n\nDon
>>> data = {"instrument":"<b>sax</b>"}
>>> t.render(Context(data))
"I love the sound a &lt;b&gt;sax&lt;/b&gt; makes.\nA shiny <b>sax
```

Without modification, the `<b>` tags get escaped. Using either the `| safe` filter or the `{% autoescape %}` tag allows you to include HTML inside your data.

There is one more way to effect auto-escaping behavior: changing the data. Django has a subclass of a Python `str` called `SafeString`. You get a `SafeString` by calling the `mark_safe()` utility method on a string. Alternatively, If you are dynamically building snippets of HTML you can use `format_html()` which also returns a `SafeString`. Any string marked as safe will not be escaped. Operations on `SafeString` objects return regular `str` objects, if you need to modify a `SafeString` you will have to mark it as safe again.

Let's play with these utilities in the same REPL session. Use the same template as before, but this time modify the context data to contain a string marked as safe. The `mark_safe()` function takes the string you want to mark safe as an argument. Add a safe string with a `<br/>` tag to the `instruments` context and render it like in listing 3.13.

Listing 3.13. Using `mark_safe()` to mark data as safe in the Django REPL

```
>>> from django.utils.safestring import mark_safe
>>> instrument = mark_safe("drums<br/>")
>>> data = {"instrument":instrument}
>>> t.render(Context(data))
"I love the sound a drums<br/> makes.\nA shiny drums<br/> is good
```

The `format_html()` function works like `str.format()`, using braces as placeholders and passing in arguments used to populate the string. The `format_html()` method can save a lot of typing if you're building a chunk of HTML, where otherwise you'd need to compose and use `mark_safe()` on individual pieces. In the same REPL session build a phrase that contains paired `<i>` tags with a placeholder between them like in listing 3.14.

Listing 3.14. Using `format_html()` to mark data as safe in the Django REPL

```
>>> from django.utils.html import format_html
>>> instrument = "bass"
>>> instrument = format_html("big <i>{}</i>", instrument)
>>> data = {"instrument":instrument}
>>> t.render(Context(data))
"I love the sound a big <i>bass</i> makes.\nA shiny big <i>bass</
```

User input and the safety of strings

Escaping strings becomes that much more important when you start dealing with user input. Malicious users may give you data that intentionally messes up your web pages. Be careful using any of the tools that turn off escaping, make sure you know what you will be allowing on your page.

There are even more string management utilities in `django.utils.html` which deal with rendering JSON, doing tag processing, and dealing with those pesky strings input by users. See the documentation for more details: https://docs.djangoproject.com/en/4.2/ref/utils/#module-django.utils.html.

Escaping is all about the micro-level of HTML, making sure your data gets rendered as you intended. At the macro level you have a different problem: HTML is repetitive. The Django template language includes a way to compose and inherit blocks to help you write less HTML.

3.8 Composing templates out of blocks

Modern HTML pages tend to have a lot of moving parts, navigation bars, footers, side-bars, breadcrumbs, and more. In addition to having lots of pieces, the structure of web pages can be complex in order to be responsive across different platforms and browsers. Much of the source used to manage this complexity gets repeated on most pages across a website. To help with this, the Django Template Engine allows you to define a system of reusable components called *blocks*.

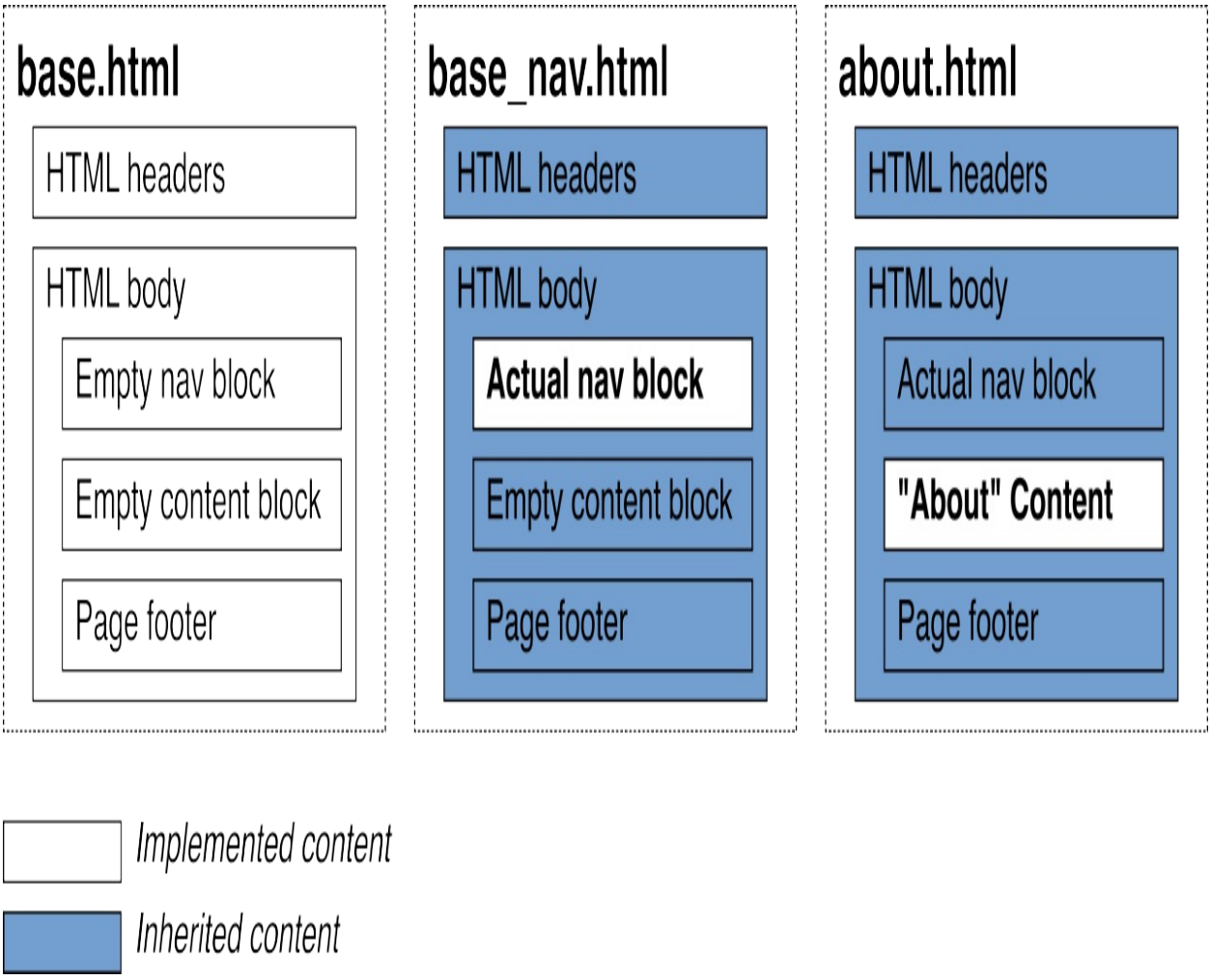
A template block labels a section of a page with a name using paired `{% block %}` and `{% endblock %}` tags. Template pages can be composed together in an inheritance hierarchy with children gaining the structure of their parent. Any blocks in a child page with the same name as the parent page replace the block in the parent page.

Consider the `<title>` tag in head section of an HTML page. General pages in your site might set the title to "RiffMates", while specific pages may want to replace this with more specific information. In a parent template you could declare a block with the name `title` and then any inheriting child page could override this block with the specific name.

You perform inheritance with the `{% extends %}` tag. A child page declares that it extends a parent, and inherits its parent's structure. The most common use of this is to have a base page that defines the look of your site with some placeholder blocks for content. Regular pages then extend the base page and only declare the blocks that need overriding. I typically call my main parent page base.html. This main parent page contains all the boilerplate HTML that defines the look and feel of a site. Included in the boilerplate is a block named `content` that is empty. Any other page in the site then extends the base page, declaring a `content` block that contains the details for that page.

For more complicated sites you may want to have a couple of variations on the main page. A common pattern is to have a main page without navigation, a page that inherits from that with navigation, and then your actual site pages. Figure 3.4 shows an About page using this hierarchy.

Figure 3.4. Page inheritance for component reuse



3.8.1 Writing a base template file

To put all this into practice let's write a parent page for your website. This page contains the HTML common to all pages on the site. It defines two blocks: one for the title area, and a second for content. Call the file base.html and put it with your other templates in the RiffMates/templates directory. Listing 3.15 shows Example content for this parent. If you want prettier web pages, this is the place to insert code for your favorite web style framework like Bootstrap or Tailwind.

Listing 3.15. Base HTML template for inheritance

```
<!-- RiffMates/templates/base.html -->
<!doctype html>
<html lang="en">
```

```
<head>
  <title>
    {% block title %}RiffMates{% endblock %}
  </title>
</head>
<body>
  {% block content %}
  {% endblock content %}
</body>
</html>
```

The closing `{% endblock %}` can optionally include the same name as its corresponding `{% block %}` tag. If you don't specify the name, the most recent block gets closed. By including the name of the block in the closing tag you make your code clearer in the case of nested blocks.

To use your new base.html you create a child page containing the `{% extends %}` tag. This tag takes a single argument: the name of the template it is extending. Inside the child file you override the blocks declared in the parent and insert the corresponding content. Create news2.html, a new version of the News page. This file declares two blocks: one to override the title, the other to override the content.

Inside a block there is a handy context variable named `{{ block.super }}`. This variable contains the content of the parent block. This is useful if you want to add to the parent block rather than completely replacing it. Inside news2.html write content similar to that in listing 3.16.

Listing 3.16. A new 'News' file

```
<!-- RiffMates/templates/news2.html -->
{% extends "base.html" %}

{% block title %}
  {{block.super}}: News
{% endblock %}

{% block content %}
  <h1>RiffMates News (2)</h1>

  <ul>
    {% for item in news %}
      <li>{{ item }}</li>
```

```
    {% endfor %}
  </ul>

{% endblock content %}
```

Because you added a new template file instead of modifying the existing one, you need to update your `news()` view. Edit `RiffMates/home/views.py` and change the `render()` call to point to the newly created news2.html file.

Once you have done that run the development server and visit `http:localhost:8000/news/` again to see the change. The page will be similar to before, but this time with the "2" tacked into the `<h1>` tag to prove it worked. Although the result in the browser is the same, news2.html is shorter than news.html, and if you start styling to your base page it will get applied across every template that inherits from base.html.

Inheritance is useful for creating common base files and reusing structure throughout your web pages. You may though want to have a chunk of HTML repeated within your page instead. A nice companion to inheritance is the ability to include other templates inside your HTML.

3.8.2 Including an HTML snippet

I heard you want to have templates in your templates. Well, there is a tag for that. The `{% include %}` tag loads a sub-template into your template. There are two common uses for this: organizing your template files and reusable components. HTML files have a tendency to get overly long. If you want to organize them in pieces you can compose a page from a series of included sub-templates. The simplest use `{% include %}` takes a single argument: the name of the template to be included. Listing 3.17 shows a sample file that includes two sub-templates.

Listing 3.17. An HTML file that includes two sub-templates

```
<!-- Sample template using include -->
{% extends "base.html" %}

{% block content %}

    <h1>Nicky's Blog</h1>
```

```
    {% include "blog1.html %}

    {% include "musicians_list.html" %}
{% endblock content %}
```

Once you have sub-templates, you can include them into multiple files, giving you reusable components. This is similar to a function call in your code: you encapsulate some HTML together and then compose it into other templates as needed. Like a function, you can even declare arguments to your sub-template. The `{% include %}` tag has an alternate form that uses the keyword `with`. This form gives you the ability to declare key/value pairs as context in the sub-template.

Consider the case where you are using a third-party comment section such as Disqus on your website, or including page view tracking JavaScript. In both these cases you need to include some HTML that is parameterized for the given page. You can build this in a reusable fashion by creating a sub-template with a variable and then use the `with` clause when instantiating the `{% include %}` tag.

The following is a sub-template that is part of a pagination footer. The view rendering the template takes a query string specifying a page number. You'll learn how to write this kind of view in a later chapter. For now imagine a view that renders different content based on a query parameter named `page`. Each instance of the sub-template contains links to the previous and next page. The sub-template in listing 3.18 has two links based on variables named `prev` and `next`. All of this is wrapped in `{% if %}` tags; each link only shows for non-empty values. Remember that empty values are treated as `False` for the purposes of the `{% if %}` tag.

Listing 3.18. A pagination footer

```
{% if prev %}
    <a href="?page={{prev}}">Prev</a>
{% endif %}
{% if next %}
    <a href="?page={{next}}">Next</a>
{% endif %}
```

The preceding sub-template can be reused in any web page through `{% include %}`. To populate the values, the tag uses the `with` format providing numbers for `prev` and `next`. Listing 3.19 shows the use of the sub-template for page number 5.

Listing 3.19. Including a sub-template with values

```
<h3> More Content </h3>

{% include "pagination.html" with prev=4 next=6 %}
```

Normally the first argument to `{% include %}` is a string containing the name of the sub-template to include. The tag also supports the use of variables to specify the sub-template name. If you're doing something particularly dynamic, you can get the sub-template name from the data context. That means you can include different sub-templates based on the context set in your view.

Templates are a powerful tool to manage HTML in a manner more like your Python. Instead of writing large files with lots of repetition, you build composable parts. Instead of hard-coding content, you build segments that include conditional rendering and loops that change the page based on the context data passed in from a view. All of this results in more maintainable code. The next step is to extend this power to your views: instead of using hard-coded data dictionaries you retrieve content from a database.

3.9 Exercises

Practice makes perfect, here are some activities that you can try on your own:

1. Create a new base HTML file using your favorite web style framework such as Bootstrap or Tailwind. Once you have something pretty, test it out by changing 'news2.html' to extend from it.
2. Write a more advanced version of the 'News' page by adding a `news_advanced()` view to `home/views.py`. Each data item in this view is a tuple containing the date and subject of the news. Create a 'news_adv.html' template that shows the data in a table. Look-up the `{% cycle %}` tag and the `|date` filter in Appendix C and apply them to your

table, using `{% cycle %}` to apply striping to the table and `|date` to format the news item's date.

3.10 Summary

- The Django Template Engine is system that allows you to compose and render text (typically HTML) like code.
- Django has its own templating language that supports rendering variables, conditional segments, looping, inheritance, and composition.
- Templates get rendered using a context that provides data for the result.
- `Template` objects can be created directly, but the more common case is to use the `render()` shortcut in your views
- Tags provide a way to control how your content gets rendered (like conditionals or loops), or can act as function calls returning data (like inserting a named URL).
- Filters work in conjunction with rendered variables, modifying them in place. This allows you to change the appearance of data inside the template itself rather than in the view.
- URLs can be named and then referenced in your code or in a template without hard coding a value.
- The `TEMPLATES` section in settings.py controls what template engine gets used and where that engine looks for template files.
- The Django Template Engine automatically escapes characters that are meaningful to HTML. You can control what is an isn't escaped through a variety of tags, filters, and the `SafeString` object.
- Templates can inherit from other templates. This gives you a way to create a common base file with your boilerplate HTML and keeps your actual pages smaller.

4 Django ORM

This chapter covers

- Interacting with a database through the Django ORM
- Creating objects that map to database tables
- Querying the database
- Building object relationships
- Loading and storing content through fixtures

This chapter introduces you to Django's *Object Relational Mapping* (ORM), an object-oriented abstraction for interacting with databases. Django provides a way to create, read, update, and delete both data and tables in a database to add storage capabilities to your website.

4.1 Interacting with a database

Until now, *RiffMates* is composed of mostly static content. You learned how to write views that are called when a user visits a URL, and inside those views you created small dictionaries to provide context for rendering templates. Hard-coding data dictionaries in a view is rather limited. Most websites have a storage layer that is queried inside the view. Storage gets used for:

- User accounts
- Inventory
- E-commerce transactions
- Experience customization
- User created content
- Site data

If the phrase "site data" seems vague to you, that's because it is. A lot of websites store and display data that is specific to that site. A movie site has data for movies, a book site has data for books. For Nicky's *RiffMates* site

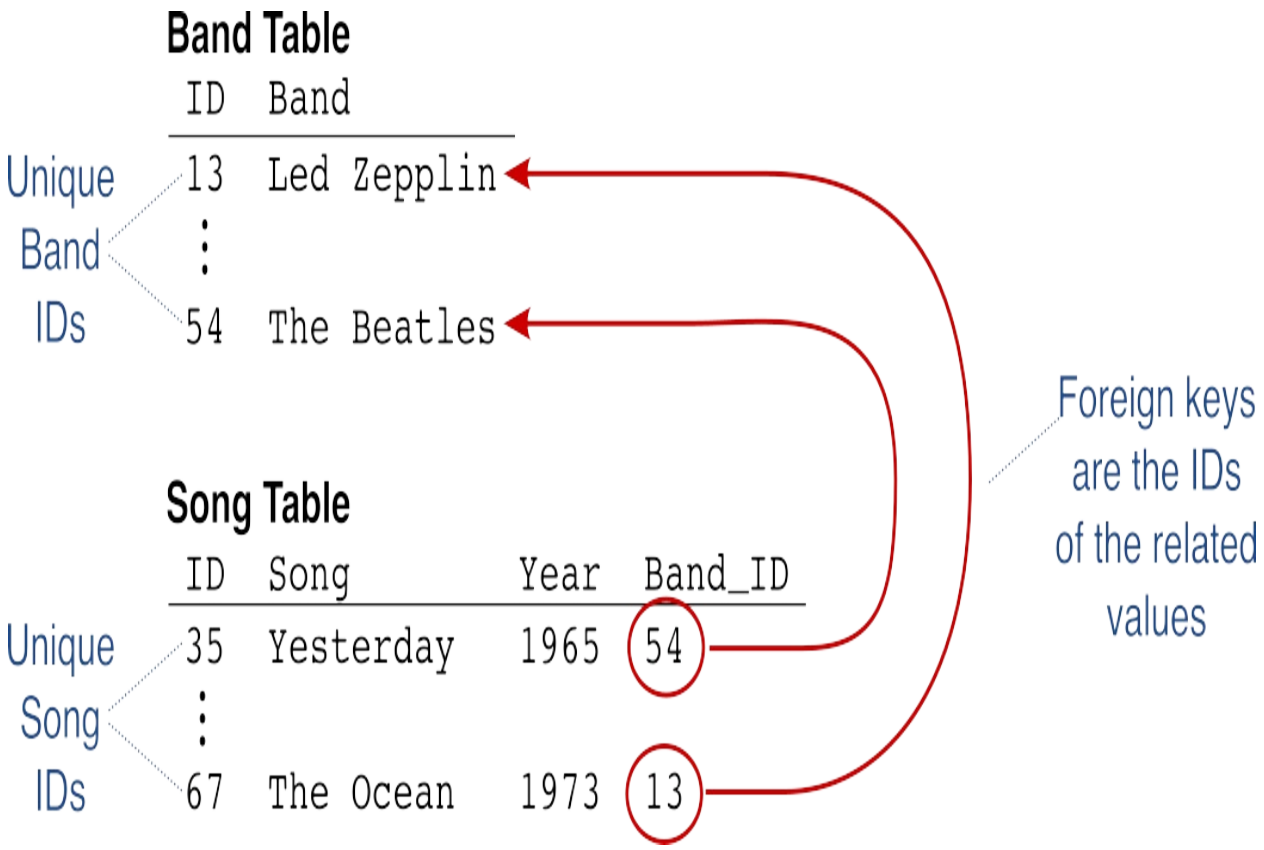
you need:

- Musicians
- Bands
- Classifieds
- Venues
- Venue Line-ups

As *RiffMates* grows, so will this list. There are a variety of approaches for storing this data, but by far the most common is to use a relational database. You'll recall that a relational database stores content in a series of tables similar to how a spreadsheet works. The columns in a table have a data type, typically storing strings or numbers. To represent a band, your columns might include the band name, the date they were formed. Each row in the table corresponds to a single band. To be able to reference a band, the table includes an identity column, typically an auto-incrementing number provided by the database. This identity is called the table's *primary key*.

Columns can also indicate a relationship between data items, hence the "relational" in "relational database". To represent a song, your columns include the name of the song and like the bands, a unique identifier. To link a band and a song, either the band or song table includes a column that references the other's unique identifier. You can see an example of this in figure 4.1.

Figure 4.1. Inter-table relationships between bands and songs



Relational databases have their own programming language called *Structured Query Language* (SQL). SQL includes structural directives for managing tables and query directives for handling the data inside the tables. One way of dealing with storage for your website is to write SQL embedded inside your Python, executing it through a connection to your database. In this case, when you query the table with song data, you get back a series of rows from the song table. Any reference to a band identifier would mean another SQL query, this time to the band table.

Creating tables means writing scripts in SQL. If you decide to add a "composer" column to the song table, that means another SQL script. It doesn't take long before you are writing a lot of SQL, fortunately there is an alternative.

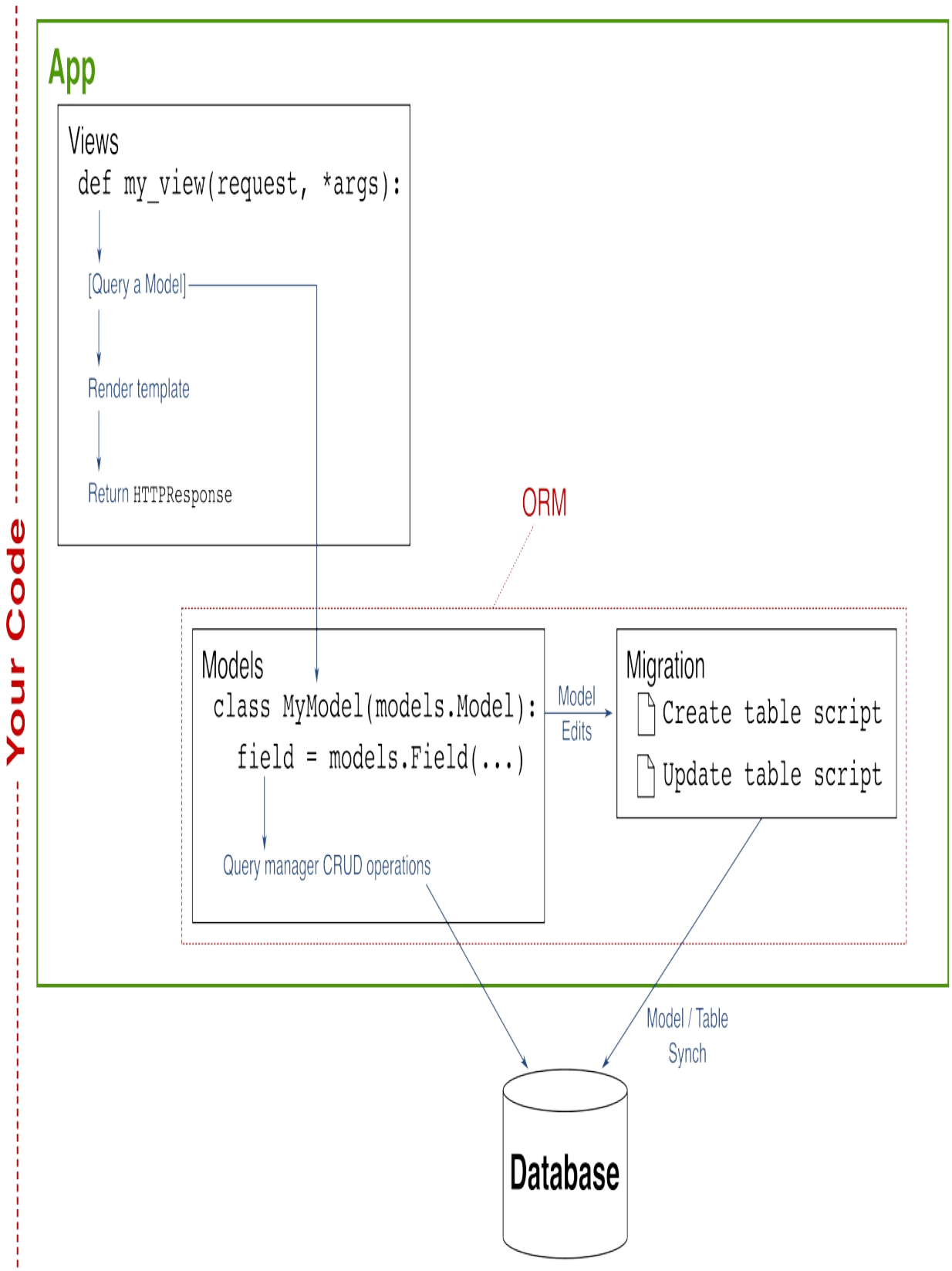
Object Relational Mapping (ORM) is a programming technique that maps content in a relational database to object-oriented code. Instead of thinking in terms of tables, you write classes representing the data in your project. A musician class contains attributes for first name, last name, and date of birth,

then the ORM maps this class to tables in the database. This abstraction means you don’t need to write a single line of SQL.

The downside of this mapping is you are now defining the structure of your data in two places: your classes and the database. Adding a stage name to your musician means changing the class but also keeping the database in sync. Django’s ORM includes a table management feature called *migration*. The migration process tracks changes you make to your classes and writes the corresponding SQL scripts for you.

Figure 4.2 shows how the ORM fits into your Django code. You build models in your app, those models are managed through migration scripts, then inside of your views you interact with the database to populate content for your pages.

Figure 4.2. The Django ORM and where it connects to your views


Your Code
App
Views
def my_view(request, *args):
[Query a Model]
Render template
Return HTTPResponse
ORM
Models
class MyModel(models.Model):
field = models.Field(...)
Query manager CRUD operations
Model
Edits
Migration
Create table script
Update table script
Model / Table
Synch
Database

Understanding databases is a deep topic in its own right, and beyond the scope of this book. This chapter will give you enough background in relational databases to understand how to use the ORM, but you may want to dig more into databases.

4.2 ORM Models

At the heart of an ORM is a mapping between classes and tables in a relational database. Unlike Python's dynamic typing, where you can define what kind of data goes in a variable at runtime, relational databases are picky about the data types of a column. To define a mapping between a class and a table, you need to be explicit about the data type of the columns. Django's ORM provides a base class that you inherit from, and a series of fields that specify the types of data for the mapping.

Let's start by defining a musician class that contains the first name, last name, and date of birth of the musician. The first question is: where do you put this? In chapter 2, when you created a Django app, one of the skeleton files built for you was models.py. This file is where Django expects you to build you ORM models. Currently, you only have the home app that contains general content for your site. Create a new app to contain the data definitions and views for musicians and their bands, call it "bands". Remember, you do this with the `manage.py startapp` sub-command:

```
(venv) RiffMates$ python manage.py startapp bands
```

In addition to creating the app, recall that you need to tell Django about it by adding it to the `INSTALLED_APP` configuration value inside RiffMates/RiffMates/settings.py like in listing 4.1.

Listing 4.1. Tell Django about your `bands` app

```
# RiffMates/RiffMates/settings.py
...
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
```

```
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'home',
    'bands', #1
]
...
```

Running the `startapp` sub-command created skeleton files for you in the bands directory, including the models.py file where you define ORM classes. You tell Django that your class is an ORM model by inheriting from `django.db.models.Model`. The skeleton model.py file imports the `models` package into its namespace for you.

The `models` package includes both the base class for ORM models and all the data type fields for mapping to database columns. A musician class that includes first name, last name, and date of birth, needs three attributes, each of which is a field that corresponds to the stored data type. These attributes serve a dual purpose: they both define the data type of the column in the corresponding database table, and they are used to access the content of the field during a query. A `first_name` attribute with the field type `CharField` both states that the first name gets stored as a character field in the database and is the access point for getting a musician's first name when you query the database.

To represent a musician, you create a `Muscian` class and map three fields for the first name, last name, and date of birth. Unlike Python where strings can have an arbitrary length, relational databases want you to be specific about string sizes. The `CharField` requires a `max_length` argument to indicate the size of the column. There are text fields that don't have this limit, but their storage tends to be inefficient; best practice is to use the `CharField` for fields like names. Your first and last name fields are both `CharField`, while the date of birth gets stored using a `DateField`.

How much space?

Deciding how much is a reasonable size for any field is somewhat based on experience (read: guesswork). In the old days, using 10 characters for a first name was quite common. Christopher has 11 characters. I used to frequently

receive mail addressed to "Christophe". Names are culture specific and making assumptions about them can get messy quickly. With disk space being cheap now, being more generous with your field lengths is achievable.

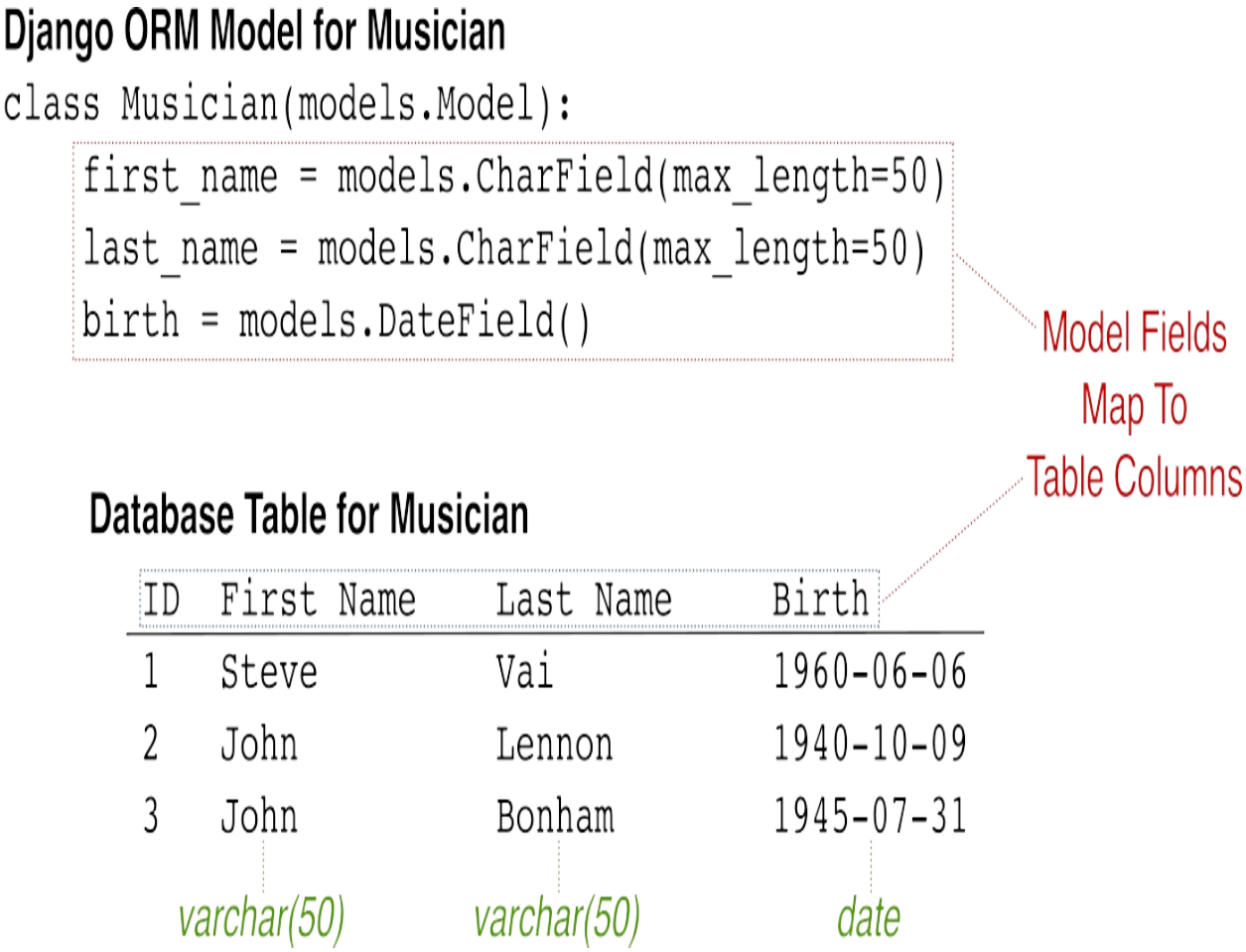
To define your first `Model` class, edit RiffMates/bands/models.py and change it to look like listing 4.2.

Listing 4.2. Musician ORM Model class

```
# RiffMates/bands/models.py
from django.db import models

class Musician(models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
    birth = models.DateField()
```

Defining the model is only the first step. Now you need to create the corresponding table in the database. To map your `Musicians` class to a table, the ORM needs to execute some SQL on the database. Django abstracts this SQL using some Python called a *migration script*. As you make changes to your ORM models, Django tracks the changes and creates new versions of the migration scripts. You then apply the migration scripts to the database. Both the creation and running get done through management sub-commands. The mapping for the `Musicians` model to the table created in the database after the migration gets performed is shown in figure 4.3.

Figure 4.3. `Musicians` ORM `Model` object to database table mapping



The sub-command that creates migration scripts is `makemigrations`. The first time you run this, it creates a *migrations* directory in the app, and an initial migration script. Subsequent calls to the command detect any changes to the model and if add more migration scripts.

Run the `makemigrations` sub-command on the `bands` app using the following command:

```
(venv) RiffMates$ python manage.py makemigrations bands
```

Django responds by telling you what it did, in this case it created the initial migration script containing a model named `Musician`:

```
Migrations for 'bands':
  bands/migrations/0001_initial.py
    - Create model Musician
```

The migration scripts are Python programs and part of your project like any other code file. The details of these scripts and how and when you might modify them by hand gets covered in a later chapter.

With the initial script in place, you use another management sub-command to perform the migration. The `migrate` sub-command runs any outstanding migration scripts that have not previously run on the database. Under the covers, Django is storing the state of migrations in the database itself, and it knows what scripts have been run and which are new.

Recall from chapter 2 the `18 unapplied migration(s)` warning when first running the development server. Django ships with several apps that help you manage users and the database. These apps themselves have models and corresponding migration scripts. When you first ran the development server, Django was telling you that it had detected the presence of un-run migrations. To fix the warning, you ran the `migrate` sub-command. Now that you have your own migration script, you do the same thing to sync your model changes with the database. Run the following:

```
(venv) RiffMates$ python manage.py migrate
```

Django tells you what migrations ran and on what apps:

```
Operations to perform:
  Apply all migrations: bands
Running migrations:
  Applying bands.0001_initial... OK
```

If you skipped the migration step in chapter 2 because you didn't care about the warning, your output will also include all the migrations for the `admin`, `auth`, `contenttypes`, and `sessions` apps. Your project now contains a db.sqlite3 file: the SQLite single-file database. This file can be examined with appropriate tools to see the results of the ORM executing.

4.3 SQLite and `dbshell`

By default, Django is configured to use SQLite as its database. *SQLite* is a local, single-file database. "Local" in this case means on your machine (as

opposed to over the network), and "single file" means the whole database resides in a file on your file system. SQLite is one of the most popular embedded databases out there, and even if you've never used it before, you've probably used it before. It is very likely that one of the apps on your phone uses SQLite as its storage.

 **Note**

Django supports several databases besides SQLite and third-party libraries add even more connectivity support. For details on how to configure Django for a database besides SQLite, see appendix B on running Django in a production environment.

When using the default SQLite database, the first time you use the ORM, Django creates the database file, calling it db.sqlite3. If you have SQLite installed on your computer (available from https://www.sqlite.org), you can use the `sqlite3` command to open the database. If you don't want to install another tool on your machine, that's ok, Django has a management sub-command that does the same thing.

The `dbshell` sub-command invokes the database's equivalent of a REPL. Django supports multiple databases with `dbshell` mapping to the database currently configured for your project. This means you don't have to remember your database's specific command, you can always just run `dbshell`. This is a convenience, but once inside the command you are using the database's interactive tool. The operations you perform on the database and how you perform them are specific to the database's tool.

Run the `dbshell` sub-command to enter SQLite's interactive program:

```
(venv) RiffMates$ python manage.py dbshell
```

When SQLite starts, it gives you its version information and prompts you with `sqlite>`:

```
SQLite version 3.37.0 2021-12-09 01:34:53
Enter ".help" for usage hints.
sqlite>
```

Inside the SQLite shell you can run SQL queries directly. To differentiate shell commands from SQL, SQLite uses a dot (.) suffix for its own commands. The welcome message tells you to use the `.help` command to find out more. The output of `.help` is rather lengthy, it is a full listing of everything you can do in the shell.

One of the commands available to you allows you to can see all the tables in the database. Try the `.tables` command:

```
sqlite> .tables
```

Are there a few more tables than you expected? Remember, Django provides several built-in apps that you migrated, each of which results in a table. There are also system tables; `django_migrations` is where the ORM tracks which migrations have run. Your `Musician` class gets mapped in the `bands_musician` table:

```
auth_group                  bands_musician
auth_group_permissions      django_admin_log
auth_permission             django_content_type
auth_user                   django_migrations
auth_user_groups            django_session
auth_user_user_permissions
```

Django ORM Models name their tables *app*, underscore (*)*, *_classname*. For your `Musicians` class, that results in the *band* app name and the *musician* model name. This is only a default, you can control your table names if you wish, more on that in a later chapter.

The `.schema` command shows you the SQL that was executed to create a table. Run this for the `bands_musician` table:

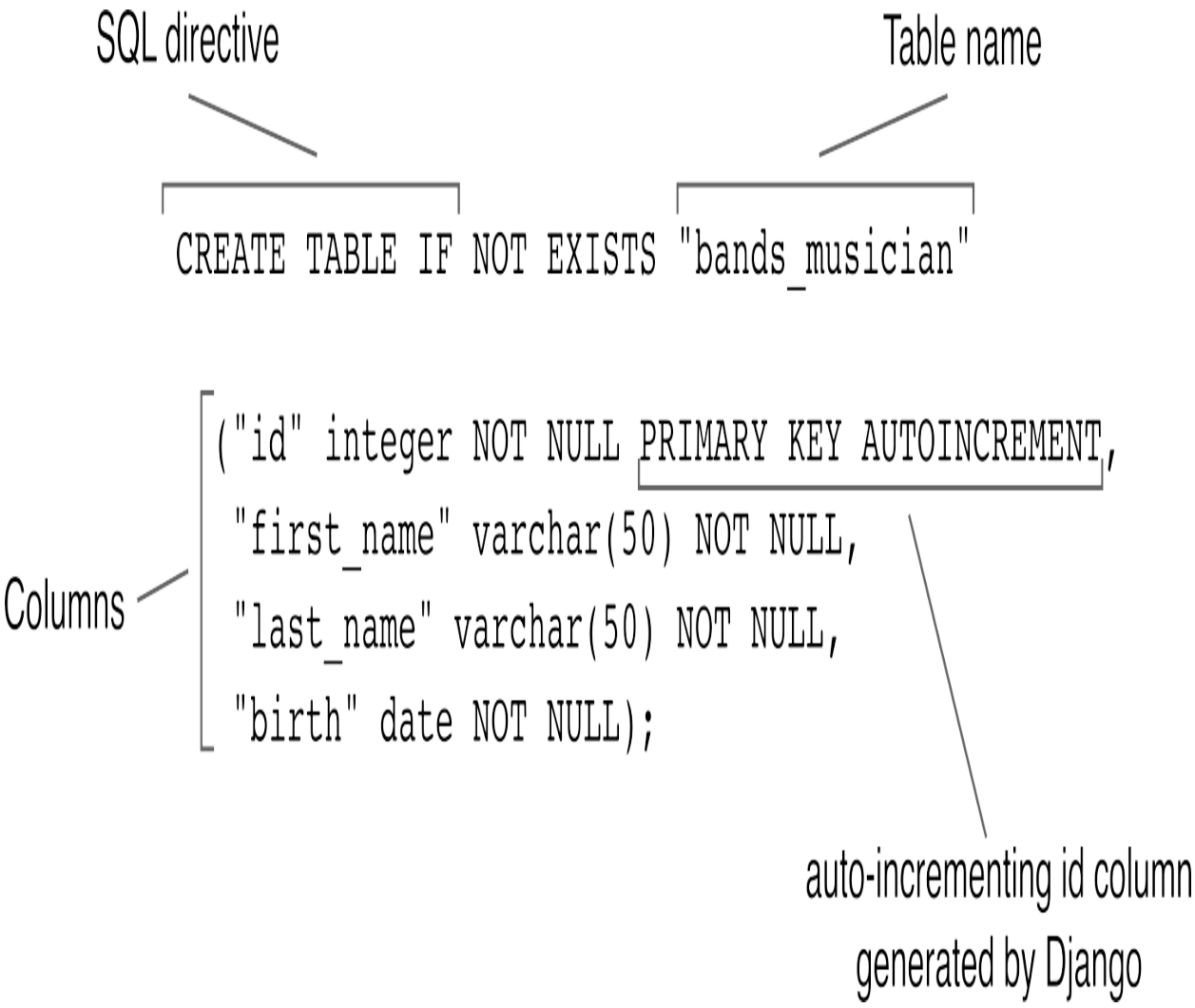
```
sqlite> .schema bands_musician
```

If you're new to SQL, the output can be a little daunting:

```
CREATE TABLE IF NOT EXISTS "bands_musician" ("id" integer NOT NUL
```

The advantage of an ORM is that you don't have to be an expert in SQL. Figure 4.4 breaks the statement down into its component parts.

Figure 4.4. SQL used to create the table storing musicians info



The SQL that creates the table includes the `CREATE TABLE` command, the name of the table to be created (`bands_musician`), and four columns: `id`, `first_name`, `last_name`, and `birth`. Your `Musicians` model did not define a `id` attribute, Django does that for you automatically. It uses an `integer` and declares that it can't be empty (`NOT NULL`), is the identifier other tables used to reference it (`PRIMARY KEY`), and it automatically increments when new rows get added (`AUTOINCREMENT`). The `first_name` and `last_name` columns are `varchar(50)`. A `varchar` is what SQL calls the `CharField`, and the `50` is the max length defined in your model. The `birth` field is of type `date`.

Not only is the `id` field not allowed to be empty (`NOT NULL`), but your three attribute fields are the same. This is the default behavior. You can control this

through additional options to the field declaration. More on this later.

You can use the SQLite command-line tool to directly enter SQL queries. Your `Musician` class doesn't have any data yet, so there is nothing to see. You could write an SQL `INSERT` statement, but that isn't the ORM way of doing things. Leave the SQLite tool by running `.quit` and learn the ORM way.

4.4 Model Queries

Each `Model` class is both declarative and functional. Inheriting from `models.Model` and using the fields from the `models` package defines the structure of the tables that the ORM creates and manages for you. The base class also comes with a *query manager*, an interface for interacting with data in the database. You can create your own query manager but for the vast majority of what you'll do with the ORM, the default query manager will be sufficient. The default, named *objects*, is attached to the base class.

The query manager is a class itself and its methods get used for operations relating to your `Model`. You've created `Musician` and seen the resulting table in the database, now it is time to make some data. The `.create()` method on the query manager does exactly this. The arguments to `.create()` are the fields in your class that you wish to populate. As the `.create()` method is generic, you can't use positional arguments for the fields, you have to use keyword arguments for everything.

The other complication with arguments is which fields are required and which are optional. Because `.create()` is generic, the compiler can't tell if you're missing an argument. `Musician` doesn't have optional fields, so you need to provide keyword arguments for all the fields: `first_name`, `last_name`, and `birth`. If you miss one, it will be at runtime when the Django executes the underlying database statement that you will receive an error.

Field types map to corresponding Python values. The names in `Musician` are `CharField` attributes and map to Python strings, while `birth` is a `DateField` which maps to a Python `date` object.

To create your first Musician, run the Django REPL using the `shell` management sub-command, import the `Musician` object from `bands.models`, and call the `.create()` method, with arguments for your favorite musician:

```
(venv) RiffMates$ python manage.py shell
Python 3.11.1
Type "help", "copyright", "credits" or "license" for more informa
(InteractiveConsole)
>>> from bands.models import Musician
>>> from datetime import date
>>> steve = Musician.objects.create(first_name="Steve", last_name
```

The `.create()` call inserts the content into the database and returns a corresponding `Musician` object. You can display the value of `steve` in the REPL:

```
>>> steve
<Musician: Musician object (1)>
```

The result isn't too friendly. When you display an object like this, the Django REPL casts the object into a string to show the result. The `Model` base class implements `.__str__()`, the special method called when an object gets cast to a string. The provided implementation shows the class name and the object's `id` in parentheses. Your first `Musician` object got an `id` of `1`.

To make it easier to view and debug your objects, it is best practice to override the base class's `.__str__()` method. Exit the Django shell, open RiffMates/bands/models.py, and add the contents of listing 4.3 to your `Musician` class.

Listing 4.3. Overriding `.__str__()` in your `Musician` class

```
# RiffMates/bands/models.py
...
class Musican(models.Model):
    ...
    def __str__(self): #1
        return f"Musician(id={self.id}, last_name={self.last_name
```

Save the file and re-enter the Django shell. The database is meant as permanent storage, you've already created Mr. Vai, now you want to fetch

him back. The `.objects` query manager provides several methods for querying data out of the database and returning them as objects or groups of objects. One of the simpler queries is `.objects.first()`, which returns the first item in the database of that object type. I don't find I use this method much in my views, but it can be handy if you're mucking around in the REPL and need a quick piece of data to play with.

To fetch Mr. Vai's `Musician` object, first you'll need to re-import `Musician`, as closing the shell before will have lost the import state. Once you have the `Musician` class, you run the `.first()` method on the `.objects` query manager and it returns the first (and only) `Musician` object:

```
(venv) RiffMates$ python manage.py shell
Python 3.11.1
Type "help", "copyright", "credits" or "license" for more informa
(InteractiveConsole)
>>> from bands.models import Musician
>>> steve = Musician.objects.first()
>>> steve
<Musician: Musician(id=1, last_name=Vai)>
```

With your new `.str()` in place, the display value of `steve` in the REPL is more readable. Steve is kind of lonely, use `.create()` to add a few more musicians to your database:

```
>>> from datetime import date
>>> Musician.objects.create(first_name="John", last_name="Lennon"
<Musician: Musician(id=2, last_name=Lennon)>
>>> Musician.objects.create(first_name="John", last_name="Bonham"
<Musician: Musician(id=3, last_name=Bonham)>
```

You have some data to play with and a fantasy rock trio to that would make any record company salivate. As you might guess, to go along with `.first()` is a `.last()` query which returns the most recent item added to the database. Try it out on your own. Fetching a single object is kind of boring though, you can get sets of data and filter the content as well.

4.4.1 Using `.all()` and `.filter()` to fetch `QuerySet` results

To find all the objects in a table you call the query manager's `.all()` method.

This returns a `QuerySet` object, a wrapper to the query you ran. Try it out yourself:

```
>>> Musician.objects.all()
<QuerySet [<Musician: Musician(id=1, last_name=Vai)>, <Musician:
```

Individual items, or a subset of the result of a `QuerySet` can be accessed using square brackets similar to a `list` or `tuple`. Re-run the `.all()` query, store the result, then access the second item using `[1]`:

```
>>> result = Musician.objects.all()
>>> result[1]
<Musician: Musician(id=2, last_name=Lennon)>
```

The `QuerySet` object is a lazy object. That's not me being mean, that means it waits as long as possible to be evaluated. You might have hundreds or thousands of musicians in your database. Running `.all()` could result in a large amount of data coming back. The `QuerySet` object wraps the query but doesn't execute it immediately. You can further refine your query by performing operations on a `QuerySet`. Accessing an item in the `QuerySet` is what causes it to be evaluated. In the Django shell, running the `.all()` directly casts the result as a string, invoking the evaluation.

Achieving Laziness

When you call a method on a query manager, it is returning a `QuerySet` object that represents an SQL statement that has not yet been run on the database. Calling further methods on the `QuerySet` results in refining the underlying query, adding more SQL to the statement. It isn't until you perform an action that requires data to be returned that the SQL is executed.

Some query manager methods, like `.first()` return data, and thus result in the SQL statement being executed. Converting a `QuerySet` to a list, or accessing an individual element through an index also executes the SQL.

Databases are optimized for searching and filtering on items. As much as possible, you want the database to do the work. You can ask for all musicians and search the result for those with first name "John", but this is inefficient. Not only do you have to write the search code yourself, but if the database

you are using is on the network, all the data has to be sent over the wire. Instead, you want to ask the database to run queries such as this for you.

The query manager provides the `.filter()` method for accessing a subset of the objects from a table. The arguments to `.filter()` are key/value pairs where the keys are the fields you are searching, and the values are the content you are looking for. To find all the musicians with the first name "John", you call `.filter()` using `first_name="John"` as the argument:

```
>>> Musician.objects.filter(first_name="John")
<QuerySet [<Musician: Musician(id=2, last_name=Lennon)>, <Musicia
```

Because `QuerySet` objects are lazy, you can chain them together. Re-run the previous query, store the result, then call `.first()` on the result:

```
>>> result = Musician.objects.filter(first_name="John")
>>> result.first()
<Musician: Musician(id=2, last_name=Lennon)>
```

The `.filter()` method is extremely powerful. You can perform matches on sub-strings, or do comparative matching on numeric or date values. All of this is performed by altering the arguments to the method.

4.4.2 Field lookups

You called `.filter()` to find an exact match using `first_name="John"`, but what if you want everyone who’s first name begins with the letter "J"? Querying `.filter(first_name="J")` won’t work. That looks for an exact match of first name "J". The `.filter()` method supports the modification of the field name to specify a change in how the query gets performed. These modifications are known as *field lookups* and are indicated using a double-underscore (__).

The `__startswith` field lookup is how you accomplish begins-with-J. You still use the `.filter()` method, but instead of specifying `first_name`, you specify `first_name__startswith`:

```
>>> Musician.objects.filter(first_name__startswith="J")
<QuerySet [<Musician: Musician(id=2, last_name=Lennon)>, <Musicia
```

This can take a little getting used to. I have caused many bugs by forgetting that the base behavior is exact match, or by sticking an asterisk into the string. It can feel a little anti-Pythonic, but it maps to the underlying SQL. The more familiarity you have with SQL, the less weird this all seems.

There are many field lookup modifiers. All value comparison gets done using field lookups: greater than, less than, greater than equal, etc. Although it might feel natural to specify "J*", that doesn't translate into queries like "after this date", or "smaller than this number". To find our musicians born in 1945 or later, use the `__gte` field lookup to find dates greater than or equal to January 1, 1945:

```
>>> Musician.objects.filter(birth__gte=date(1945, 1, 1))
<QuerySet [<Musician: Musician(id=1, last_name=Vai)>, <Musician:
```

Some of the more commonly used field lookups are in table 4.1.

Table 4.1. Commonly used field lookups

Field Lookup	Name	Description
`__contains`	Contains	Text field contains the given value
`__icontains`	Case-insensitive contains	Text field contains the given value, ignoring case
`__gt`	Greater than	Objects with a field greater than the given value
`__gte`	Greater than equal	Objects with a field greater than or equal to the given value

`__in`	In iterable	Objects with a field value matching one of the values given in an iterable. Example: `id__in=[1, 4, 6]` matches objects with an ID field of 1, 4, or 6.
`__lt`	Lesser than	Objects with a field lesser than the given value
`__lte`	Lesser than equal	Objects with a field lesser than or equal to the given value
`__startswith`	Starts with	Text field starts with the given value
`__istartswith`	Case-insensitive starts with	Text field starts with the given value, ignoring case
`__endswith`	Ends with	Text field ends with the given value
`__iendswith`	Case-insensitive ends with	Text field ends with the given value, ignoring case

Database dependencies

Not all field options work as advertised due to underlying database implementations. For strings containing ASCII, SQLite only does case-insensitive matching for sub-strings. For strings containing values outside the ASCII range, SQLite only does exact matching. Aren't computers fun?

For a full list of the peculiarities of Django interacting with your database of choice, see: https://docs.djangoproject.com/en/4.2/ref/databases/.

There are many more field lookups built into Django, including a set specific for querying dates and times. The whole list is available in the Django documentation: https://docs.djangoproject.com/en/4.2/ref/models/querysets/#field-lookups.

4.5 Modifying your data

You've created and queried data, but that's just not enough. Data changes over time. You need to be able to edit and delete data. To demonstrate, create another `Musician` object:

```
>>> Musician.objects.create(first_name="Joseph", last_name="Trude
<Musician: Musician(id=4, last_name=Trudeau)>
```

This new object has two problems: first my dad was born on the 23rd, not on the 22nd. Second, he doesn't qualify as a musician. I love him dearly, but his relationship to pitch is a challenging one. Deal with the first problem first. As I didn't store the result of the `.create()` in a variable, I need to run a query to get the object back. The `.get()` method of the query manager retrieves a single object, but only if there is a single match. Run a query using `.get()`, giving `id=4` and store that in a variable this time:

```
>>> dad = Musician.objects.get(id=4)
>>> dad
<Musician: Musician(id=4, last_name=Trudeau)>
```

With the object in hand, you can alter the attributes on the object. This on its

own is not sufficient, you are only changing the object in memory. The `.save()` method on your object executes the underlying SQL to sync your object up with the database. Change the `birth` value to be the 23rd, then call `.save()` to update the database:

```
>>> dad.birth
datetime.date(1938, 2, 22)
>>> dad.birth = date(1938, 2, 23)
>>> dad.save()
```

Querying the same object into a new variable proves the value in the database has changed:

```
>>> joseph = Musician.objects.get(first_name="Joseph")
>>> joseph.birth
datetime.date(1938, 2, 23)
```

Recall that `.get()` only returns a single match. If there is no match, or if there are multiple matches, `.get()` raises an exception. Try querying an `id` that doesn't exist yet, or `first_name="John"` to see the result:

```
>>> Musician.objects.get(id=42)
Traceback (most recent call last):
  File "<console>", line 1, in <module>
  File "RiffMates/venv/lib/python3.11/site-packages/django/db/mod
    return getattr(self.get_queryset(), name)(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "RiffMates/venv/lib/python3.11/site-packages/django/db/mod
    raise self.model.DoesNotExist(bands.models.Musician.DoesNotEx
>>> Musician.objects.get(first_name="John")
Traceback (most recent call last):
  File "<console>", line 1, in <module>
  File "RiffMates/venv/lib/python3.11/site-packages/django/db/mod
    return getattr(self.get_queryset(), name)(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "RiffMates/venv/lib/python3.11/site-packages/django/db/mod
    raise self.model.MultipleObjectsReturned(
bands.models.Musician.MultipleObjectsReturned: get() returned mor
```

As my dad himself will admit he doesn't belong in our musicians table, it is time to fix the second problem. The `.delete()` method on your object instance deletes the corresponding object from the database. Note that unlike the other database operations you've preformed, this one is directly on the

instance itself. Use the `joseph` value you queried before and delete it:

```
>>> joseph.delete()
(1, {'bands.Musician': 1})
```

The `.delete()` method returns a tuple containing the number of objects deleted and a dictionary with key/value pairs for the type and number of objects deleted. Most of the time you just ignore the response from `.delete()`.

Calling `.delete()` does not change your object, it only affects the database. You can still access all the values in `joseph`. In fact, you can create a new entry in the database with the same values as the old one by calling `.save()` on the object. The value of the unique ID field will change, but all the other fields would be back. To see the effect of delete on the database, run a query looking for my non-musician father and note the entry is no longer there.

Your users aren't going to have access to your database, or they better not, that is just asking for trouble. To display your data you need a view. The query methods you've learned so far can be implemented in views for display to your users.

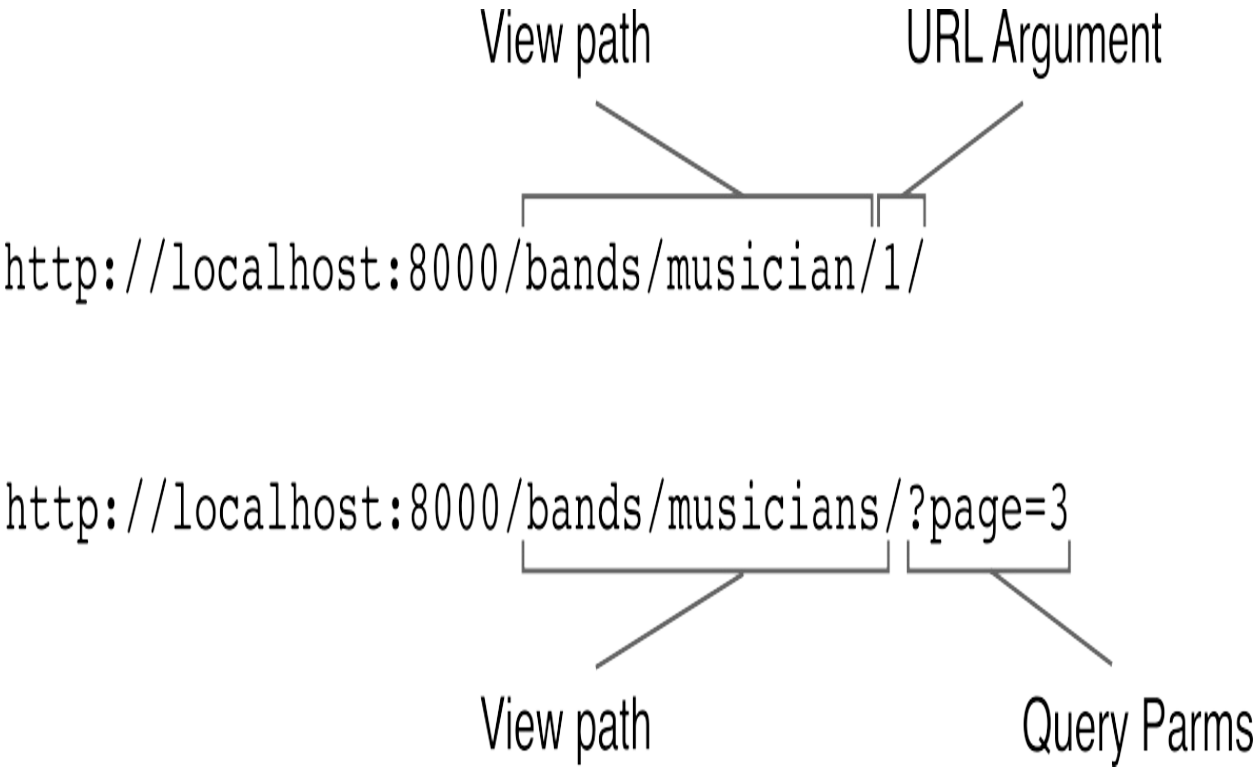
4.6 Querying models in views

Many of the views you want to write need to be parameterized. You wouldn't write a view for every musician in the database, instead, you want to write a single view that takes a musician as an argument. There are two ways to parameterize a view:

- URLs based arguments
- Query parameters

Figure 4.5 shows both types of parameterized URLs.

Figure 4.5. URL based arguments and query parameters



You may be more familiar with query parameters if you've ever seen key/value pairs in a URL after a question mark. Django supports these, but also allows you to capture parts of a URL as arguments to your view. Captured parts is the recommended way of using arguments with your URL, but query parameters still get used for optional behaviors in a view.

4.6.1 Using URL arguments

Let's start with Django's preferred case: URL based arguments. To build a page that displays information for a single musician, you need a way of specifying which musician. As your database already has a unique identifier for each musician, the typical solution to this problem is to use the database ID as the argument to the view.

A view to show musician details uses the ID of a `Musician` object as an argument. Remember that the URL `path` object is what defines the mapping between the URL in the browser and the called view. To support arguments to the view, you specify this in the `path` object declaration, indicating the arguments and their type using angle brackets. Inside the view, you perform a lookup of the `Musician` object for the given ID from the database. The

resulting object gets rendered through a template.

Django provides a shortcut named `get_object_or_404()` to help you fetch an object or error out if the argument is invalid. This shortcut looks takes two or more arguments. The first argument is the `Model` object to query, in this case the `Musician`. The second and subsequent arguments get used to form the look-up query. They use the same format as the query manager's `.get()` method, as that's what the shortcut is calling for you. The most common case is `id=object_id`, searching for a single object that has the corresponding ID, usually passed in as an argument to the view.

Why 404?

If the arguments to `get_object_or_404()` don't result in a single object, either the arguments match multiple arguments or one isn't found, the shortcut raises an `HTTP404` exception. Django automatically handles this, showing a 404 page instead of the result of the view.

This mechanism is rather important. Although your site probably won't show links with bad arguments, URLs are part of the public space: your users could type any argument at all. You should never trust your users, and your view should always be able to handle bad arguments. Raising an `HTTP404` error is a convenient way of short-circuiting your view and letting Django do the work.

Django provides a default, kind of plain 404 page, but you can customize your own for your site. See appendix B on using Django in production environments to learn how.

Once an object gets found, the rest of the view is similar to the views you've written before. You use the `render()` shortcut to render a template, passing in your object as context. Listing 4.4 is a view that looks up a `Musician` object by ID and renders RiffMates/templates/musician.html.

Listing 4.4. A view for displaying a `Musician` object based on an argument

```
# RiffMates/bands/views.py
from django.shortcuts import render, get_object_or_404 #1

from bands.models import Musician #2
```

```
def musician(request, musician_id):
    musician = get_object_or_404(Musician, id=musician_id) #3

    data = {
        "musician": musician, #4
    }

    return render(request, "musician.html", data)
```

Like your other views, this one needs to be registered. The `path` object is a little different from those you've seen before. This one includes a capture specifier to indicate that part of the URL is to be used as an argument. Capture specifiers are inside of angle brackets and consist of the type of the argument and the name used in the view. For example `<int:musician_id>` indicates the argument is an integer and is named `musician_id` in the view. All URLs are text, by including the data-type in the capture specifier, Django can automatically cast the given argument to your desired data-type. Django supports several capture types, the three most common are:

1. `str`—any non-empty string excluding the forward-slash (/) path separator
2. `int`—any integer zero or more
3. `slug`—a special string value consisting of letters, numbers, and hyphens

The capture specifier is part of the path-matching process. If your URL specifies an integer component, but your user sends in a non-integer, the path does not match. This means your view doesn't get called. Django treats it as a 404 without you needing to worry about it.

Up until now you have registered your URL `path` objects directly in the main urls.py file. It is actually good practice to associate the URLs for an app with the app. You can do this by creating a new URL-routing file in the app and then including its contents in the main file.

A URL pattern file in an app uses the exact same structure as the main file. Create `RiffMates/bands/urls.py` like you see in listing 4.5.

Listing 4.5. App specific URL pattern declaration file for bands

```
# RiffMates/bands/urls.py
from django.urls import path

from bands import views

urlpatterns = [
  path('musician/<int:musician_id>/', views.musician, name="music
]
```

Using separate URL-routing files for each app tends to simplify the code you need in the file. Without them, the main file needs to alias the view module as all apps have the same name for their views: views.py. Having a bands-app-specific URL file only uses the bands' app views.py file, so the alias is unnecessary.

Django does not automatically find and register app specific URL routing files, you need to register them. The `django.urls` module provides a function called `include()` that enables you to include all the URLs in a module. The `include()` function gets used as part of a `path` object, registering all the URLs in the included module under a path. This way, all the URLs in a module start with the same path prefix. To avoid the need to import many modules, the `include()` function takes a string that names the module whose content is to be included. Open RiffMates/RiffMates/urls.py and make the changes you see in listing 4.6.

Listing 4.6. Register all the URL routes in the bands app

```
# RiffMates/RiffMates/urls.py
...
from django.urls import path, include #1
...
urlpatterns = [
    ...
    path('bands/', include("bands.urls")), #2
]
```

When adding more views to the `bands` app, you now only need to register the URL in RiffMates/bands/urls.py. Any new URL routes there are included under `bands/` through the above change to the main route file.

To complete your view you'll need a template. Recall from listing 4.2 that the

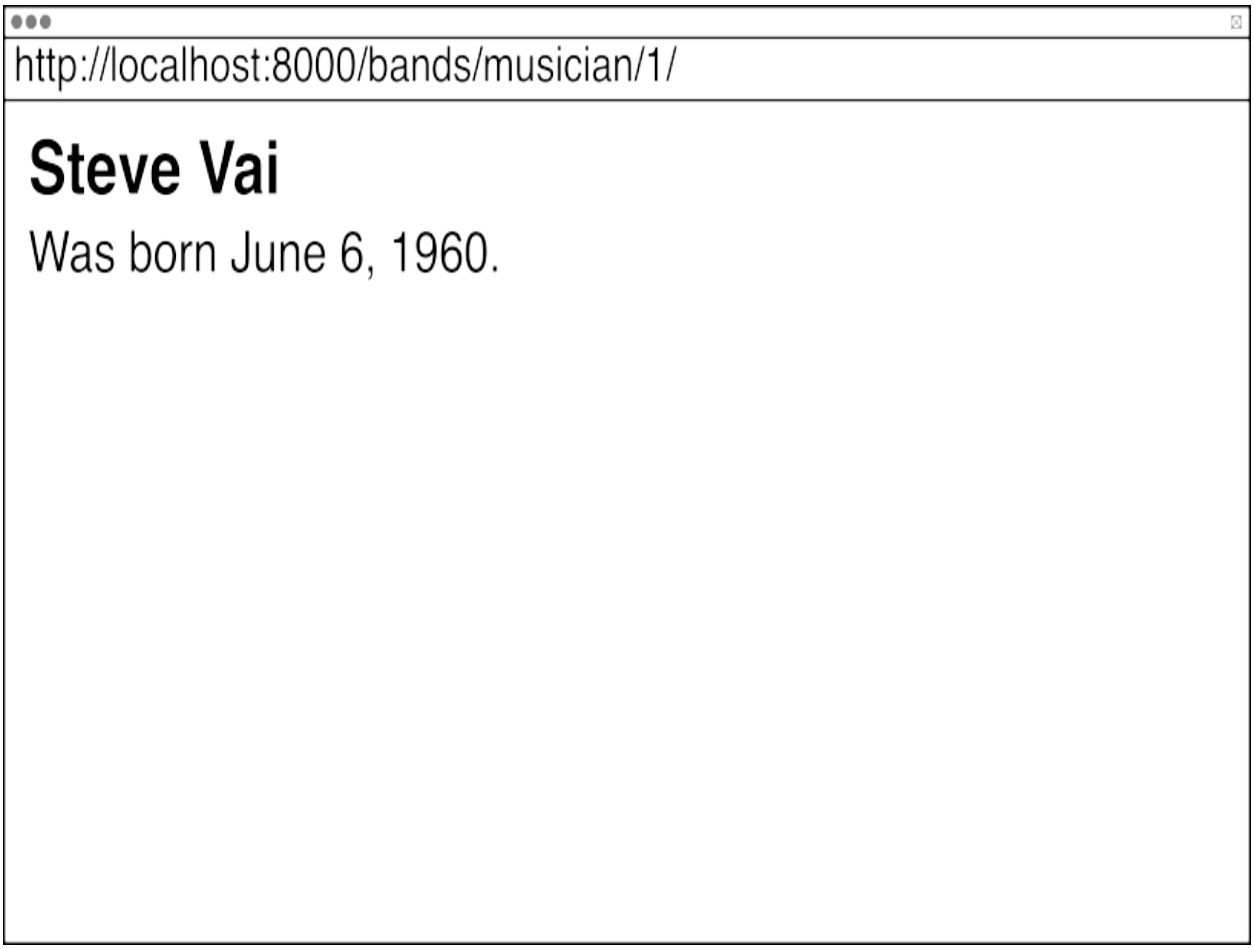
`render()` shortcut passed in `musician.html` as the template to be rendered. Feel free to be more creative, listing 4.7 contains a bare minimum template. Create a new file named RiffMates/templates/musician.html for this content.

Listing 4.7. Template for musician detail information

```
<!-- RiffMates/templates/musician.html -->
{% extends "base.html" %}

{% block title %}
  {{block.super}}: Musician Details
{% endblock %}

{% block content %}
  <h1>{{musician.first_name}} {{musician.last_name}}</h1>

  <p> Was born {{musician.birth}}. </p>
{% endblock content %}
```

Your new view is ready to go. Fire up the old development server and visit the URL: `http://localhost:8000/bands/musician/1/` to see Steve Vai's musician details page. It will look something like 4.6, depending on how seriously you took the challenge to be creative.

Figure 4.6. Musician view showing Steve Vai



By editing the URL and changing the numeric value, you can query different musicians in the database. Shortly, you'll be creating a listing page that has links to many musicians, but you want to make sure the detail pages work before you get there. Test a few different `Musician` ID values.

You should also check an ID value that doesn't exist, first to ensure that the error handling code is working, and second so you can experience the wonder that is a Django error page. When your code raises an `HTTP404` exception, Django does one of two things, depending on the value of `DEBUG` in your configuration RiffMates/RiffMates/settings.py file. In a production environment, where `DEBUG=False`, Django shows a generic 404 page with very little information on it. You don't want to expose the details of your code to the untrustworthy public.

In a development environment, where `DEBUG=True`, Django shows you a debugging error page, complete with information on what went wrong. In the

case of a bad `Musician` ID, the error is: "No Musician matches the given query." The error page also includes a list of all the URLs Django attempted to match. This can be handy if you're trying to figure out why the URL you typed is causing a 404 error. In the case of a 404 raised by `get_object_or_404()`, you'll see the URL that got matched, but that the argument wasn't valid. Figure 4.7 contains the debug error page for a musician ID that doesn't exist in my database.

Figure 4.7. Page not found as no musician exists with ID 100

http://localhost:8000/bands/musician/100/

Page not found (404)

No Musician matches the given query.

Request Method: GET
Request URL: http://127.0.0.1:8000/bands/musician/100/
Raised by: bands.views.musician

Using the URLconf defined in RiffMates.urls, Django tried these URL patterns, in this order:

1. admin/
2. credits/ [name='credits']
3. about/
4. version/
5. news/ [name='news']
6. adv_news/ [name='adv_news']
7. bands/ musician/<int:musician_id>/ [name='musician']

The current path, bands/musician/100/, matched the last one.

You're seeing this error because you have DEBUG = True in your Django settings file. Change that to False, and Django will display a standard 404 page.

Getting a 500 error

Pointing to a bad URL, or passing invalid arguments to one, results in a 404 page. Coding errors result in a 500 error. If `DEBUG=True`, the 500 error page is similar to the 404, and includes a stack-trace of your code showing the offending line, and a variety of information about the calling state.

Like with the 404, in production mode the 500 error is generic, showing very little information, and it also can be customized for your application.

The visitors to your site probably aren't interested in guessing the ID of the musician they're looking for. In addition to your musician's details page, you're going to want some sort of index listing as well.

4.6.2 Listing pages

A view doesn't have to take arguments to take advantage of the database. An index listing page could query all, or some, of the objects in the database and display them to the user. Each object in the result can then link to the corresponding detail page.

It is common practice to have the object page named in the singular and the listing page in the plural. The `musician` view you wrote before displays a single musician. The corresponding listing view would be called `musicians`, plural. The database query inside the `musicians` view looks for all the `Musician` objects in the database. To make the listing easier for the user, the query for all objects can be modified using the `.order_by()` call. This is called an *annotation* and is one of many that allow you to change the behavior of a query or add information to the resulting objects. To have the query order musician objects by last name, use `.order_by("last_name")` on the query.

The entire query result gets passed from the view to the template. The template then iterates over the result displaying a URL for each object. Recall that the `{% url %}` tag returns the URL corresponding to a name that you gave the `path` object when registering the route. Way back in listing 4.5, the `path` registering the `musician` view got named `musician`, so it can be rendered through the `{% url %}` tag. In addition to taking the name of a URL,

the tag also takes any arguments. For the `musician` view, the argument is the musician's ID, which is accessed as an attribute of the object.

Edit RiffMates/bands/views.py and add the code for the `musicians` view as seen in listing 4.8.

Listing 4.8. Index page for `Musician` objects

```
# RiffMates/bands/views.py
...
def musicians(request):
    data = {
        'musicians':Musician.objects.all().order_by('last_name'),
    }

    return render(request, "musicians.html", data)
```

You'll need to register the route for your new view. As you now have a URL file specific to the bands app, that is the best place to register the URL. This time the URL has no argument, so you're back to the simpler form. Edit RiffMates/bands/urls.py and add the new `path` object like in listing 4.9.

Listing 4.9. Adding the `musicians` index view to the bands app URL file

```
# RiffMates/bands/urls.py

urlpatterns = [
...
    path('musicians/', views.musicians, name="musicians"),
]
```

Listing 4.10 contains a simple template for your view. Create RiffMates/templates/musicians.html and add content similar to the listing's.

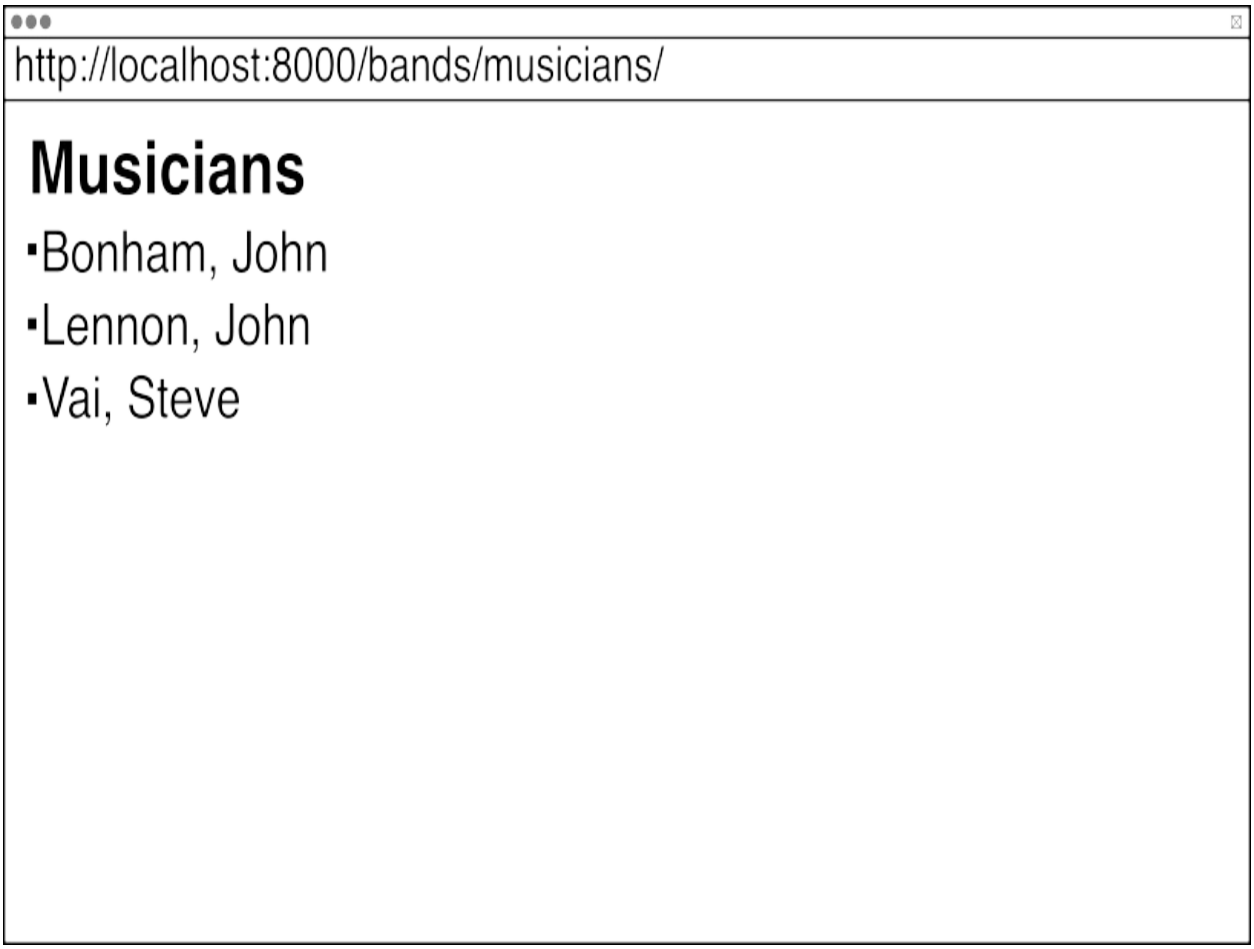
Listing 4.10. Template for the `musicians` view

```
<!-- RiffMates/templates/musicans.html -->
{% extends "base.html" %}

{% block title %}
  {{block.super}}: Musician Listing
{% endblock %}
```

```
{% block content %}
  <h1>Musicians</h1>

  <ul>
    {% for musician in musicians %}
      <li> <a href="{% url 'musician' musician.id %}"> #1
        {{musician.last_name}}, {{musician.first_name}}
      </a> </li>
    {% empty %}
      <li> <i>No musicians in the database</i> </li>
    {% endfor %}
  </ul>

{% endblock content %}
```

With the view written, the route registered, and a template defined, you're all set to visit your musician's listing page. Start your development server and go to `http://localhost:8000/bands/musicians/`. The output should be similar to figure 4.8.

Figure 4.8. Musician listing page



When you have more content in your database, the listing page is going to become cumbersome. There are a few ways of dealing with this, one is to limit the number of results in a page and support multiple pages.

4.6.3 Using query parameters for pagination

It is bad practice to support an unlimited number of objects in a query. Not only does this become information overload for your users, but it can be detrimental to your database. Django provides a utility to carve your queries up into pieces allowing you to show a subset on the page. The `Paginator` class takes an iterator of objects, and a limiter. The iterator can be a `QuerySet`, so you can pass the result of an object query directly to a `Paginator`. The limiter specifies the maximum number of objects per page.

Once you have a `Paginator` instance, you access page objects inside it. Each page object indicates the items on the page and whether there are pages

before or after the page. This allows you to build a template with *previous* and *next* links to continue in the pagination.

The number of the current page in a pagination sequence is a perfect use of URL query string parameters. By using a URL parameter rather than an embedded argument, you can specify default behavior more easily.

Query parameters are accessible as part of the `HTTPRequest` object that is the first argument of every view. Your request contains a special dictionary named `GET`. This dictionary is actually an instance of a Django class called `QueryDict` but for our purposes you can treat it like any other dict. For any query parameters in the URL there is a corresponding key/value pair in the `GET` dict.

To add pagination to the `musicians` view, the view needs to:

- Add a `Paginator` object populated by the musicians listing
- Check for a page number query parameter
- Handle out of range page numbers
- Pass the current page to the template for rendering

The code in listing 4.11 is the new version of the `musicians` view, applying all of these features.

Listing 4.11. Musicians listing view supporting pagination

```
# RiffMates/bands/views.py
from django.core.paginator import Paginator #1
...
def musicians(request):
    all_musicians = Musician.objects.all().order_by('last_name')
    paginator = Paginator(all_musicians, 2) #2

    page_num = request.GET.get('page', 1) #3
    page_num = int(page_num) #4

    if page_num < 1: #5
        page_num = 1
    elif page_num > paginator.num_pages: #6
        page_num = paginator.num_pages
```

```
    page = paginator.page(page_num) #7

    data = {
        'musicians':page.object_list, #8
        'page':page, #9
    }

    return render(request, "musicians.html", data)
```

The template for this view must change as well. You need to add links at the bottom for any "previous" and "next" pages. The `page` object passed into the template has methods to help you: `.has_other_pages()`, `.has_next()`, `.has_previous()`, and `.next_page_number()`. Using these inside of conditional blocks you can appropriately render links that point to other pages in the pagination result. Remember that object methods get accessed the same way as attributes in the template language, you won't need any parenthesis.

Listing 4.12 shows the conditional block that goes on the bottom of RiffMates/templates/musicians.html. As the `musicians` view passes in `pages.object_list` as `musicians` in the context, the rest of the page can remain unchanged.

Listing 4.12. Musicians listing view supporting pagination

```
<!-- RiffMates/templates/musicans.html -->
<!-- ... -->

  {% if page.has_other_pages %}
    {% if page.has_previous %}
      <a href="{% url 'musicians' %}?page={{page.previous_page_nu
        Prev</a>    
    {% endif %}
    {% if page.has_next %}
      <a href="{% url 'musicians' %}?page={{page.next_page_number
        Next</a>
    {% endif %}
  {% endif %}
```

With the changes in place, you have a paginated version of your musicians listing page. Run your development server and re-visit the page. Try the URL with and without the `page` query parameter. You may also want to modify the

`Paginator` object, changing how many items per page to see how the page changes.

Musicians are great, but to make a great tune you need to group them together. To capture a band in your database you need to create relations between musicians.

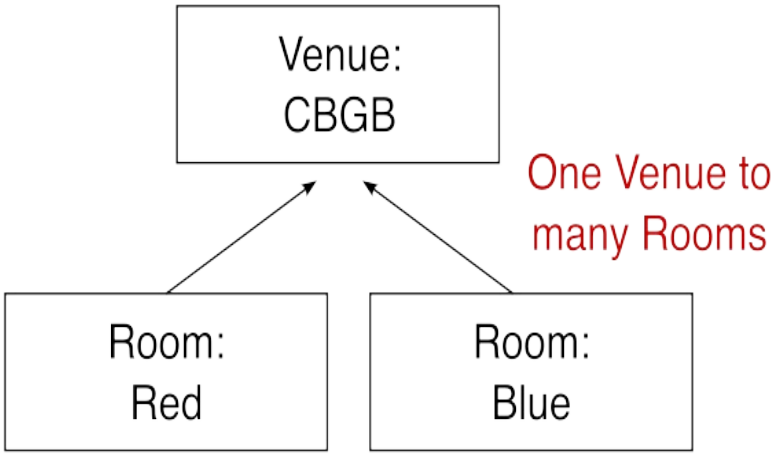
4.7 Model relations

A single object in the database will only get you so far. The "relational" in "relational database" is all about the connections between objects. There are two ways to relate objects to each other in a database. The first is to add a relationship column to your object table. The relationship column contains the ID of the related object. This is known as a *foreign key*. Foreign key relationships are one-to-many. This means multiple things can be related, but they are related through a single object. A venue where bands play can have multiple rooms each with their own schedule, but each room can only be associated with a single venue. That’s one venue and many rooms, for a one-to-many relationship.

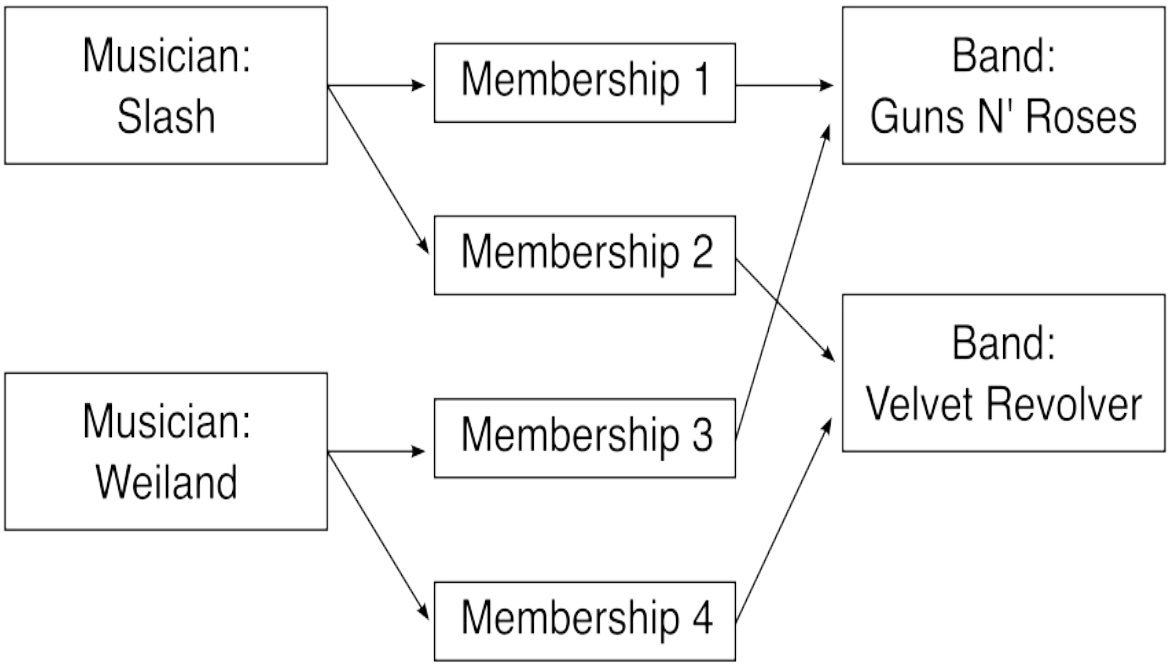
The second way to inter-relate objects in a database is to add a new table. The new table has the sole purpose of specifying object-to-object relationships. This kind of table has three columns: a unique ID, a foreign key to the first kind of object, and a foreign key to the second kind of object. Each row in the database expresses the relation between two objects. This allows for many-to-many relationships. As musicians are a fickle bunch, they might belong to more than one band at a time. To express this relationship in the database you need a dedicated table that associates a musician with a band. To associate a musician with multiple bands you need multiple rows that contain the musician’s ID and each related band.

Figure 4.9. One-to-many vs many-to-many relationships

One-to-Many Relations



Many-to-Many Relations



Many Musicians to many Bands
through a relationship table

Django supports both foreign key and many-to-many relationships. It does this through `Model` fields named `ForiegnKey` and `ManyToManyField`.

4.7.1 One-to-many relationships

A foreign key relationship can feel a little backwards. In writing, you might describe a venue containing multiple rooms. You're less likely to say there are multiple rooms all belonging to the same venue, it just doesn't flow as well. But, that's how you do it in a database. This all comes down to which table contains the relationship column. If you put a "rooms" column on the venue table, you can only associate each venue with a single room. Remember, each row in the venue table is for a single venue. If instead, you put a "venue" column on the room table, each row describing a room can point at a single venue. This allows a venue to be associated with many rooms, but a room only to be associated with a single venue.

Expressing a foreign key relationship in Django requires the addition of a field to your `Model`. The `ForeignKey` field requires at least two arguments. The first is a reference to the related `Model` object. This reference can either be the `Model` class itself, or optionally a string naming the class. The string format is useful to avoid having to import models from other modules and to solve potential circular import problems. If the relationship is between models in the same file, you can use the class itself. If the relationship is between models in different files (and likely different apps), you should use the string specifier.

The second argument to the `ForeignKey` field is the `on_delete` parameter. In older versions of Django this was optional, but it is now a required argument. This parameter specifies how Django is to behave if the associated object gets deleted. The choices for `on_delete` are a series of constants defined in the `django.db.models` module. The two most common choices are:

- `CASCADE`—the associated object also gets deleted
- `SET_NULL`—the field to NULL

Other choices are available as well, including being able to define a function that is called when the model gets loaded. See

https://docs.djangoproject.com/en/4.2/ref/models/fields/#django.db.models.Fo for a complete listing. Up until now, all the fields you defined were required fields. In order to use `on_delete=SET_NULL`, the field must be allowed to contain a NULL value. To do this, pass `null=True` to the field constructor. Without it, the default of `null=False` is assumed.

Django comes with a web-based tool for managing the objects in your database called the Django Admin. You can create objects in the tool, including specifying inter-object relationships. The Django Admin allows you to define if the UI permits a relationship to be empty. To give you fine-grained control, Django distinguishes between NULL in the database and an empty value in the Django Admin. To allow empty values you pass `blank=True` to the field constructor. You'll learn more about the Django Admin and these controls in a later chapter.

Let's add two new models to *RiffMates*: one for our venues and one for the rooms in those venues. Both the venues and the rooms need to have names, and additionally, the room needs to specify its associated venue. The `Room` model uses a `ForeignKey` field to create this relationship. If a venue gets removed from the database, it doesn't make sense to keep the associated rooms, so the `on_delete` value for the `Room` is `CASCADE`. Open RiffMates/bands/models.py and add the `Venue` and `Room` models shown in listing 4.13.

Listing 4.13. `Venue` and `Room` models and their one-to-many relationship

```
# RiffMates/bands/models.py
...
class Venue(models.Model):
    name = models.CharField(max_length=20)

    def __str__(self):
        return f"Venue(id={self.id}, name={self.name})"


class Room(models.Model):
    name = models.CharField(max_length=20)
    venue = models.ForeignKey(Venue, on_delete=models.CASCADE) #1

    def __str__(self):
        return f"Room(id={self.id}, name={self.name})"
```

Remember that a Django `Model` is only a proxy to the table in the database, just because you updated your code doesn't mean the database has magically changed. To sync your code with the database you need to create a new migration file. This is done using the `makemigrations` management sub-command:

```
(venv) RiffMates$ python manage.py makemigrations bands
```

The output from the command tells you what was done:

```
Migrations for 'bands':
  bands/migrations/0002_venue_room.py
    - Create model Venue
    - Create model Room
```

With the new migration file in place, you apply the changes using the `migrate` management sub-command:

```
(venv) RiffMates$ python manage.py migrate
```

Running `migrate` causes the changes to be synchronized to the database. The output from the command details which migrations got performed:

```
Operations to perform:
  Apply all migrations: admin, auth, bands, contenttypes, session
Running migrations:
  Applying bands.0002_venue_room... OK
```

With the database ready, you can go into the Django REPL and create some objects. You want to create a `Venue` instance first, so it is available to be passed into any `Room` objects. As `Room` does not specify the `null` argument, the default value of `False` is used. A `Room` can't be NULL and must be associated with a `Venue`. The following REPL session creates a `Venue` and two associated `Room` objects. Don't exit it yet, you'll be doing a bit more in a second.

```
(venv) RiffMates$ python manage.py shell
Python 3.11.1
Type "help", "copyright", "credits" or "license" for more informa
(InteractiveConsole)
>>> from bands.models import Venue, Room
>>> cbgb = Venue.objects.create(name="CBGB")
```

```
>>> red = Room.objects.create(name="Red", venue=cbgb)
>>> blue = Room.objects.create(name="Blue", venue=cbgb)
```

Each instance of your `Room` object can directly access its associated `Venue` object. Using dot-notation you can get at the properties of the `Venue`:

```
>>> red.venue
<Venue: Venue(id=1, name=CBGB)>
>>> red.venue.id
1
>>> red.venue.name
'CBGB'
```

What should be unique?

The relationship between the `Venue` and `Room` objects is based on their ID values, which are automatically generated by the database. Two different venues could both have rooms named "Red" without a problem as the ID is what determines the relationship, not the value of the `name` field.

In fact, there is nothing currently stopping you from creating two `Room` objects with the same name and pointing them both at the same `Venue`. Django provides mechanisms for controlling this which are discussed in a later chapter.

In your best Ron Popeil voice, read the next sentence out loud. But wait, there's more! In addition to the forward relationship being accessible, you can go backwards as well. The object that a `ForeignKey` field points to automatically gets a query manager to access the associated objects. The query manager is named after the associated model with `_set` stuck on the end. Your `Venue` model has a `room_set` query manager. All the queries you learned to use on the `.objects` query manager apply to this one, the difference being the objects queried are only those associated through a foreign key. Run the `.all()` query on the `Venue` instance's `room_set`:

```
>>> cbgb.room_set.all()
<QuerySet [<Room: Room(id=1, name=Red)>, <Room: Room(id=2, name=B
```

Because `room_set` is an attribute, and queries are callables, you can pass a `Venue` object into a template and iterate over the associated rooms using `{%`

`for room in venue.room_set.all %}`. In fact, one of the exercises in this chapter asks you to do just that.

You can go a long way with one-to-many relationships, but sometimes they're just not enough. Musicians can't have their creative freedom boxed in, man, they have to be free! To allow musicians to be promiscuous in their choice of bands, you need many-to-many relationships.

4.7.2 Many-to-many relationships

Foreign keys in a database are a column that contains the ID of a row either in the same or a different table. A `ForeignKey` field in Django constructs one of these columns on the table corresponding to your `Model`. As the column is on the `Model`, you can at most create a one-way relationship. To create a many-to-many relationship you need an entirely separate table. Technically, these tables contain foreign keys, but in Django you don't use a `ForeignKey` field to construct them. In fact, you don't have to construct the table at all. The `ManyToManyField` is used to create many-to-many relationships. This field is a little different from the other fields you've used so far. Every other field results in a single column on the object's table. The `ManyToManyField` doesn't result in a new column but in a new table.

In order to be able to relate musicians to bands (both plural), you need a many-to-many relationship. The `ManyToManyField` can go on either the `Musican` or the `Band` class. When Django sees this kind of field it creates a relationship table with three columns: a unique ID for the relationship, and a column for each of the two keys of the related objects. The table is named based on the app and the two models participating in the relationship.

Besides establishing my age and musical preferences, figure 4.10 shows three tables for: musicians, bands, and their many-to-many relationship. Famous guitarist Slash (id=62) is a member of both Guns N' Roses (id=13) and Velvet Revolver (id=54). Singer Scott Weiland (id=81) is a member of both Stone Temple Pilots (id=27) and Velvet Revolver (still id=54). The `Band_Musicians` table is how the band memberships get expressed. There is a row for Slash and Guns N' Roses (id=36); a row for Slash and Velvet Revolver (id=63); a row for Scott and STP (id=45); and for Scott and Velvet

Revoler (id=64).

Figure 4.10. Database tables for many-to-many relationship between bands and musicians

Band Table

ID	Band	Founded
13	Guns N'Roses	1985
⋮		
27	Stone Temple Pilots	1989
⋮		
54	Velvet Revolver	2002

Unique Band IDs

Musicians Table

ID	Musician	Birth Date
61	Axl Rose	1962-02-06
62	Slash	1965-07-23
⋮		
81	Scott Weiland	1967-10-27

Unique Musician IDs

Musician Slash (13) is a member of both Guns N' Roses (13) and Velvet Revolver (54)

Band_Musicians Table

ID	Band_ID	Musician_ID
35	13	61
36	13	62
⋮		
45	27	81
⋮		
63	54	62
64	54	81

Unique Relationship IDs

Musician/Band Memberships

As a many-to-many relationship is two-way, you can put the `ManyToManyField` on either of the two participating objects and get a similar result. Semantically though, it tends to be clearer if the object representing a conglomerate is where the relationship gets placed. A band is a group of people, so it feels clearer if the `ManyToManyField` is on the `Band` object. Besides the relationship, the `Band` model also needs a name field. Open RiffMates/bands/models.py and add the code for the `Band` object shown in listing 4.5.

Listing 4.14. `Venue` and `Room` models and their one-to-many relationship

```
# RiffMates/bands/models.py
...
class Band(models.Model):
    name = models.CharField(max_length=20)
    musicians = models.ManyToManyField(Musician)

    def __str__(self):
        return f"Band(id={self.id}, name={self.name})"
```

As with earlier changes to Django `Model` classes, you need to run the `makemigrations` and `migrate` management sub-commands to sync the database with your `Model` code:

```
(venv) RiffMates$ python manage.py makemigrations bands
Migrations for 'bands':
  bands/migrations/0003_band.py
    - Create model Band
(venv) RiffMates$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, bands, contenttypes, session
Running migrations:
  Applying bands.0003_band... OK
```

`ForeignKey` fields get populated as an attribute when you create an object, like you did passing a `Venue` into your `Room` object. In a many-to-many relationship, the relation is in a separate table, meaning the relation needs to be created as a separate step. Before doing anything else, you need a `Band` object, enter the Django REPL and create it as follows:

```
>>> from bands.models import Band
>>> beatles = Band.objects.create(name="The Beatles")
```

To add Mr. Lennon to The Beatles, you need a reference. You can query him based on his ID if you remember it, or as his last name is currently unique, you can run a `.get()` query based on that:

```
>>> from bands.models import Musician
>>> lenon = Musician.objects.get(last_name="Lennon")
>>> lenon
<Musician: Musician(id=2, last_name=Lennon)>>>>
```

Now that you have a reference to a musician and a band, you can associate the two together. The `Band` class has a query manager named after the `ManyToManyField`: `musicians`. The `.add()` method on the `musicians` query manager adds a `Musician` to the `Band`:

```
>>> beatles.musicians.add(lenon)
```

As `musicians` is a query manager, you can run queries on it. Running `.all()` returns all the `Musician` objects currently associated with the `Band`. In this case, that's only Mr. Lennon:

```
>>> beatles.musicians.all()
<QuerySet [<Musician: Musician(id=2, last_name=Lennon)>]>
```

But wait, there's more! Yep, still more. Sort of the same kind of more. You can also go backwards. The `Musician` class now has a query manager named `band_set`. Using this query manager you can determine which bands a given musician belongs to:

```
>>> lenon.band_set.all()
<QuerySet [<Band: Band(id=1, name=The Beatles)>]>
```

Because this is a many-to-many relationship, musicians can be associated with multiple bands. Let's create a super-group trio band named "Wishful Thinking" and add all three of our musicians:

```
>>> vai = Musician.objects.get(last_name="Vai")
>>> bonham = Musician.objects.get(last_name="Bonham")
>>> wishful = Band.objects.create(name="Wishful Thinking")
>>> wishful.musicians.add(lenon, vai, bonham)
>>> wishful.musicians.all()
<QuerySet [<Musician: Musician(id=1, last_name=Vai)>, <Musician:
```

Recall that with the `ForeignKey` field, you need to be explicit about the behavior when a related object gets deleted. Deleting a `Venue` without deleting the corresponding `Room` objects doesn't make a lot of sense, so you `CASCADE`. With a many-to-many relationship this problem doesn't exist. If you wanted to delete Mr. Lennon, Django takes care of removing any associated `Band` relationships. Note that this does not remove the `Band`. If you delete all the members of "Wishful Thinking", you will still have a "Wishful Thinking" `Band` object, it just won't have any related `Musicians`. If you are trying to remove all the musicians and their associate band, you need to make separate explicit delete calls on both object types.

4.8 Model Fields

Django 4.2 ships with 28 data fields and 3 relationship fields. Some fields map directly to an SQL data-type, while others are a convenient abstraction that add features on top of the SQL data-type. For example, the `EmailField` is usually implemented as text in the underlying database, but also provides validation to make sure that the text input meets the standard format of an e-mail address.

When you define a field, you can pass in arguments that control the construction of the underlying column. You saw this with `max_length` on a `CharField`. Some arguments can be used with all fields, and some are field-specific. This section highlights frequently used fields and their arguments. For a complete listing, see the Django Model field reference document: https://docs.djangoproject.com/en/4.2/ref/models/fields/.

4.8.1 Popular Cross-field Field Options

When creating the `ForeignKey` field that connected the `Venue` and `Room` objects, you used the `null` and `blank` field options. Both of the options are available across all fields.

The `null` option specifies whether the column in the database can contain the NULL value. This is a boolean option, and defaults to `False`. NULL in SQL indicates a lack of a value. Depending on the column this can get a little confusing. A `DateField` configured with `null=True` has two types of state:

empty and containing a date value. By contrast, a `CharField` with `null=True` has three types of state: empty (NULL), empty string (`''`), and containing a string. Generally it is advised to use an empty string with text and leave `null=False`.

Closely related to `null` is the `blank` option. The `null` option specifies the behavior in the database, whereas the `blank` option specifies the behavior in the Django Admin and web forms. A value of `blank=False` (the default), means that the Django Admin and web based forms treat this field as required. It is possible to have `blank=True` and `null=False` if you are programmatically populating a field between form submission and writing the value to the database. This combination is unusual though, typically the values of these two options are the same.

The separation between `null` and `blank` exposes a design feature of Django, the differentiation between database and model validation. The fields in your `Model` may validate their data. `Model` level validation happens before values get saved to the database. This is done both to help ensure the content can be saved to the database and to add additional validation for some fields. The `EmailField` example mentioned earlier does this. `Model` level validation ensures that the content meets the format for an email address, whereas the database validation may only care if the content is text. How this is implemented may vary between databases. If your database provides an email datatype, Django generally will take advantage of it. If it does not, it will use model validation and then store as text.

The `choices` field configuration option is a `Model`-level validation that provides a set of values that the field can contain. This option takes an iterable of pairs, usually written as a list of tuples. Each pair specifies the value stored in the database and a display string to go with it. For example, a field containing a country-code could specify: `choices=[ ("US", "United States"), ("CA", "Canada"), ("MX", "Mexico")]` if you only supported countries in North America. When this field gets shown in the Django Admin or in a web form, a `<select>` drop-down box gets used, with the full string of the Country's name visible to the user and the resulting value for the `<select>` being the corresponding two-letter country code.

It is common practice to build an attribute into your `Model` class that specifies

the choices value. An `Address` class could contain a `COUNTRY_CHOICES` attribute specifying the previous example's listing. The `choices` option would then point to the attribute. Doing it this way makes the list of choices available anywhere the model gets imported. There are third party libraries that define tuple sets that are frequently used. For example, the "django-localflavor" (https://pypi.org/project/django-localflavor/) package includes sub-modules for many countries and choices-compatible listings for states and provinces within them.

Databases improve look-up performance through the implementation of a special data structure known as an *index*. Indexes are similar to dictionaries in that they provide a quick way of looking up a given value based on a key, rather scanning an entire list. The ID field of your Django `Model` is indexed by default. This ensures that `.get(id=obj_id)` performs quickly. If a field is going to be frequently used to look a row up, that field may need an index. For example, if you add search capabilities to your site where a musician can be found by last name, adding an index to the `last_name` `CharField` will make a significant performance improvement. The `db_index` field option is a boolean, setting it to `True` causes Django to add an index on the corresponding column.

There are times when you want the value of a field to be unique within a table. Django provides several field options to enforce this: `unique`, `unique_for_date`, `unique_for_month`, and `unique_for_year`. The `unique` field operates at both the `Model` validation level and applies a constraint to the database. When `unique=True`, only one object in the table will be allowed to have the value found in that field. Any attempt to save another object with the same value will result in an exception. Django automatically applies an index to this column as it needs to look values up on every save of an object to ensure uniqueness. The `unique_for_date`, `unique_for_month`, and `unique_for_year` options are special cases for `DateTimeFields`. The uniqueness constraint gets applied to the date, month, or year portion of the field, while the other parts (for example the timestamp) are not considered.

4.8.2 Popular Fields

The field you've used so far is probably the most popular one out there:

`CharField`. There are several variations on this field that are essentially text storage mechanisms but with different validation and default values. The `EmailField`, `URLField`, and `SlugField` all inherit from `CharField`, adding validation for email addresses, URLs, and slug identifiers (mixes of letters, numbers and dashes, often used as part of a URL). The `EmailField` defaults to a max length of 254, the `URLField` to 200, and the `SlugField` to 50. Validation for these three variants almost always get done at the `Model` level unless the underlying database has a special storage type for them.

A close relative of the `CharField` is the `TextField`. A `CharField` requires a fixed amount of allocated space, whereas the `TextField` is an open block. This variation is due to an implementation detail at the database level. Some databases implement both of these fields as the same datatype, while others treat them separately. In the Django Admin or in web forms, the `CharField` is presented as an entry widget, while the `TextField` is presented as a multi-line text widget. This is default behavior and is under your control if you wish. You'll learn more about this in the chapter on forms and user input.

Django provides a wide variety of numeric fields, but they all boil down to either integer or float storage. The `IntegerField` and `FloatField` are the base mechanism for storing ints and floats, while variations such as `BigIntegerField` and `PositiveSmallIntegerField` change the validation and possibly the amount of storage required by the data. The latter is database dependent.

In the chapter on user data you will learn about file uploads. These use a field that holds a file path with file management on your server. The general version of this is the `FileField` while the `ImageField` adds validation that the uploaded content was a recognized image format.

One of the more recent additions to the list of fields is the `JSONField`. Although you could store JSON as text generally, some databases have implemented a datatype that is JSON aware. The advantage of this field is you access the corresponding attribute as if it is a Python object. Under the covers, the field automatically serializes the object into JSON and stores the result as encoded text. There are limitations to which database and database version support this field, and any Python object in the field must be serializable to the JSON format. SQLite is compatible with the `JSONField` if

a particular extension is enabled. The extension is called JSON1 and whether it is on by default depends on the SQLite distribution and the operating system you're working on. See https://code.djangoproject.com/wiki/JSON1Extension for information on how to determine if you can use this extension in your environment.

4.9 Fixtures

Manually entering data into your system is tedious. Django provides two management sub-commands that let you import and export data. Django calls the serialized data a *fixture*. Fixtures come in a variety of formats, including JSON, JSONL, and XML. You can also use YAML if you have the PyYAML library installed.

A fixture is a text version of the data you have in your database. This includes all the information needed to create the exact state of the tables involved. Object IDs are in the fixture, both as primary keys of an object and to express inter-object relationships. The `dumpdata` management sub-command outputs the fixture data to the screen in JSON format by default. The command optionally takes a list of apps whose data you are interested in. As the data is in JSON, the `json.tool` module built into Python can be handy, running it directly through `python -m` pretty-prints the output. The following line dumps the band app to the screen in JSON, pretty printed:

```
(venv) RiffMates$ python manage.py dumpdata bands | python -m jso
```

The output is a JSON representation of all the musicians, bands, venues, and rooms you created in the database. The content is serialized as a JSON list. The following shows two portions of the output, the serialized Steve Vai `Musician` object and the serialized "Wishful Thinking" `Band`:

```
[ #1
    {
        "model": "bands.musician", #2
        "pk": 1, #3
        "fields": { #4
            "first_name": "Steve",
            "last_name": "Vai",
            "birth": "1960-06-06"
```

```
        }
    },
...
    {
        "model": "bands.band",
        "pk": 2,
        "fields": {
            "name": "Wishful Thinking",
            "musicians": [1, 2, 3 ] #5
        }
    },
...
]
```

The companion to `dumpdata` is intuitively named `loaddata`. You specify what data to load through a filename or an app name. Dump the data again, this time without the pretty-printing, and redirect the content into a file named bands.json:

```
(venv) RiffMates$ python manage.py dumpdata bands > bands.json
```

This next bit can be a little nerve wracking. It is time to delete your database. As the fixture stores exactly what is in the database, loading it will overwrite the content that is there. You won't see any difference. If you delete your database (or rename it, if that makes you more comfortable), you can then use the fixture to re-create it.

Deleting your database sets you to zero though. Those migrations you ran won't exist. After deleting your database, but before loading the fixture, you need to run the `migrate` sub-command. The following sequence removes and then restores your database from the fixture:

```
(venv) RiffMates$ $ rm db.sqlite3
(venv) RiffMates$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, bands, contenttypes, session
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
```

```
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying bands.0001_initial... OK
  Applying bands.0002_venue_room... OK
  Applying bands.0003_band... OK
  Applying sessions.0001_initial... OK
(venv) RiffMates$ python manage.py loaddata bands.json
Installed 8 object(s) from 1 fixture(s)
```

Your data has been restored. Poke around in the Django REPL to prove this to yourself.

Fixtures are commonly used during testing. It is good practice to know the state of a database before performing a test, that way you can determine whether the result was what you expected. To standardize this process, you can associate fixture files with an app. You do this by creating a `fixtures` directory in the app. In this case, instead of passing a filename to the `loaddata` sub-command you specify the name of the app. Create a fixtures directory in the bands app and move bands.json inside of it. With that done, you can delete your database again, and repeat the restoration process. This time, use the app's name instead the filename directly:

```
(venv) RiffMates$ mkdir bands/fixtures #1
(venv) RiffMates$ mv bands.json bands/fixtures/ #2
(venv) RiffMates$ rm db.sqlite3
(venv) RiffMates$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, bands, contenttypes, session
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
```

```
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying bands.0001_initial... OK
  Applying bands.0002_venue_room... OK
  Applying bands.0003_band... OK
  Applying sessions.0001_initial... OK
(venv) RiffMates$ python manage.py loaddata bands #3
Installed 8 object(s) from 1 fixture(s)
```

Fixtures are a great way to insert or overwrite some data in your database, but they are a little fragile. Because inter-object relations are all specified using ID numbers, editing a fixture can be a delicate process. If you need to edit a field, like `Band.name`, that isn't too bad. If you want to add a new object you need to make sure that you use the right value for the ID. A fixture that contains the same ID for two different objects of the same type will result in an error. Fixtures are helpful for dealing with small amounts of data, and loading previously dumped content is useful. I recommend against using them to create new objects or managing large amounts of data.

4.10 Danger zone, or know thy SQL

When working with Django `Model` classes, it can be easy to forget that everything is a proxy to the database. The abstraction is helpful and allows you to quickly write Python to solve your data storage needs. The query manager attached to your `Model` allows you to do wonderfully complex queries. All of this is good, right up to the point where it is not.

If you start thinking of a `Model` as a Python object rather than as a proxy to a table, you could end up with some very inefficient SQL. An extreme example of this happens when you map conceptual objects to a `Model`. Consider a website for a school where you have students, teachers, and staff. From an object-oriented perspective you might design a student object, a teacher

object, and a staff object. Translated into a Django `Model` that results in three tables. If each of those has a name field, it is easy to make a mistake and have each field be a different length. If you want to add address information, you have to add it in three different places.

A flatter design is better. Instead of students, teachers, and staff, you have a person. The person has a role attribute to determine whether someone is a student, teacher, or staff. This design results in all the people in the same table.

This is an overly simplified example, but the object-oriented tools and design techniques that you use in Python may not translate well to a database. You need to consider the resulting database structure when thinking about your Django `Model` design.

The impact of the database structure affects the performance of your database. Consider the many-to-many relationship you built in this chapter between musicians and bands. To query the members of a band, a look-up gets done on the many-to-many table. As Django returns objects rather than just their IDs, the corresponding band members must also be looked-up in the musician table. To fetch a single band and its four members, a query is done to the band table to find the band, a query with four results gets done on the band/musician many-to-many table, and a query gets performed on the musicians table for each of the four musicians. Clever SQL can reduce the performance cost of all this work, and Django does a pretty decent job of doing this for you. The more inter-relationships your objects have, the messier all of this becomes.

For production systems with high volumes, or any system with a lot of data, the underlying SQL can make a huge difference on the performance. The better you understand SQL and what the ORM is doing on your behalf, the better your results. This does not mean you need to be an SQL expert tomorrow, but be aware that although it feels like you’re writing Python, you’re actually writing SQL.

4.11 Exercises

1. Using the musician index and detail pages as a reference, create similar pages for your `Band` objects. Inside the template use the `musicians` relationship to show the members of the band.
2. Add a page that lists all `Venue` objects along with their corresponding rooms. You will need to iterate over the `room_set` query manager in the template.
3. Modify the `musicians` index view to take an additional query parameter that specifies the number of objects per page. Make sure to support reasonable default and maximum values.

4.12 Summary

- The Django ORM provides an object-oriented abstraction to a relational database.
- You inherit from `django.db.models.Model` to define a class to be mapped to the database.
- Your `Model` class uses fields to both define the type of data stored in the database and access to that data once queried.
- The `makemigrations` management sub-command creates a Python file that describes the changes to the database needed by your models.
- The `migrate` management sub-command runs any migration files that describe changes to models that have happened since the last invocation.
- Django's default database is SQLite, an embedded, single-file database.
- A database's query tool can be accessed using the `dbshell` management sub-command.
- `Model` objects have a query manager named `.objects` that is used to query contents from the database for that model. The `.create()` method is used to create objects, while the `.first()`, `.last()`, and `.all()` methods return the first item, last item, and all the items for the object, respectively.
- A subset of values can be queried using the `.filter()` method, specifying key/value pairs for the filter. A value of `first_name="John"` returns all objects where the `first_name` attribute is exactly `"John"`.
- The arguments to `.filter()` can be augmented to specify comparisons instead of exact matches. Suffixing a keyword with `_startswith` causes `.filter()` to return partial matches. A value of `first_name_startswith="J"` returns all objects with the `first_name`

field beginning with the letter `"J"`.

- After modifying a `Model` instance you call `.save()` on the object to update the database.
- The data for an object can be removed from the database using the `.delete()` method on the object itself. The Python proxy continues to exist after `.delete()` is called.
- Views can be parameterized to take arguments. Arguments can be given through URL query parameters or captured out of the URL itself.
- The `get_object_or_404()` shortcut retrieves an object from the database or raises an `HTTP404` exception in a single call.
- `Paginator` objects slice a query result into pages, allowing you to paginate the output in your templates.
- Django supports both one-to-many and many-to-many relations in the database. One-to-many are defined using `ForeignKey` fields, while many-to-many use the `ManyToManyField`.

5 Django Admin

This chapter covers

- Using the Django Admin to manage your `Model` content
- Creating a superuser
- Customizing the Django Admin
- Linking between `Model` objects in the Django Admin
- `Model Meta` properties

This chapter covers the Django Admin, a built-in web-based tool for managing your ORM `Model` objects. The Django Admin gives you screens to list, add, edit, and delete the objects in the database.

5.1 Touring the Django Admin

In the previous chapter you learned how to use the ORM to add storage capabilities to your Django site. Along the way you used two different techniques to construct data: the `.create()` method on a `Model`, and fixtures. Wouldn't it be nice if you had a graphical interface to manage your objects? Django comes with a tool that does this for you. The *Django Admin* is a customizable web based interface that allows you to create and modify your `Model` objects with very little code.

Figure 5.1 shows a Django Admin screen that lists all of the `Musician` objects in the database. The code I wrote to generate that screen is less than eighty lines long, with it you have multi-field sorting, field search, filtering, custom columns, and all the actions you need to be able to add, update, and delete musicians from the database.

Figure 5.1. Parts of the `Musician` listing page in the Django Admin

As the name implies, the Django Admin's purpose is administering your website. Its interface isn't the prettiest, and its design is focused on users who control your site. Before using the Admin, you need to understand the basics of user permissions. Django is a multi-user framework, meaning you can

create websites that have user accounts, and each account can be restricted to a subset of activities. Django has fine-grained permission control, and you can get quite specific about who can do what, but it also has a high-level mechanism that is good enough for most situations. Django divides users into three classes:

1. General User
2. Staff
3. Admin

So far, when visiting *RiffMates*, you've been acting as an anonymous general user: one who isn't logged in. All the pages you've built have been public and so all users, including anonymous ones, are allowed to visit them. You'll learn all about building pages with restrictions in a later chapter.

The Django Admin is behind a login wall, you don't want just anybody modifying your data. A user that is marked as *staff* is allowed to log in to the Django Admin. What they can do inside the Admin is dependent on their permissions. A user that is marked as *admin* automatically has permission to do everything in the Django Admin.

`django-admin`, Django Admin, admins, and superusers

Django uses the word "admin" a lot, and to mean different things. It can be a little confusing.

Term	Meaning
`django-admin`	The management script used to create new Django projects
Django Admin	The web interface allowing administrators to manage `Model` objects

Admin	Permission level applied to a user with unrestricted access to the Django Admin
Super User	Another term for an admin user, mostly used in the `createsuperuser` management sub-command

Personally, I've never built a Django site where I've bothered with the staff designation. For sites that are complicated enough to have varying permission levels, I typically find that I want more control over the interface than the Django Admin provides. In this case I end up building my own pages for staff-type activities. The admin user though, is quite valuable. You can go a long way to being able to manage your site without having to write very much code using the Django Admin and admin users.

 **Note**

I'm going to keep referring to "admin" in the singular, but it is a property of a user account. You can have as many users with admin permission as you like.

5.1.1 Creating an admin user

The Django Admin allows you to create and manage users, but you have a bootstrapping problem: you have to be able to login to the Django Admin in order to use the Django Admin. To get around this, there is a management sub-command for creating user with admin privileges. The `createsuperuser` sub-command asks you a few questions and gets you going:

```
(venv) RiffMates$ python manage.py createsuperuser
Username (leave blank to use 'ctrudeau'): admin
Email address: admin@example.com
Password:
Password (again):
The password is too similar to the username.
```

```
This password is too short. It must contain at least 8 characters
This password is too common.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

I'll tell you a secret: I was a little naughty when selecting my password. Django has a set of rules for password strength, but the `createsuperuser` sub-command allows you to ignore them. It prompts to make sure you're ok with being naughty, and then lets you proceed.

 **Warning**

When running a production site you must guard access to the account that hosts your Django project. Anyone that can run sub-commands can create a user with admin permissions and then will have full access to your site.

With your newly created admin user in hand, you can now visit the Django Admin web interface.

5.1.2 The default Django Admin

If you were paying close attention, you might have noticed that the default `INSTALLED_APPS` configuration setting includes an app called `django.contrib.admin`. Likewise, the default main `urls.py` file also registers a `path` object for `admin/`. The Django Admin web interface is ready to go out-of-the-box. If you don't want the Django Admin, you can remove it by modifying those two files.

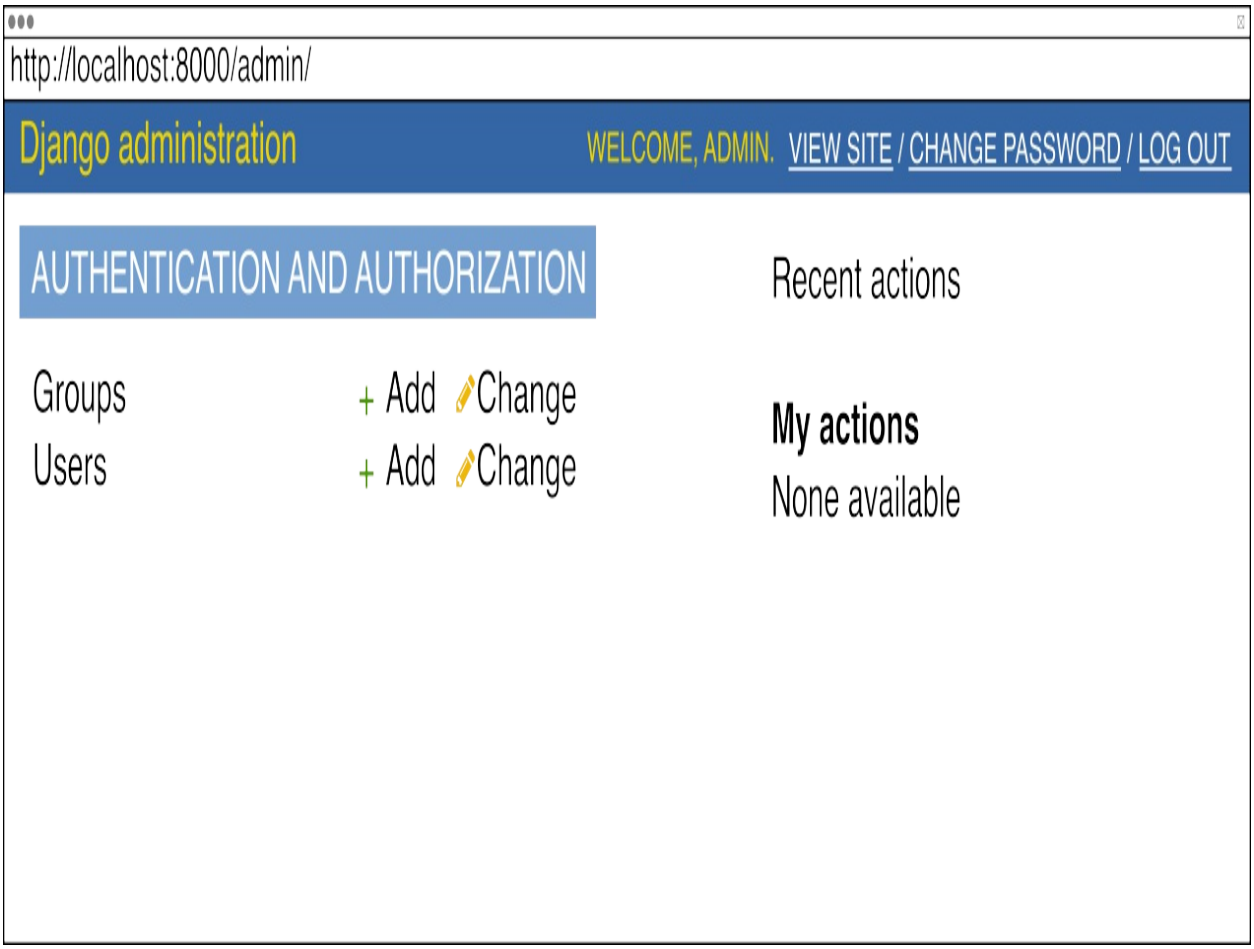
As the Django Admin is already configured, and you created an admin user, you can visit it by running the development server and going to `http://localhost:8000/admin/`. The page prompts you with a login screen.

You've probably seen a login page before and hopefully know just what to do. Use the username and password you gave to the `createsuperuser` sub-command, and authenticate to the Django Admin.

Once inside, you will see the homepage which shows you all the objects that are registered with the Admin grouped by their app. As you haven't

registered any `Model` files yet, you see the two defaults: *Users* and *Groups* as in figure 5.2.

Figure 5.2. Django Admin homepage with links for User and Group `Model` objects

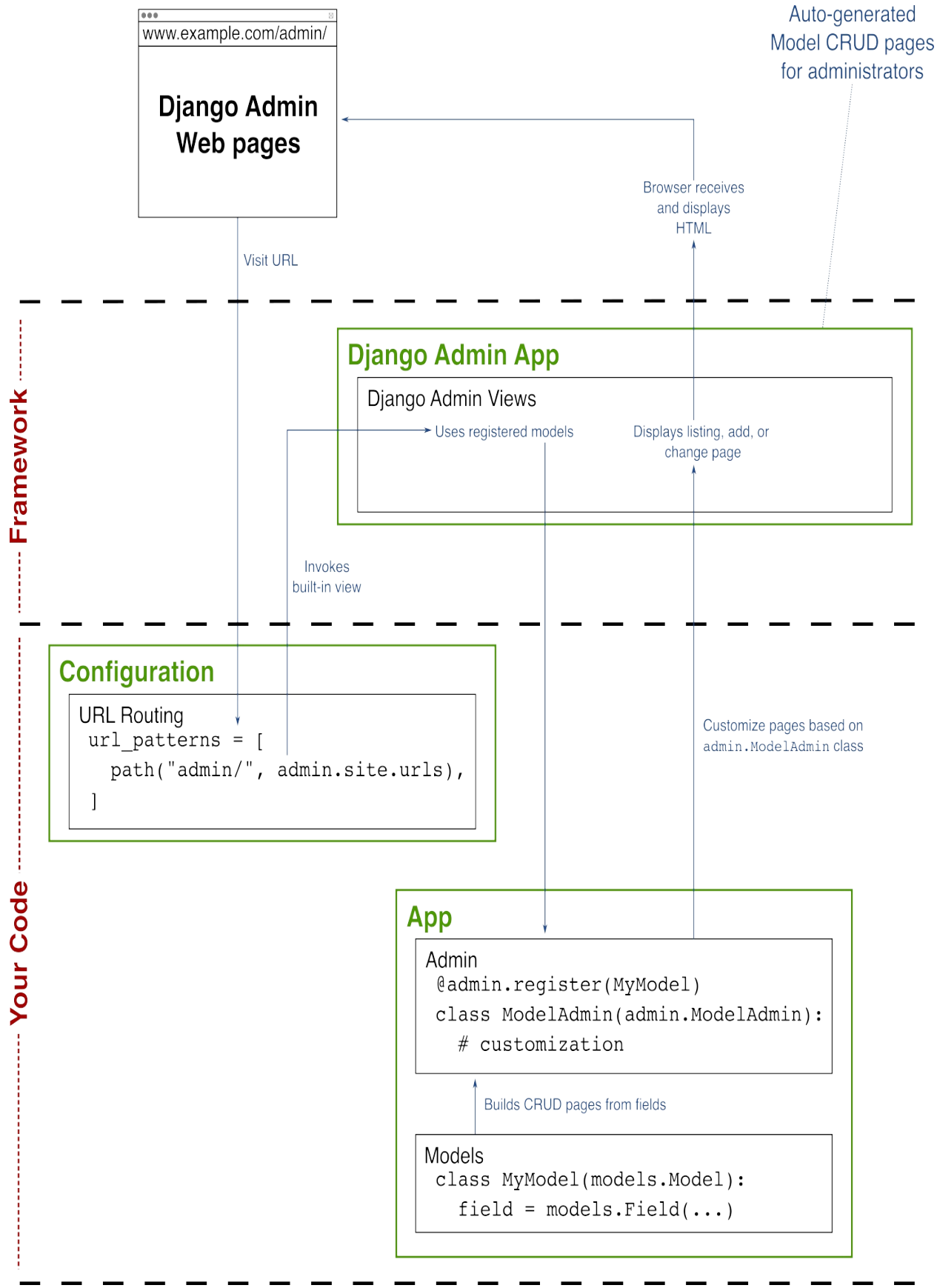


You can explore these links if you like, but a later chapter deals with managing users and will cover this information in more detail. If you click the Users link you will see a list of all your users, which at the moment is just the newly created admin account. If you are exploring, you can always get back to the homepage by clicking the "Django administration" logo in the top left corner. The right side of the page mentions actions, these include things done to objects recently. As you haven't done anything yet, the actions list is empty. When you add, edit, or delete an object you will see information on that in this list.

5.1.3 Adding your own models

Any ORM `Model` that you define can be added to the Django Admin with just a few lines of code. When you created your app using the `startapp` sub-command, Django created models.py and admin.py files for you. You've used the models.py file to define `Model` classes, the admin.py file is where you declare that you want a `Model` to participate in the Django Admin. To make this declaration, create an `admin.ModelAdmin` class and register it against the `Model` class you want in the Django Admin.

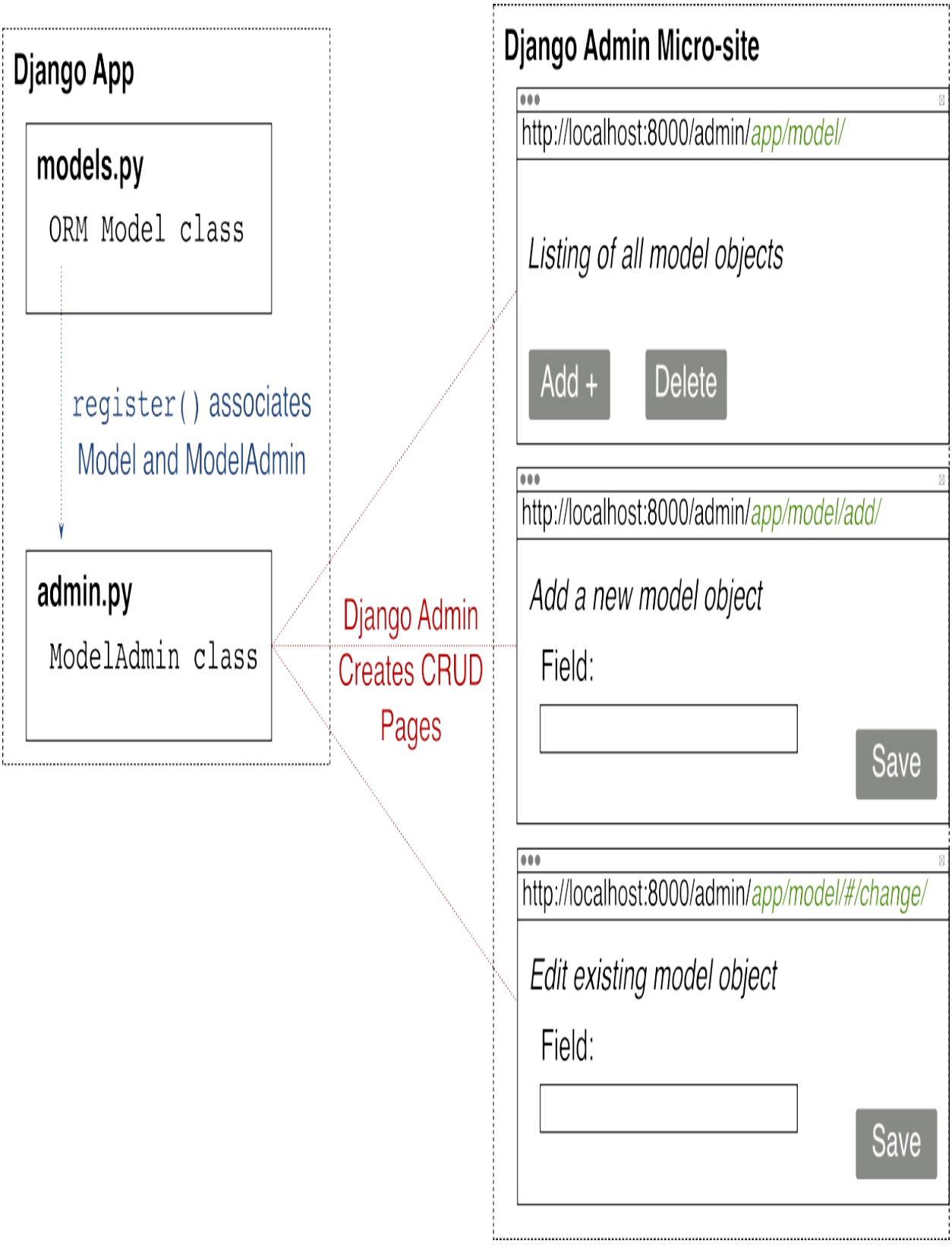
Figure 5.3. The Django Admin is an app that ships with Django giving administrators the ability to create and modify registered `Model` objects


www.example.com/admin/
Django Admin Web pages
Auto-generated Model CRUD pages for administrators
Browser receives and displays HTML
Visit URL
Framework
Django Admin App
Django Admin Views
Uses registered models
Displays listing, add, or change page
Invokes built-in view
Configuration
URL Routing
url_patterns = [
path("admin/", admin.site.urls),
]
Customize pages based on admin.ModelAdmin class
Your Code
App
Admin
@admin.register(MyModel)
class ModelAdmin(admin.ModelAdmin):
# customization
Builds CRUD pages from fields
Models
class MyModel(models.Model):
field = models.Field(...)

When Django loads your app, it looks for admin.py files, the same way it looks for models.py files. To add a `Model` to the Django Admin, you create a class that inherits from `admin.ModelAdmin` and call the `register()` function to associate the class with the model. The `register()` function can also be used as a class decorator. This is my preference, as it keeps the registration close to the `admin.ModelAdmin` class declaration.

Each registered `admin.ModelAdmin` class gets a set of *CRUD* pages. CRUD, besides being a very fun acronym, stands for: *Create*, *Read*, *Update*, and *Delete*, the actions you take on most pieces of data. The *listing* page acts as one of the Read pages, showing all the objects for the given `Model` class. Each listed object is a link, taking you to a *change* page where you can Update the fields of the given object. The Django Admin also has links to an *add* page to Create new objects, and a Delete page as well. Figure 5.4 shows the relationship between `Model` and `admin.ModelAdmin` classes as well as the resulting Django Admin screens.

Figure 5.4. `Model` class, `admin.ModelAdmin` class, and associated Django Admin screens



Let’s look at a concrete example and register the `Muscian` ORM class with

the Django Admin. To do this, you create a class that inherits from `admin.ModelAdmin` inside of admin.py. I typically call mine the name of the ORM class with `Admin` as a suffix, `MusicianAdmin` in this case. The default behavior of the `admin.ModelAdmin` class includes all the CRUD pages, so your new class can have an empty body, using just the `pass` keyword. The skeleton admin.py file generated by `startapp` imports `django.contrib.admin` for you, and the `register()` function lives inside that module. I use `register()` as a class decorator, passing in the `Musician` ORM class itself to make the association between the new admin class and the ORM. That's it. One class import and three lines of code gives you a complete set of CRUD pages behind a secure login wall. Listing 5.1 shows the code you need inside of admin.py to get this all to work.

Listing 5.1. The `MusicianAdmin` class

```
# RiffMates/bands/admin.py
from django.contrib import admin

from bands.models import Musician #1

@admin.register(Musician) #2
class MusicianAdmin(admin.ModelAdmin): #3
    pass #4
```

Fire up your development server and go back to the Django Admin Home page: `http://localhost:8000/admin/`. The page now has a new section for the "Bands" app and contains links to add a `Musician` and view the "Musicians" listings page. Click "Musicians" to see the listing page. The data you created in the previous chapter shows in the listing. The result is similar to figure 5.5. The listing page has a navigation menu on the left-hand-side similar to the homepage, it is collapsible, and for the sake of space I collapsed it. The small chevron (≫) icon can be clicked to expand the nav back to normal.

Figure 5.5. Listing page showing all `Musician` objects in the database and corresponding CRUD operations

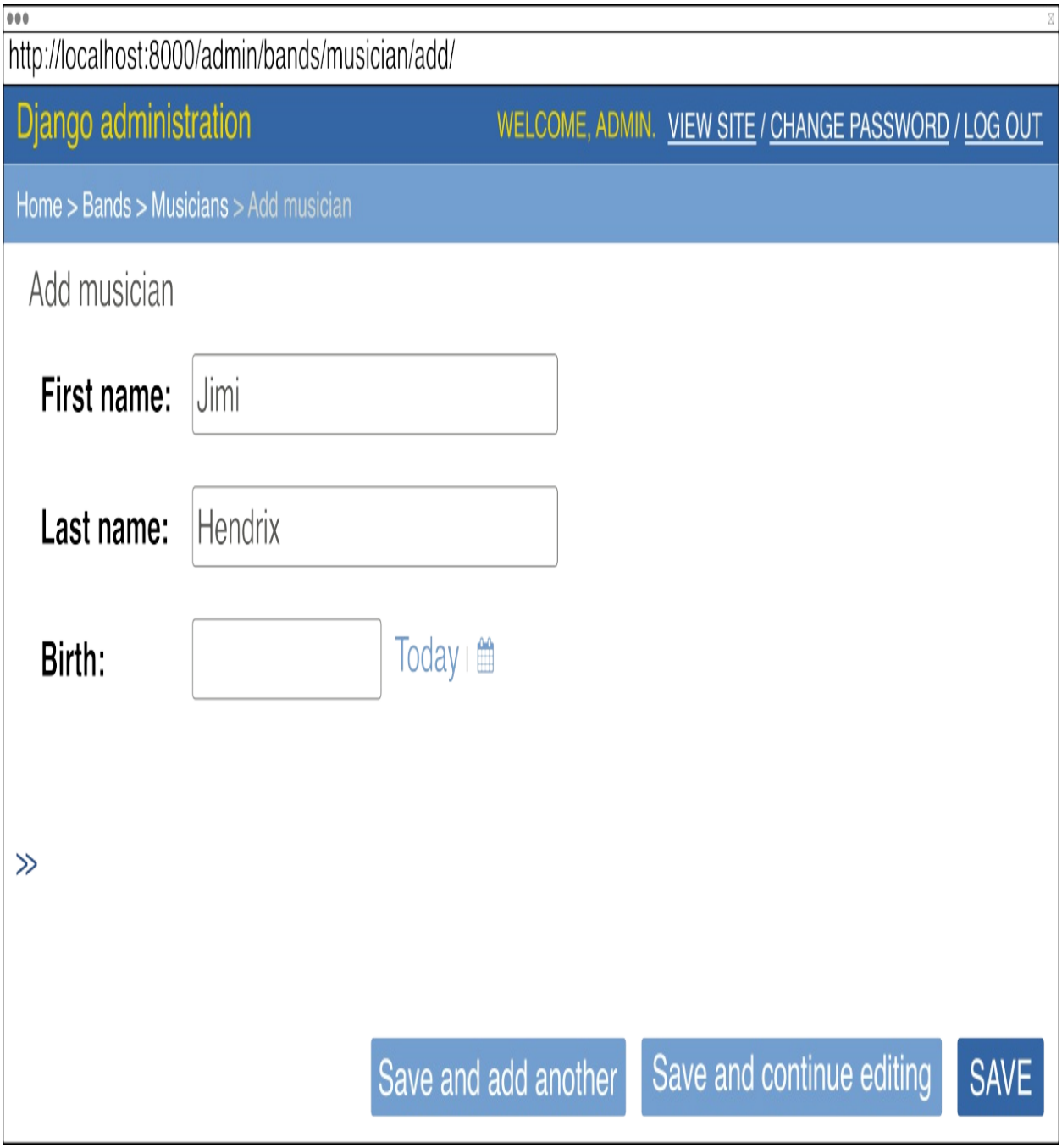
On the listing page, underneath the header, a breadcrumb navigation widget is now displayed. This navigation aid has: links to the homepage, a page similar to the home page but containing only the Bands app objects, and the name of the screen you are currently on.

The main feature of the listing page is the listing of `Musician` objects. Note how each object gets shown on this page. Does it look familiar? It is the output from the `.str()` method on your `Musician` `Model` class. If you had not overridden `.str()`, this listing would show the less helpful "Musican object(#)" default. The listing page acts as the central control for your `Musician` objects, you can:

1. Add a new `Musician`
2. Edit an existing `Musician`
3. Delete one or more `Musician` objects

You add a new `Musician` by using the "ADD MUSICIAN +" button in the top right corner of the page. Click it now. You'll get a form with all the fields for a `Musician`. Figure 5.6 shows the "ADD" screen with the fields filled in to add Jimi Hendrix to the database. The "Birth" field has an optional date picker, although for 1942, that'd be a lot of clicking. Once you have filled in the form, you have three choices for saving: "Save and add another" saves the record and puts you back to this same page; "Save and continue editing" saves the record and keeps you editing Jimi; and "SAVE" saves the record and returns you to the listing.

Figure 5.6. Add `Musician` page



Back on the listing page, to edit a `Musician` you click the link showing the contents of the *`str()`* method. The Django Admin refers to edits as "changes". The "CHANGE" page is similar to the "ADD" page, but comes pre-populated with the `Musician` being edited.

To delete a `Musician`, you can either go to the "CHANGE" screen and push the "DELETE" button, or on the listing screen, select one or more `Musician`

objects through the check-boxes and choose "Delete selected musicians" from the "Action" drop-down.

All these pages get created for you because you wrote a simple `admin.ModelAdmin` class. That's a lot of benefit for very little work and what you've seen so far is merely the default. You can customize the class to control the appearance and features of the page.

5.2 Customizing a listing page

The Django Admin's class listing page shows each of the objects in the database for the corresponding class. By default, each object gets represented using the class's `.__str__()` method. You can change this behavior by adding the `list_display` attribute to the `admin.ModelAdmin` class, that controls what columns get shown on the listing page. The attribute takes an iterable, usually a `tuple`, with an item for each desired column. Each item is a string: naming an object attribute, an argument-less object callable, or a callable on the `admin.ModelAdmin` class itself. In the callables cases, the result returned from the method is the value for the column.

Let's replace the default display for the `MusicianAdmin` class. Add the `list_display` attribute to the `MusicianAdmin` class, setting it to a `tuple` containing `("id", "last_name", "show_weekday")`. The first two items in this `tuple` result in a `Musician` object's ID and `last_name` fields getting used as columns. The `show_weekday` value, in this case, is for a callable.

My `show_weekday` callable is kinda silly, it shows the day of the week the artist was born. It can be implemented on either the `Musician` class or on the `admin.ModelAdmin` class. On the `Musician` class, it needs to be a method without any arguments. As I only want this "feature" in the Django Admin, it is better practice to put it on the `MusicianAdmin` class instead. In this case, the method takes a single argument, the object to be displayed, and gets called once for each object in the listing. The return value of the method populates the row in the column.

The title of the column for a callable is based on the name of the callable. Underscores get turned into spaces and words get displayed in small caps.

You can separate the name of your callable from the title name with one weird thing. You may have heard that everything in Python is an object, even functions, well, this goes for methods on objects as well. Like all objects, you can assign attributes to a method. It feels sort of strange, and there aren't many use cases for it in the real world, but Django uses this language feature. The `.show_weekday()` method gets the optional string attribute `.short_description`, which if present, gets used as the column title instead of the method name.

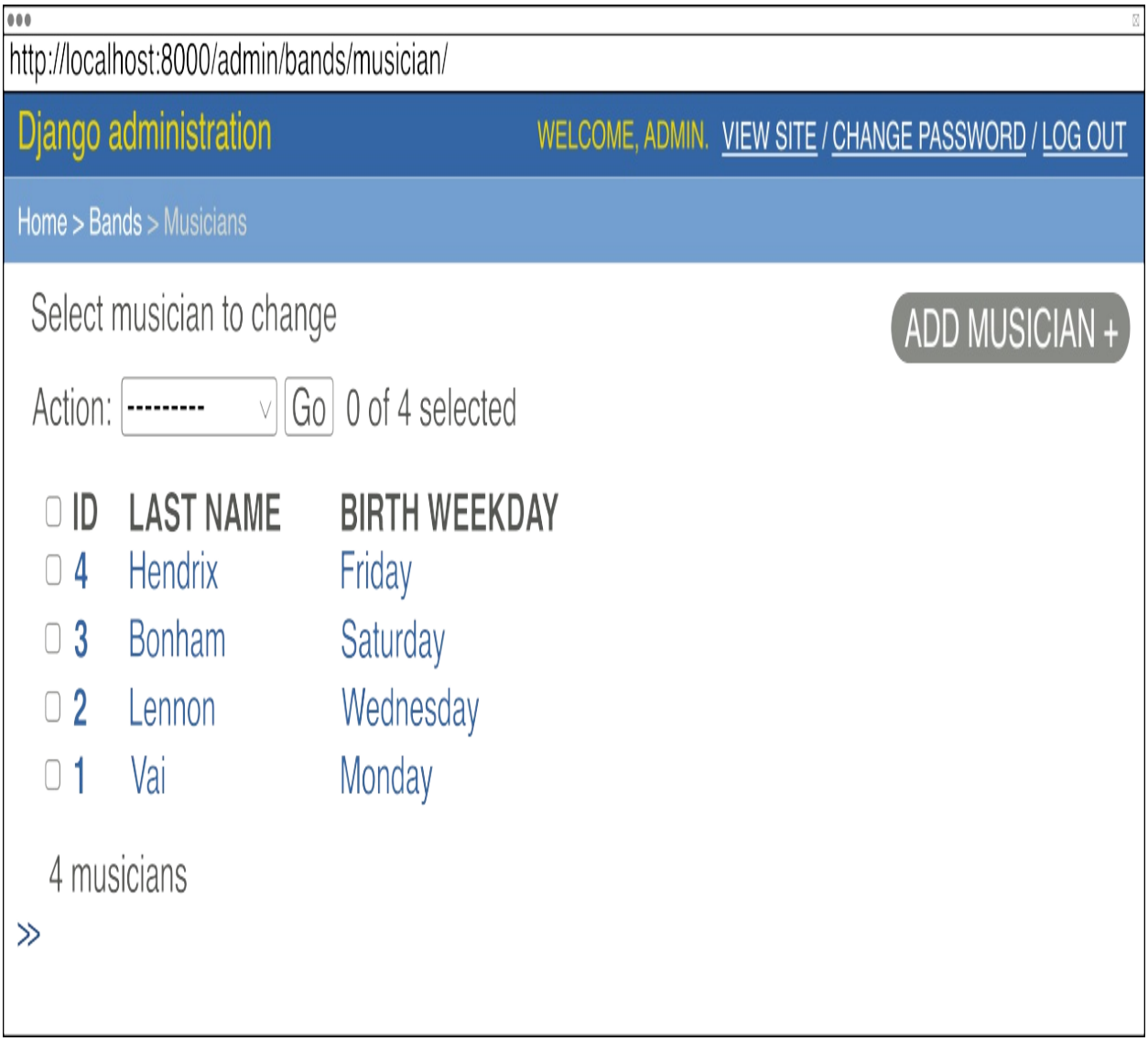
Listing 5.2 shows the new version of `MusicianAdmin` that defines the `list_display` attribute and the `.show_weekday()` method, including the `.short_description` attribute that renames the column title. Update your admin.py with this content.

Listing 5.2. Customized `MusicianAdmin` class with three columns

```
# RiffMates/bands/admin.py
from django.contrib import admin
from bands.models import Musician

@admin.register(Musician)
class MusicianAdmin(admin.ModelAdmin):
    list_display = ('id', 'last_name', 'show_weekday') #1

    def show_weekday(self, obj): #2
        # Fetch weekday of artist's birth
        return obj.birth.strftime("%A") #3
    show_weekday.short_description = "Birth Weekday" #4
```

With the above changes in place, start your development server and visit the `Musician` class's listing page. It now looks like 5.7.

Figure 5.7. Customized `Musician` listing page



5.2.1 Sorting and searching

It isn’t obvious unless you mouse-over them, but the column headers in the listing page are clickable. Clicking a column sorts it in ascending order. Once a column gets sorted, mouse-over reveals a toggle to turn sorting off. You can sort by multiple columns, allowing "first sort-by then sort-by" ordering. A small number shows in the header to indicate the sorting precedence. Sorted columns also show a small triangle that when clicked changes from ascending to descending sort order.

Sorting gets performed at the database level for efficiency. You haven’t seen it yet, as you only have four objects, but the Django Admin supports

pagination. If you have a lot of objects, Django isn’t loading them all into memory to perform the sort, but getting the database to do the work. This has a consequence: your custom columns can’t participate in the sort as they aren’t in the database. The "BIRTH WEEKDAY" column created in the previous example is not clickable as you can’t sort by it.

The Django Admin also supports search. To enable this feature, you add a `search_fields` attribute to the `admin.ModelAdmin` class. Like `list_display`, this takes an iterable of field names, in this case the fields that matched against a search term. When `search_fields` is present, the Django Admin adds a search box at the top of the page.

Different search behavior can be specified with a suffix to the name of the participating field. Without a suffix, the search term gets used as a sub-string for matches. For example, using `last_name` in `search_fields` and searching for "e", results in listing Hendrix and Lennon because they have the letter "e" in their last names.

The `_startswith` suffix causes search terms to match the beginning of the field. Configuring `first_name_startswith` and searching on "j" returns Jimi Hendrix and the two Johns: Bonham and Lennon. The `__exact` suffix makes search terms require exact matches (case-insensitive).

The `search_fields` attribute can take multiple fields, and you can mix and match the suffixes as you like within the values. Listing 5.3 modifies the `list_display` attribute of `MusicianAdmin` to add the `first_name` field as a column, and makes the first and last names searchable.

Listing 5.3. Modified `MusicianAdmin` class to support search

```
# RiffMates/bands/admin.py
...
class MusicianAdmin(admin.ModelAdmin):
    list_display = ("id", "last_name", "first_name", "show_weekda
    search_fields = ("last_name", "first_name", )
    ...
```

5.2.2 Filtering

In addition to searching, you can filter content as well. The way you implement this is similar: you add the `list_filter` attribute to your `admin.ModelAdmin` class. The simplest form of `list_filter` is the name of a field to filter by.

Filters get shown as a widget on the right-hand-side of the listing page. For fields in the `list_filter` configuration, all values for the field get shown as filter-links. This can be problematic.

For a field that has a limited selection, like a `BooleanField` or anything restricted using the `choices` attribute, a complete list of values is fine. For a wide-open field, like `last_name`, this becomes both ugly and a performance limitation. For a large dataset, you do not want every single last name in your database showing on the right-hand-side, for this case you should use search capabilities instead.

The default filter for a `DateField` shows a small set of values to choose from: today, the last seven days, this month, and this year. For our muscian's birthdays, this isn't going to be helpful. You can define a custom filter list by extending the `SimpleListFilter` class. This class has two methods, one that defines the values to show in the filter listing, and the other that determines the corresponding filter to run on the `QuerySet` used for the page.

Although I've been using some famous musicians as my examples, *RiffMates* is really about musicians seeking musicians. It is reasonable to assume anybody in the database actively looking for another musician is alive. Filtering by the last few decades is likely good enough.

A custom list filter class extends `admin.SimpleListFitler`, defines two attributes, and two methods. The attributes are `title`, and `parameter_name`, specifying the title of the filter, and name of what is being filtered by. The methods are `.lookups()` and `.queryset()` responsible for naming the filter choices and performing the filter query, respectively. Figure 5.8 shows a high-level version of this code.

Figure 5.8. An overview of a custom list filter

admin.py

```
class YourListFilter(admin.SimpleListFilter):
    title = "filter title"
    parameter_name = "filter_on"

    def lookups(self, request, model_admin):
        ...

    def queryset(self, request, queryset):
        ...
```

Returns the filter choices to be displayed

Performs the selected filter on the data

The `.lookups()` method, takes two arguments: the page's `request` object, and a reference to the associated `admin.ModelAdmin` class. The method is responsible for returning an iterable of tuples, each containing a short and long form display of the filter choice. The choices are strings, and the short-form is used in the URL as a query parameter to the filtered page. The long form is what is displayed to the user as the filter link. This structure is similar to what Django expects in the `choices` attribute of a field constructor.

The `.queryset()` method is responsible for doing the actual filtering. It takes two arguments, the page's `request` object, and a `QuerySet`. When Django Admin calls this method, it has already preformed a query used to construct the page. The resulting `QuerySet` object gets passed into `.queryset()`, allowing you to further perform actions on it. This method returns the modified `QuerySet` object with filtered contents.

Let's define a custom filter that provides the last ten decades as date groups to filter by, calling it `DecadesListFilter`. Its `.lookups()` method returns an iterable of decade ranges, and its `.queryset()` method filters the `QuerySet` argument by the decade chosen.

To calculate the range of decades, start with the current year, ignoring the current decade, then count the ten decades preceding it. The `.lookups()` method returns a `list` containing a series of `tuple`, with each `tuple` containing the beginning year of decade as a string, and another string showing the year range of the decade. The first value gets used as a query parameter in the URL, and although it would be nice for it to be an integer, it has to be a string. When the `.queryset()` method uses it, it has to convert it back to an integer to perform the filter.

The `.queryset()` method uses the query parameter argument to affect a filter by date range. The `QuerySet` argument will be the entire set of `Musician` objects, the filtered version returned needs to be limited to just the chosen decade. This filtering is done by calling `.filter()` on the `QuerySet`, passing in two limiters on the `birth` field. Recall, that the `.filter()` method supports the use of suffixes to do comparisons. A decade range is all the dates greater than or equal to January 1st of the first year of the decade, limited by dates less than or equal to December 31st of the last year in the decade. Applying the `_gte` and `_lte` suffixes on the `birth` field in the call to `.filter()` achieves this result.

When a user clicks a link in the filter widget, the same listing page gets reloaded but with a corresponding query parameter set. To save you from having to process the query parameter yourself, the class has a `.value()` convenience method that returns chosen filter argument. As our filter argument is a number containing the chosen decade, you convert it back to an integer, then use this to construct the decade date ranges for your `.filter()` call.

Listing 5.4 shows the `list_filter` argument to the `MusiciansAdmin` class that references the new `DecadeListFilter`, and the class code for the filter itself. Modify your admin.py file with the changes shown, and you'll be able to filter your musicians by decade.

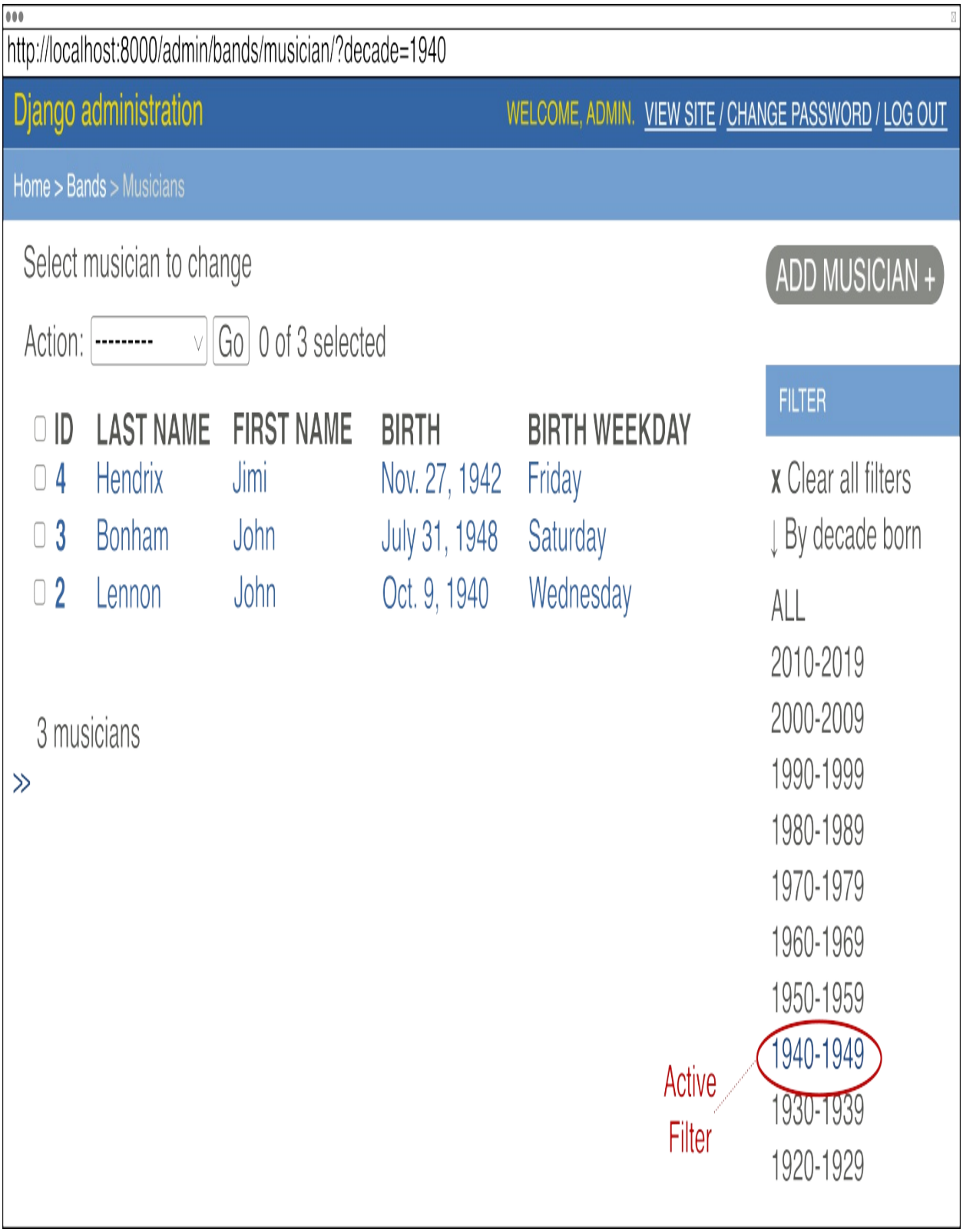
Listing 5.4. Custom list filter based on decades

```
# RiffMates/bands/admin.py
...
from datetime import datetime, date #1
```

```
...
class DecadeListFilter(admin.SimpleListFilter): #2
    title = 'decade born'     #3
    parameter_name = 'decade' #4

    def lookups(self, request, model_admin):
        result = []

        this_year = datetime.today().year
        this_decade = (this_year // 10) * 10  #5
        start = this_decade - 10
        for year in range(start, start - 100, -10): #6
            result.append( (str(year), f"{year}-{year+9}") ) #7

        return result

    def queryset(self, request, queryset):
        start = self.value()  #8
        if start is None:
            return queryset

        start = int(start)
        result = queryset.filter(
            birth__gte=date(start, 1, 1),  #9
            birth__lte=date(start + 9, 12, 31),
        )

        return result
...
class MusicianAdmin(admin.ModelAdmin):
    list_display = ("id", "last_name", "first_name", "birth",  #1
        "show_weekday", )
    list_filter = (DecadeListFilter, ) #11
```

To make it a little more obvious, the code above also adds `birth` to the `list_display` attribute, so you can see if the decade filter is working. With the code in place, start your development server and take a look at your new `Musician` listing page. Figure 5.9 shows a page filtered from 1940-1949.

Figure 5.9. Listing page filtered by the 1940's



Django comes pre-packaged with some other filters. You can dive deep into the documentation to see what is available, or learn to further customize your

own: https://docs.djangoproject.com/en/4.2/ref/contrib/admin/filters/.

When playing with your new list filter, take a look at the resulting URL. The first value in each `.lookups() tuple` gets used as the query parameter. The query parameter itself is defined through the `parameter_name` attribute. All of this is still a web application, Django is just doing many things on your behalf. Because the Django Admin uses query parameters to modify what is displayed on the page, you can take advantage of this and add features for yourself, you'll do just that in the next section.

5.3 Cross-linking between related objects

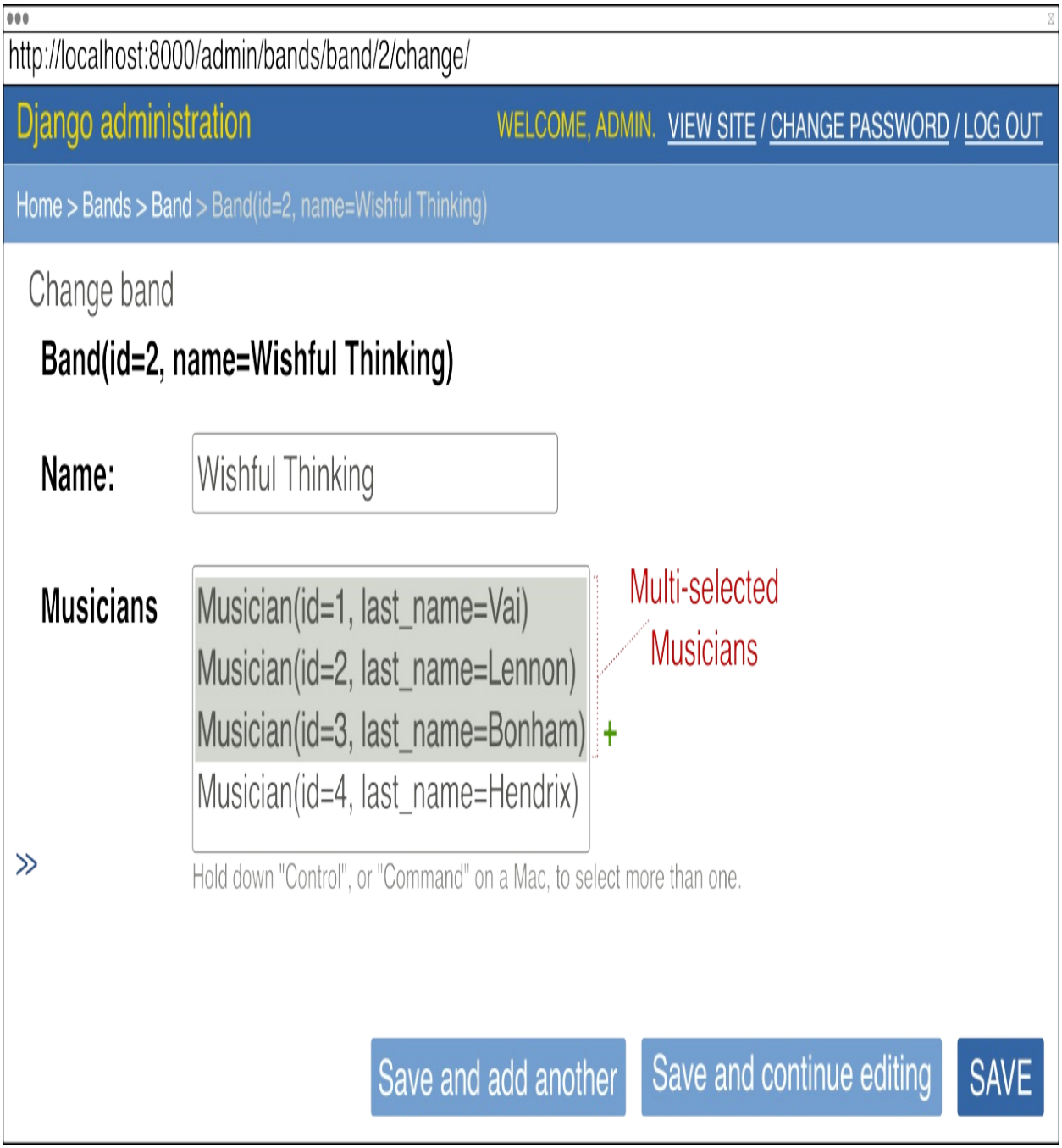
So far you've only added `Musician` to the Django Admin. Adding a `Band` object is no different, but has the added wrinkle that the `Band Model` includes a relationship to the `Musician` class. To see how the Django Admin deals with inter-object relationships, create a bare minimum `BandAdmin` class like in listing 5.5.

Listing 5.5. `BandAdmin` class

```
...
from bands.models import Musician, Band #1
...
@admin.register(Band) #2
class BandAdmin(admin.ModelAdmin):
    pass
```

With your new class in place, run the development server, visit the `Band` listing page, then click on "Wishful Thinking". Figure 5.10 shows the "CHANGE" page for the object including a multi-select widget for choosing the related `Musician` objects.

Figure 5.10. `Band` "CHANGE" screen with associated `Musicain` objects



For large datasets this screen can be rather slow in loading. To populate the `Musician` multi-select, every single possible `Musician` needs to be queried and shown. One way to help the performance is to keep the `.str()` result as short and simple as possible.

You saw how the Django Admin uses query parameters to display a subset of the results on the listing page when you use filters. Whether or not you have a

filter, you can take advantage of this feature in your own code. You can add query parameters to the URL and change what is shown on the screen. This is handy if you want to display a single object without using the "CHANGE" page. For example, the URL: `http://localhost:8000/admin/bands/band/?id=2` displays the listing page with only the `Band` with an ID of 2 in the list.

The filter query parameters also support the same field lookup modifiers as the ORM. The URL: `http://localhost:8000/admin/bands/musician/?id__gte=3` shows the `Musician` listing page containing only those `Musician` objects with an ID greater-than or equal to 3. A more useful field lookup modifier is `__in`, passing it a comma-separated list of numbers limits the content of a listing page to just those IDs.

Using this feature, you can add a column to the listing page with clickable links to related objects. For example, a column on the `Musician` page that links to the musician's bands. To do this, add another callable that returns a link. You don't want to hard-code the link to the band listing page, you want to look it up. Recall the `{% url %}` that does this kind of look-up in a template, the Python code equivalent is the `reverse()` function in the `django.urls` module. All the Django Admin URLs are named, so if you know the pattern, you can look up other Django Admin pages using `reverse()`.

The name of a Django Admin listing page is `admin:__module_class_changelist`, where *module* and *class* get replaced with the module and class name of the `Model` object listed. Calling `reverse()` with the right argument gives you the URL of the listing page, which you augment with a query parameter to filter the page.

Let's add a band-membership link to the `Musician` listing page. You need a new callable for the column, I've named mine `show_bands`. Inside the callable use `reverse()` to look up the URL for the `Band` listing page. This page is named: `admin:bands_band_changelist` ("bands" for the app, "band" for the `Model`). To filter the listing by a musician's bands, you need the IDs of those bands. Query those IDs, then construct an `?id__in` query parameter to append to your listing page URL.

Recall from the chapter on templates that by default Django escapes HTML.

You can’t just return text with an <a> tag inside it, you need to mark it as safe for rendering. The `format_html()` function from the `django.utils.html` module lets you build safe HTML. This function uses paired braces ({}) as placeholders, allowing you to parameterize a string with an HTML link inside it. Like with `show_weekday`, add the `.short_description` attribute to change the column title.

Make the changes in listing 5.6 to your admin.py file and then visit the Django Admin to see your new content.

Listing 5.6. `MusicianAdmin` class with links to `Band` listing page

```
# RiffMates/bands/admin.py
...
from django.utils.html import format_html  #1
from django.urls import reverse
...
@admin.register(Musician)
class MusicianAdmin(admin.ModelAdmin):
    list_display = ("id", "last_name", "first_name", "birth", "sh
        "show_bands")  #2
    ...

    def show_bands(self, obj):
        bands = obj.band_set.all() #3
        if len(bands) == 0:        #4
            return format_html("<i>None</i>")

        plural = ""       #5
        if len(bands) > 1:
            plural = "s"

        parm = "?id__in=" + ",".join([str(b.id) for b in bands])
        url = reverse("admin:bands_band_changelist") + parm
        return format_html('<a href="{}">Band{}</a>', url, plural
    show_bands.short_description = "Bands"
```

Start your development server and return to the `Musician` listing page. You now have a "Bands" column with clickable links. Each link goes to a filtered version of the `Band` listing page, showing only those bands the musician is a member of.

5.4 `Model` meta-properties

You've seen how the `.str()` method on a `Model` class changes the corresponding Django Admin listing page. Other properties of the `Model` can also effect the Django Admin. This is another one of those cases where Django takes advantage of a seldom used Python language feature. I heard you like classes… how about a class inside your class?

Django `Model` classes can optionally define a nested class named `Meta`. Django uses the attributes of this class to modify the default behavior of the resulting `Model` object. Some attributes are only used by the Django Admin, and others change impact the behavior of the `Model` itself, such as modifying the underlying database queries.

The following sections give details on some of the more popular `Meta` attributes. For a full list, see: https://docs.djangoproject.com/en/4.2/ref/models/options/.

5.4.1 Sort order

The `ordering` `Meta` attribute specifies the default sort order in response to queries on the object. This impacts both the order of the objects in the Django Admin listing pages, as well as any query you run in your code. The underlying SQL gets an `ORDER BY` clause attached automatically. This attribute takes a list of field names and supports a dash (-) prefix to indicate descending sort order. Example:

```
class VenueStaff(models.Model):
    # first_name, last_name, employee ID number fields
    ...

    class Meta:
        ordering = ["last_name", "-employee_id"] #1
```

The default sort order is none. SQLite appears to return the `Musician` objects in reverse order by ID, but this should not be depended upon. Ordering costs overhead, and requires the database to do additional work. It is handy though. To change the default sort order of the `Musician` class to be based on

`last_name` and then `first_name`, make the following change in your RiffMates/bands/models.py file:

```
class Musician(models.Model):
    ...

    class Meta:
        ordering = ["last_name", "first_name"]
```

Once you've made the change, try running `Musician.objects.all()` in the REPL, or visit the Django Admin `Musician` listing page to see the difference.

5.4.2 Indexes

In the chapter on the ORM you learned about specifying an index in the field declaration of a `Model`. You can also declare indexes through `Meta` attributes. Using this mechanism allows you to have multiple fields participate in a single index.

The attribute name for this is `indexes`. It takes a list of `models.Index` objects, with each object instance specifying the participating fields. Example:

```
class VenueStaff(models.Model):
    # first_name, last_name, employee ID number fields

    class Meta:
        indexes = [models.Index(fields=["last_name", "first_name"
```

 **Note**

Remember, indexes make a big performance difference on query results for the fields being searched. Without an index, the database needs to scan the entire table until it finds a matching result. The index works like a hashtable, providing a shortcut to finding your values quickly.

5.4.3 Uniqueness

The Django ORM automatically creates the ID field for you as a unique identifier for an object in the table. There may be situations where you want

to ensure that one or more other fields is also unique within a table. The `unique_together` attribute allows you to do just that.

This attribute takes a list of lists. The nested list contains the fields that are to be unique when combined, and all of that is in a list as you can specify more than one set of fields. As a short-cut, you can specify this as a single list if you only have one set of fields that are to be unique together.

Consider the `Room` object that defines a room in a `Venue` from the ORM chapter. You don't want two rooms to have the same name in the same venue. You can ensure this by making the `Room` `name` and the `venue` `ForeignKey` unique together:

```
class Room(models.Model):
    # name and venue field declaration

    class Meta:
        unique_together = [["name", "venue"]]

        # alternate short-cut version as there is only one "uniqu
        # unique_together = ["name", "venue"]
```

The enforcement of the `unique_together` clause is done at the database level. If you create an object that violates this in code, an exception will be raised. If you attempt to do the same in the Django Admin, you will get an error message.

More recent versions of Django have a general purpose mechanism for specifying constraints on your tables. Django has constraint classes, one for uniqueness, the other for enforcing values. There is also an abstract base class, so you can write your own. The `unique_together` attribute duplicates the constraint case. It hasn't been deprecated yet, but the documentation indicates it may be in the future. Writing your own Constraints requires ORM features beyond the scope of this book, see: https://docs.djangoproject.com/en/4.2/ref/models/constraints/ for more information.

5.4.4 Object names

Inside the Django Admin, the name of your `Model` is directly derived from the `Model` class name. For example, the `Musician` class appears as "musician", "musicians", "Musician", and "Musicians" depending on the context. Take a look at the `Musician` listing page; on it, you find "Select musician to change" in the title, and "Musicians" in the navigation breadcrumbs.

If your `Model` class is multiple words specified in "CamelCase", Django treats that as "camel case", capitalizing when necessary and appending the letter "s" to pluralize. For example, `MusicianClassified` becomes `musician classifieds` in plural sentences. If you're not happy with how this is done, the `vebose_name` and `verbose_name_plural` attributes give you control. These two `Meta` attributes are available on the class, but I've only ever seen them used by the Django Admin.

A `verbose_name` containing capitals, appears that way everywhere. If the attribute is lower case, it is capitalized where Django sees fit. I don't like the mixed-case class name in the navigation bar, so I often set `verbose_name` to use the capitalization I want. Without `verbose_name`, `MusicianClassified` shows as `Musician [.underline]#c#lassifieds` in the nav. I set `verbose_name` to `Musician Classified` in order to get `Musician [.underline]#C#lassifieds` in the nav. The downside of this is the title becomes "Select Musician Classified to change." Pick your preference.

The most common use of `verbose_name_plural` is for words that don't pluralize in the normal way, or are already plural. Consider a model named `Box`:

```
class Box(models.Model):
    # field declarations
    ...

    class Meta:
        verbose_name_plural = "boxes"
```

Use of `verbose_name_plural` alone, or along with `verbose_name`, gets your `Model` names showing properly. My mother the retired kindergarten teacher thanks you for your proper grammar.

5.5 Exercises

1. Add search capabilities to the `BandAdmin` class.
2. Add default sorting by name to the `Band`, `Venue`, and `Room` classes
3. Create `admin.ModelAdmin` classes for the `Venue` and `Room` `Model` classes, including: custom linked columns methods like with `Musician` and `Band`; and searchable name fields.
4. Add a "Members" column to the `Band` listing page, that contains multiple links, one to each `Musician` in the `Band`. Note that appending to a string makes it unsafe again. Use `mark_safe` to fix that.

5.6 Summary

- The Django Admin is a built-in web interface for creating, editing, and deleting your `Model` objects.
- You create `admin.ModelAdmin` classes to register your `Model` classes with the Django Admin.
- Access to the Django Admin is restricted to special users only: staff and admins (a.k.a. superusers).
- A superuser account is created using the `createsuperuser` management sub-command.
- The homepage of the Django Admin lists all registered `Model` objects grouped by their app.
- Each `admin.ModelAdmin` class generates a listing page and pages for adding, changing, and deleting objects.
- Columns on the listing page get determined by the `list_display` property of your `admin.ModelAdmin` object.
- The `list_display` attribute contains the names of `Model` fields, argument-less `Model` methods, or a method on the `admin.ModelAdmin` class.
- Columns based on `Model` fields are sortable by clicking the column header. Sorting by multiple columns is supported as well as ascending and descending order.
- A callable used by `list_display` can return HTML, including links to other Django Admin pages as long as it is a safe string.
- Django Admin listing pages support search using fields named in the

`search_fields` attribute on the `admin.ModelAdmin` class.

- A filter on the listing page restricts what data gets shown. Filters get enabled through the `list_filter` attribute on the `admin.ModelAdmin` class.
- Custom filters inherit from `SimpleListFilter` and define the filter choices and a method that modifies the listing page's `QuerySet`.
- Registered URL routes, including Django Admin's, can be looked up in Python using the `reverse()` function.
- Listing pages support query parameters to show a subset of the page contents. These parameters can be mixed with a URL look-up to add links to related objects in a custom column.
- You can nest an inner `Meta` class inside a `Model` class to control the behavior of the objects. Commonly used parameters control indexes on the table, field uniqueness, and the name of the object in the Django Admin.

Appendix A. Installation and setup

Python's growth in popularity means it has become more common for it to be installed on your system by default, but that isn't always the case. And sometimes its presence on your system is a problem: macOS ships with Python, but a much earlier version. This appendix briefly outlines some of the choices you have for setting up your system and where to find more information.

A.1 Installing Python

There are several ways of getting Python onto your computer, some of which are specific to an operating system. Personally, I use https://www.python.org/, the home of Python, and download the installable packages directly. Packages are available for Windows (in 32-bit, 64-bit, and ARM64 variations), on macOS as a universal installer (both Intel and Apple Silicon), and directly as source code. There is also a reference page for where to locate ports to other platforms.

Most Linux distributions include a variety of versions of Python using their built-in package managers. Building from source is also always an option.

To determine if you have Python installed on your system and what version it is, do the following at a terminal:

```
$ python --version
Python 3.11.4
```

On Windows you can use the same command running from a PowerShell.

A.1.1 Windows installation options

There are two other common options for installing Python on Windows:

- Microsoft Store package

- Windows Subsystem for Linux (WSL)

Python is now available in the Microsoft Store. Use the store app and search for Python and Windows will take care of the rest for you.

As of Windows 10, you can run Linux inside Windows using the Windows Subsystem for Linux. Running Python inside the WSL is slightly more complicated than running it directly from Windows, as you need to be inside the Linux environment, but it has the added benefit that most deployment scenarios are Unix based. By using the WSL, your development environment will likely be a closer match to your production environment, which tends to lessen the challenges experienced during deployment. Documentation on installing the WSL is available here: https://learn.microsoft.com/en-us/windows/wsl/install.

A.2 Virtual environments

When you install a third-party package in Python using `pip` or its equivalent, the tool downloads Python code from the *Python Package Index* (PyPI) and puts it in the `site-packages` directory. When you `import` a package in your code, Python looks in this directory, as well as a few other places.

The Cheese Shop

Python isn't named after the snake, but after the British comedy troupe Monty Python. When it came to building a site to manage Python packages, the coders called it the "Cheese Shop" after a famous Monty Python skit. Eventually, reason won out, and it was renamed the Python Package Index, but it is still occasionally affectionately named using its original moniker.

It is really handy to be able to choose from the over 450,000 freely available Python third party libraries, but there is a complication. Packages change over time and multiple versions are available. If you created a website using Django in 2021, you likely were using Django 3.2. In the following year Django 4.0 was released. You may not want to go back to your original project and upgrade it, installing Django 4.0 overwrites Django 3.2, possibly breaking your original site.

If all that isn’t messy enough, most packages have dependencies on other packages. In 2022 when you installed Django 4.0 you automatically got asgiref 3.4.1 and sqlparse 0.2.2. What if you also wanted to insall Apache Superset, the data visualization platform, which also uses sqlparse? How do you know that the version used by Django and the version used by Superset don’t conflict?

The answer to all these questions is to use *virtual environments*. A Python virtual environment creates small copies of all the directories that Python requires in their own place and sets up a separate `site-packages` directory. The important stuff, like the interpreter itself, is linked to, so it doesn’t take up too much disk space. Each virtual environment is self-contained, so you can install a package into an environment without affecting any other environment, including the default install location.

To use a virtual environment, you activate it. Activating an environment changes your shell so that the environment’s link to the Python interpreter is first in your path, meaning it is the one your terminal finds when you run a script. It is best practice to have a separate virtual environment for each project you create. This avoids any library conflicts and means you can switch back and forth between projects without worrying about whether they’ll still work.

There are several virtual environment management tools out there. In this book, all the examples use venv, which ships with Python. To create a virtual environment using venv, type:

```
$ python -m venv my_environment
```

 **Note**

If you have multiple versions of Python installed on your system, make sure to use the version that you want to use inside the virtual environment.

This example creates a directory named my_environment containing the virtual environment. Common practice is to name the folder **venv** or **.venv**. With the directory created, you activate it. On Windows, run:

```
C:> my_environment/Scripts/activate.bat
```

On macOS and Linux, use:

```
$ source my_environment/bin/activate
```

You can switch to a different environment by activating it instead. If you wish to stop using the environment, you use the deactivate command, which is the same on all operating systems:

```
$ deactivate
```

For more information on `venv`, see the documentation: https://docs.python.org/3/tutorial/venv.html.

Virtual environment directory location

The `venv` tool gets used to create the directory containing the virtual environment in the folder location where it is run. There are two schools of thought on the placement of this directory: 1) in your project, or 2) in a central location.

Keeping your virtual environment with your project makes it clear what environment goes with the project and keeps things all together. There are even tools out there that can automatically switch your virtual environment based on what folder you are in. There are two downsides to this approach: you can't easily share virtual environments, and it complicates the use of some command line tools. Sharing virtual environments isn't that important, but it can be handy if you have multiple pieces of software that will all be using the same environment in production.

Depending on how often you use command-line tools, virtual environments inside a project folder can add extra complications. I often run a Unix `find` command to look for files, or `grep -r` to look for content in files. By having the virtual environment directory in the project, you end up searching all those files as well. As most Python third party libraries contain Python, searching for or in files ending in `.py` tends to return a lot of false positives.

Alternatively, you can keep you virtual environments in a central location.

The pros of this method are the cons of the other and vice versa. Your project directory now only contains your project files, but now you have to remember which project go with which virtual environments.

There are also alternatives out there to using `venv`: two popular tools for managing virtual environments are virtualenv (https://virtualenv.pypa.io/en/latest/) and Conda (https://docs.conda.io/en/latest/). Personally, I use virtualenv on my own projects, but it requires an extra install step over `venv`. I'm not sure I can even argue in favor of my preference: it was the tool I discovered first, and inertia has kept me there.

A.3 Editing tools

The first line of code that I ever wrote that wasn't just copied out of a book was in the 1980s on an IBM PCjr. It had a whopping 4.77 MHz CPU (that second "7" seemed really important at the time). As I progressed to Unix, my coding world was still very much based on terminals, and to this day I still prefer to use Vim (https://www.vim.org/) while running the Django development server in another terminal.

This isn't for everyone, especially if you're just getting started. The two most popular *Integrated Development Environments* (IDE) for developing Python are Visual Studio Code and PyCharm. As Python is text, any editor you're happy with will work. If you are using an IDE though, there is some extra configuration that makes your life easier.

IDEs have debugging tools built in. Because Django is a framework, and you're not really running a script, you need to perform extra steps to be able to use the debugger with code run through the Django development server. Once you've created a project in the IDE, you need to modify the run/debug profile to tell it to use the Django development server.

For Visual Studio Code, full instructions on creating a Django project and debugging can be found here: https://code.visualstudio.com/docs/python/tutorial-django. The equivalent document for PyCharm is here: http://mng.bz/x41Y.

If you’re using the IDE just to edit the files and running the development server in a terminal, the extra isn’t required. If you’re used to using the debugger and would miss coding without it, then you’ll need to make those changes.

Appendix B. Django in a production environment

If you've come from a world where most of your Python gets run locally, or you send your script to a friend through email, moving to the world of the web can be rather daunting. Even if you're an old pro at packaging things and submitting them to PyPI, the web has its own set of challenges. When you write something using Django, or any other web framework, you're no longer just writing code. If you want to put your project on the web, you're now dealing with infrastructure. It isn't quite as scary as it sounds, but there are some things you need to be aware of.

 **Note**

Throughout this appendix I will be mentioning companies that provide a variety of hosting services. This is not an endorsement; Google-ing a provider name and "alternatives" or "competitors" is a good way to get a list of choices to choose from.

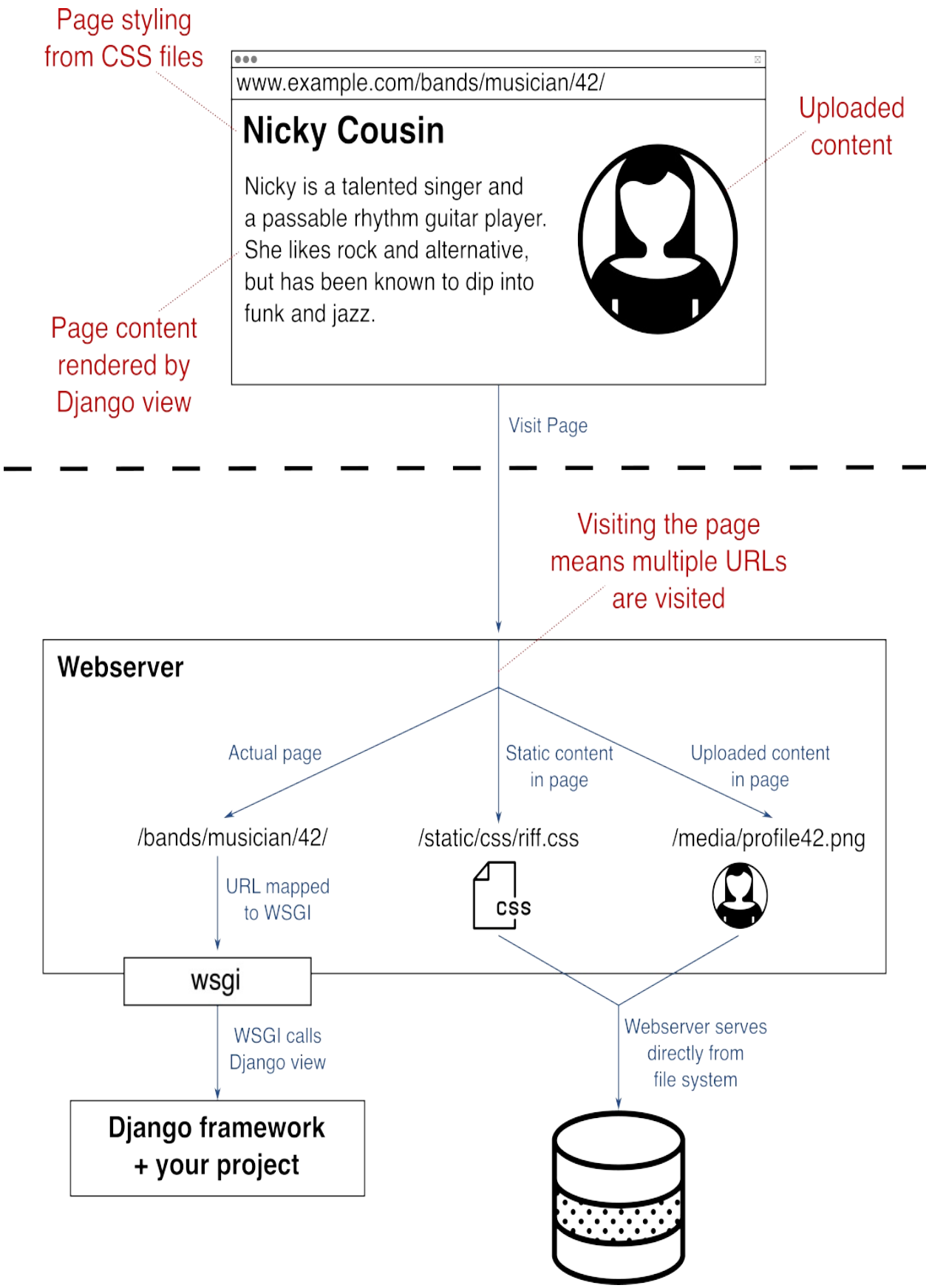
B.1 Parts of a web deployment

Throughout this book you've been using the Django development server, which, as is deftly found within its name, is only for development. It isn't hardened against attack, and it isn't optimized for performance. Even if it was, a lot of the content on your web page doesn't need Django. For many of the parts of a page the framework is overkill. Think about all the steps involved in running a view: a mapped route gets looked up, the request gets passed through multiple middleware processors (authentication, session management, CSRF handling, and more), and then finally your code gets run. And of course, your code is Python, which is not the speediest language out there. Static content like CSS files, site images, and JavaScript shouldn't be served by Django; they don't need all that extra overhead. Likewise, any

user-uploaded content also doesn’t typically need all the mechanics that your view requires.

Django is invoked through either a WSGI or ASGI interface, with a web server responsible for handling the actual call and passing it down to Django. Therefore, when you go to production, you need a web server. With such a beast in place, it makes a lot of sense to have the static and uploaded files served directly, skipping all that Django overhead. Figure B.1 shows the typical architecture with a web server fronting Django and directly hosting the static and uploaded files.

Figure B.1. Django through WSGI and other files served directly


Page styling from CSS files
www.example.com/bands/musician/42/
Nicky Cousin
Nicky is a talented singer and a passable rhythm guitar player. She likes rock and alternative, but has been known to dip into funk and jazz.
Uploaded content
Page content rendered by Django view
Visit Page
Visiting the page means multiple URLs are visited
Webserver
Actual page
Static content in page
Uploaded content in page
/bands/musician/42/
/static/css/riff.css
/media/profile42.png
URL mapped to WSGI
CSS
wsgi
WSGI calls Django view
Webserver serves directly from file system
Django framework + your project

When you go to production, you can implement everything in figure B.1 yourself, or there are providers that abstract this away for you to varying degrees. However you deploy, the same concept is still happening underneath.

Serving static content from Django

When it comes to hosting architecture there are plenty of choices, even the decision to serve static files from a web server isn't the only way. Django WhiteNoise is a third-party package that installs as middleware and serves static files directly from Django. It does this by bypassing all those extra steps I just mentioned. It isn't quite as fast as a web server, but it handles file changes and selective compression quite elegantly.

B.2 Server hosting vs Platform-as-a-Service

You can think of your hosting options as a spectrum. At one extreme (we'll call it the "left"), you have the most control over your configuration and the most work to do. At the right end of the spectrum is *Platform-as-a-Service* (Paas) where you have very little work to do, but you more or less have to do it exactly the way the provider wants you to. The spectrum isn't quite evenly distributed; it's sort of chunky and it can be divided into three pieces:

- Hardware
- *Virtual Private Servers* (VPS)
- Platform-as-a-Service

On the left side of the spectrum you buy a server, plug it in, and connect it to the internet. Depending on your internet service provider, this may not even be allowed. You might plug it in at your place, or rent a rack in a server room. Still very close to that left end is the VPS, where a hosting provider owns the hardware but gives you access to it. In the hardware case, and sometimes the VPS case, you are responsible for installing and maintaining the operating system, the web server, possibly Python, and the Django framework. The web server needs to be configured for your domain address and able to define base URL routes. The most common mappings have

`/static/` pointing to the static files on the web server, `/media/` pointing to uploaded files, and all other URLs sent to Django through WSGI.

B.2.1 Virtual Private Server

A VPS may or may not be shared. Shared servers generally are less expensive, but you run the risk of someone else on the server eating up all the CPU's cycles. Good hosting companies monitor this and keep on top of it, but it does still occasionally happen.

Even within the world of the VPS, there is a spectrum. Providers such as Linode and DigitalOcean (with their Droplets offering) start out with a Linux image which you install on top of. They also offer pre-installed packages that include a web server, which means you have less to install yourself. A little further along the spectrum is a provider like OpalStack whose dashboard includes installation modules that connect to their web server. On OpalStack you would install three modules: Django, and two file serving modules (one for your static files and the other for uploads).

One downside of a VPS is it may be up to you to maintain your associated software. For example, if you've installed your own Apache web server, you are responsible for patching it. This comes back to the spectrum: a near-bare-metal Linode instance requires more work than an OpalStack account where most of this is taken care of for you.

If your Django project uses a database, well, that's one more thing you have to worry about, isn't it? On the extreme left, you install and configure a database yourself. I generally recommend against this. Most VPS providers also offer cloud services, including databases. This is the better choice, as the database gets tuned by a professional and backups typically get handled for you.

If you have the good fortune of your site becoming very popular, you also have a new problem: scaling. With enough traffic, a single instance of a VPS is insufficient. Most VPS providers offer some load-balancing mechanism that splits traffic among different servers, but this requires more configuration.

B.2.2 Platform-as-a-Service (PaaS)

On the right side of the deployment spectrum are PaaS providers. These hosting companies abstract away all the details of deployment. They tend to be a little more expensive than VPS, but there is a lot less work, in both getting them going and maintaining them. DigitalOcean has a separate offering from their Droplets VPS called the App Platform; this and Heroku are common PaaS choices. The interface to Heroku, for example, is a GitHub repository. You point your Heroku account at your repo and when you update the main branch, Heroku automatically deploys. Their system understands Django intimately and knows exactly where the different kinds of files go and how to host them. You can even include a configuration file in your repo to fine-tune your deployment options.

There is also the container approach, using Docker or Kubernetes. If you're new to Django and the web world, I wouldn't recommend going down this path unless you are already very familiar with containers. Containers can help you scale massively, but most sites don't need them to get going. There is a fairly steep learning curve to using containers and if you're already experimenting with your first Django deployment, adding containers on top of that makes for a difficult hill to climb. If you are keen on Docker, the PaaS host Fly.io might be the right choice for you.

The previous paragraph can more or less be repeated, replacing the word "container" with the word "serverless". Serverless configurations like AWS Lambda can provide massive scaling options at the cost of warm-up time (how long it takes to start a new instance in the background), but there are a lot of moving parts needed to get Django going on Lambda. You'll need an S3 instance for storing your files, an RDS database, Lambda itself, and several other services to wire all this together. As with containers, if you aren't already intimately familiar with AWS, this isn't the place to start.

Choosing a hosting provider

Except for the most basic recommendations, the answer to "what should I do?" is always going to be "it depends". I have been using Linux since Slackware 3.2 (that's the late 90s), so a VPS is fairly comfortable for me. If

you’re familiar with the Unix world, and have installed software like web servers and databases before, the VPS is a good fit. For my smaller clients, I usually start with OpalStack; it gives full shell access and control, but They take care of the database and web server instance, so it is a nice middle ground.

At scale, you’ll want more control over how many servers you’re running, how they’re load-balanced, and where background tasks operate. Within the VPS world this means more work, but you can tune it to your heart’s content.

PaaS services tend to be a bit more expensive. Heroku used to be the go-to because it was free, but that ended in November 2022. This isn’t a reason not to use their platform; all the other PaaS providers charge as well. I personally don’t use PaaS for my clients, but I have worked with folks who do, and they swear by it. At scale, PaaS can become expensive, but if you’re serving a lot of users, automatically adding more instances to keep the site running smoothly is a big win.

B.3 Synchronous vs asynchronous

WSGI is the Python standard interface for interacting with a web server, and ASGI is its asynchronous equivalent. When you interact with a server synchronously, it doesn’t return a result to you until the result is ready. With a web page, that means getting the HTML all in one go. Synchronous systems tend to be pull-based, meaning it is up to the client (the web browser) to ask the server for something.

An asynchronous system tends to be bi-directional. You might ask for something and get a response, or the system might send you something on its own. Chat typically gets built using asynchronous technology: the server notifies you when you receive a message. To implement chat in a synchronous server your client has to ask the server "anything new for me?" with some regularity. This is known as "polling" the server.

In the last few years, there has been a trend towards asynchronous based web frameworks. Django originally was a synchronous-only toolkit, but asynchronous features have been introduced over time. Daphne, Uvicorn, and

Hypercorn are all asynchronous web servers that can interact with Django over ASGI.

For a "normal" Django project, switching to using Daphne and the like is as simple as pointing Daphne at "asgi.py" instead of "wsgi.py" in your project configuration folder. Typically, though, the reason for going asynchronous is to take advantage of the features it enables. If you want to build a chat server using Django, you're going to need extra tools. The django-channels third-party library integrates WebSockets with Django, allowing full bi-directional communication with your web server over JavaScript, and as it is Django based, it uses views to do this.

 **Note**

WebSockets are a protocol that sit on top of HTTP allowing for asynchronous bi-directional communication on the web and require JavaScript to work in the browser.

B.4 Readying Django for production

Once your infrastructure is in place, you have to deploy your project for the first time. Your code needs to be put on the server, your server may need configuration, your production database needs to be synchronized with your ORM through the `migrate` management sub-command, your static files need to be copied from your project folder to the web server's folder, and possibly more. This section walks you through each of these tasks.

B.4.1 Environment-specific configuration

At minimum, you have two environments for your code: development (usually your local machine) and production (available to the world). More complex setups are possible, and quite common in industry. In large organizations you might have a local machine, a development server, multiple levels of testing servers, and then production.

Some configuration in settings.py is common to all deployment

environments, but some is environment specific. For example, `DEBUG` should be `True` in dev and `False` in production. There are two general approaches to dealing with these differences: using environment variables and importing values.

There is a reason settings.py ends in ".py": it is a Python file. Throughout the book you have added Python variables as configuration values in settings.py, but you can also run code. You don't want to do this very much, but a few lines can offer you conditional configuration based on your deployment target.

Your operating system allows for the setting of environment variables that are accessible within a program. If you are on a Unix based system and running bash (and most other shells), the `env` command shows how many variables are currently set. In Python the `environ` dictionary inside the `os` module contains all of these environment variables. Inside settings.py you can set a value based on the contents of `os.environ`. This is particularly handy for things like passwords and keys that you don't want committed to your code repo. Here is an example that sets the `SECRET_KEY` value:

```
# RiffMates/RiffMates/settings.py
...
import os
...
SECRET_KEY = os.environ["RIFFMATES_SECRET"]
```

Instead of using environment variables, you can manage the settings.py file itself. A low-tech solution is to have "settings_dev.py" and "settings_prod.py", and, as part of your deployment, copying the appropriate file to "settings.py". There are a couple of drawbacks to this approach: 1) a lot of values will be common to both, 2) you still have the problem of not storing secrets in your repo, which means you may not want to store either of these files in your code repository.

You can also import values into a settings.py file. The mechanism I use personally is a little convoluted, but I'm happy with it. I use settings.py for values common to all environments, then import extra files based on the deployment environment. The module of the extra files is itself read out of a file. This level of indirection allows you to store all the files in your repo, but

configure for the deployment target by changing the contents of the indirection file.

I store all the environment files in a directory called "local_settings" and the indirection file is named "import_redirect". This file contains the name of the modules to import into the configuration. Code at the bottom of settings.py reads the module name from import_redirect and then dynamically imports that module.

Additionally, there is also a directory called "secrets" containing the files with the same names as the local settings. These files contain configuration values that should not be stored in the code repo like passwords and secret keys. The relationship between all these files is shown in figure B.2.

Figure B.2. Using separate sub-settings files for environment specific settings and secrets


settings.py
DEBUG = False
EMAIL_BACKEND = "... smtp.EmailBackend"
Imports localhost/X and secrets/X
Imports local & secrets version of the named module
Reads module name for secondary imports
import_redirect
chris
Name of module to import
Settings to import
local_settings/
chris.py
DEBUG = True
EMAIL_BACKEND = "... console.EmailBackend"
Can be new or can override
dev.py
prod.py
secrets/
chris.py
SECRET_KEY = "fngvf+6orgp)..."
Settings that shouldn't be checked into the code repo
dev.py
prod.py

This is probably overkill for a small project, but when you have multiple environments and multiple developers, it allows you to set configuration values specific to both a deployment target and, if necessary, a developer. For example, if we were working together, and you wanted to use the Django Debug Toolbar but I didn't, this would be possible because we'd both have our own configuration file. An example file tree looks like this:

```
RiffMates/RiffMates/local_settings/ #1
├── __init__.py
├── README #2
├── import_redirect #3
├── chris.py #4
├── michelina.py
├── dev.py
├── prod.py
└── secrets #5
    ├── __init__.py
    ├── chris.py
    ├── michelina.py
    ├── prod.py
    └── dev.py
```

The import_redirect file and the secrets directory aren't included in your code repo. Adding the following to `.gitignore` insures they wouldn't be committed accidentally:

```
# .gitignore
...
RiffMates/local_settings/import_redirect
RiffMates/local_settings/secrets/
```

The code in settings.py that does the importation work can be seen in listing B.1.

Listing B.1. Code for dynamically importing local settings and secret modules

```
# RiffMates/RiffMates/settings.py
...
# ---- local settings import overrides

# detect if in dev server mode
import sys
```

```
DEV_SERVER = (len(sys.argv) > 1 and \
    sys.argv[1] == 'runserver') #1

redirect = (BASE_DIR /
    'RiffMates/local_settings/import_redirect') #2
with open(redirect, 'r') as f: #3
    import_file = f.read().strip()

action = 'from RiffMates.local_settings.%s import *' \
    % import_file #4
exec(action) #5
if DEV_SERVER: #6
    print('Importing settings: s' % import_file)

action = ('from RiffMates.local_settings.secrets.' #7
    '%s import *' % import_file)
exec(action)
if DEV_SERVER:
    print('Importing secrets: %s' % import_file)
```

Another alternative is partway between these two options. The third-party library django-environ reads from a `.env` file in you project directory and populates the variables within, allowing you to access them. You can also use an environment variable to specify a different name for `.env`, meaning a operating system environment variable can cause `.dev_env` to be loaded in one place and `.prod_env` in another.

B.4.2 Configuring email

In chapter 6 you were introduced to the idea of an email backend. Django comes with several ways of "sending" email, including outputting what would normally be sent to the development server's console. If your Django project uses email, you're going to want to connect to an actual email server.

Email can be a little tricky. Email gets sent using the *Simple Mail Transfer Protocol* (SMTP). For small volumes, your hosting provider may include an SMTP server. Staying out of your user's SPAM folder can be a challenge and properly configuring a server so it isn't suspected of spewing junk mail is important.

For larger volumes of email, you want a service provider. Companies such as

Mailgun, SendGrid, and Mailchimp allow you to send mail through their systems and work with the common email clients such as GMail and Outlook to help ensure you stay out of the SPAM folder.

Whichever service provider you choose, you need to change the configuration in settings.py to point at their system. A sample configuration is as follows:

```
# RiffMates/RiffMates/settings.py
...
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend' #1
EMAIL_USE_TLS = True #2
EMAIL_HOST = 'smtp.example.com' #3
EMAIL_PORT = 587 #4
EMAIL_HOST_USER = 'username' #5
EMAIL_HOST_PASSWORD = 'password'
```

Of course, storing a password in clear text or in a file in your code repository is not a recommended practice. The `EMAIL_HOST_PASSWORD` value should be set using an environment variable or imported from a separate file as discussed in the previous section.

B.4.3 Logging

Whether you realize it or not, you've been using Django's logging functionality all along. The development server console messages indicating a view was visited are from the logging system. Django's logging is built on top of Python's logging, which is extremely configurable (read: a bit complicated to get going).

Python breaks the logging system down into four parts:

- Loggers
- Filters
- Handlers
- Formatters

The logger is what you interact with to log a message. The logger object has a variety of methods that correspond to the log-level of the message you are

logging. The log levels are:

- DEBUG
- INFO
- WARNING
- ERROR
- CRITICAL

The logging filter is responsible for determining which messages get recorded in the logs. The simplest filters are based on the log level. If your filter is set to "ERROR" only messages marked as "ERROR" or "CRITICAL" get recorded. You likely want different filters in different environments. In development, you want everything, which you can get by setting the log-level to "DEBUG". In production, that's a bit too noisy, and you may set it higher. The "WARNING" level is pretty typical on a production server. If a problem occurs in production, you might temporarily change the log level to capture information and help you debug the issue.

The logging handler determines where a message gets logged. The two most common destinations are the console and a file. A logging formatter specifies the format of the message when it is logged. In addition to the message getting logged, you can also add information such as the date and time, the called function, the process and thread IDs, the source code line, and the current phase of the moon. (That last one is a joke, but there are over 20 log attributes you can choose from: https://docs.python.org/3/library/logging.html#logrecord-attributes). You specify the format using a `%s` style string that names the attributes. For example, `"%(message)s"` is the format for logging only the message.

Django's own code uses different logging objects for different kinds of messages. The `django.request` logger logs the messages you've seen when using the development server. The `django.db.backends` logger is for database interactions. This classification allows you to determine which messages make it into your logs. It is also hierarchical, configuring for the `django` logger gets both the `.request` and `.db.backends` content along with everything else.

Setup for logging is done in settings.py using a configuration dictionary. At

minimum, you want a handler and a logger. Most commonly, you'll set up a handler for the console, a separate handler for writing to a file, and a couple of formatters. This allows you to log to different places in different hosting environments. Listing B.2 contains a logging configuration that has both a console and file based handler, a formatter for each, and sends all messages in the system to both handlers.

Listing B.2. Logging configuration for both the console and a file.

```
LOGGING = {
    'version': 1, #1
    'disable_existing_loggers': False, #2
    'formatters': {
        'console': {
            'format': '%(levelname)-8s %(message)s' #3
        },
        'file': {
            'format': \
            '%(asctime)s %(levelname)-8s %(message)s' #4
        }
    },
    'handlers': { #5
        'console': {
            'class': 'logging.StreamHandler', #6
            'formatter': 'console' #7
        },
        'file': {
            'level': 'DEBUG',
            'class': 'logging.FileHandler', #8
            'formatter': 'file', #9
            'filename': '/tmp/debug.log' #10
        }
    },
    'loggers': {
        '': { #11
            'level': 'DEBUG', #12
            'handlers': ['console', 'file'] #13
        }
    }
}
```

The configuration in listing B.2 is a nice example, but is probably not the best choice for your production system. In production, you don't want to log to console, and in development you typically only log to console. This

configuration is also trapping all log levels, which is likely a little verbose for a production setup.

The logging classes specified in the handler configurations point to objects in the Python standard library. I recommend using `logging.RotatingFileHandler` instead of `logging.FileHandler` as it is smart enough to change to a new file when your current log gets too big. To use the rotating handler, you must specify and additional argument of `maxBytes` that specifies the file's size limit.

There are plenty of sources to dig into for more information on logging:

- Django's Logging Documentation (https://docs.djangoproject.com/en/4.2/topics/logging/)
- Python's Logging Documentation (https://docs.python.org/3/library/logging.html)
- Python's Logging Cookbook (https://docs.python.org/3/howto/logging-cookbook.html)

If all that isn't enough, there are also centralized log-management services. These providers are API-based, and you send your log messages to their systems. They typically include log-analysis tools and allow you to have all of your servers log to the same place, giving you an overview of your entire deployment. Two popular log management services are DataDog and Sematext, and the cloud providers have centralized logging within their offerings as well.

B.4.4 Custom 404 and 500 pages

The Django development server provides a lot of handy information when something goes wrong. The 404-page displays which URLs were attempted, and the 500-page shows a complete stack trace. Neither of these are things you want your users to see. With `DEBUG=False` this information isn't shown, but the default pages won't look like your site. This is simple to remedy. Create files named "404.html" and "500.html" inside your templates folder and Django will automatically use them to display the corresponding errors. These files are Django templates, so you can extend your base template and

have the errors look like your project's other pages.

If this isn't sufficient for your error-handling needs, Django allows you to override the views called when errors have been triggered. You can replace them with your own views and perform whatever actions you want. For details on how to write custom error views, see: https://docs.djangoproject.com/en/4.2/topics/http/views/#customizing-error-views.

B.5 Static file management

In chapter 7 you learned how to use static files such as CSS and JavaScript within your Django project. The two key configuration values in settings.py for static files are:

```
# RiffMates/RiffMates/settings.py
...
STATIC_URL = '/static/'  #1
STATICFILES_DIRS = (
    BASE_DIR / 'static', #2
)
```

In production, you want the web server to be responsible for hosting static files; it is just far more efficient. The settings in the example are the same, but you also need to configure your web server. Inside the web server you map URLs to directories, so `/static/` maps to where your static files are located.

Web server permissions are typically associated with the directory of the files being served. You don't want your Django code served by the web server, only the static files. For this reason (and another that I'll get to in a second), it is best practice to copy the static files somewhere outside the project directory for serving by the web server.

Django provides a management sub-command for copying the static files to their production location. This command is `collectstatic`. As part of your deployment process, you run this command and Django copies all the static files to the production location. You inform Django of the destination location by setting `STATIC_ROOT`, for example:

```
# RiffMates/RiffMates/settings.py
...
STATIC_ROOT = OUTSIDE_DIR / 'static'
```

In chapter 7, you learned how I typically have a directory named "outside" at the same level as the project directory. I put user uploaded files in this directory, and in the example configuration, I've done the same with the production static files.

How static files get managed is determined by a pluggable component. Web servers, and web browsers, for that matter, like to cache things. If you update your site's logo, just because you replaced it on the web server doesn't mean it will get served or that the browser will even ask for it; it may use a cached copy. Enter `ManifestStaticFilesStorage`. The class name may be a mouthful, but what it does is automatically rename your static files to include their MD5 hash. An MD5 hash is a short string that represents the content of a file: changing the file changes the hashed string. By appending the hash to the file name, any change to the file changes the filename, meaning it won't be cached. As you're using the `{% static %}` tag to serve your static files, Django takes care of name translation in the URL for you.

Django allows for the customization of other kinds of storage handlers as well, and when configuring to use the MD5 static handler, you have to be careful not to overwrite the other settings. To use the MD5 static handler, include all the following in settings.py:

```
# RiffMates/RiffMates/settings.py
...
STORAGES = {
  "default": {
      "BACKEND": "django.core.files.storage.FileSystemStorage",
  },
  "staticfiles": {
    "BACKEND": \
        "django.contrib.staticfiles.storage.ManifestStaticFilesSt
  },
}
```

Prior to Django 4.2

The `STORAGES` configuration value got introduced in Django 4.2. Prior to that

you used a different configuration value:

```
# RiffMates/RiffMates/settings.py
...
STATICFILES_STORAGE = \
    'django.contrib.staticfiles.storage.ManifestStaticFilesStorag
```

This setting has been deprecated and will be removed in a future version of Django.

Using copies of static files in production means extra steps and complicates simple deployments, but it allows you to do fantastic things at scale. For example, you could have your static files served from an Amazon S3 bucket, or a *Content Delivery Network* (CDN). Both of these require custom handlers and there are third-party packages that integrate the handling. For a bit more information and some pointers to other libraries, see: https://docs.djangoproject.com/en/4.2/howto/static-files/deployment/#staticfiles-from-cdn.

B.6 Other databases

Throughout the examples in this book you've been using SQLite as your database, but Django supports four other databases out of the box:

- PostgreSQL
- MariaDB
- MySQL
- Oracle

There are also a number of third party packages available that plug more database engines into your Django project. The ORM provides a common abstraction across all these databases, but it is just an abstraction. When you write code in a `Model`, Django is mapping your Python into SQL equivalents.

Different databases may handle the same SQL statement differently. For example, SQLite implements both the `CharField` and `TextField` as a text blob, with no size limit, so it ignores the `max_length` attribute of the `CharField`. The `Model` still validates the length, but in a complicated set-up

where multiple programs are sharing the database, you could end up with a `CharField` containing more characters than `max_length` value.

SQLite is the default database, with "db.sqlite3" being the default database file. You've seen this file in your project, and if you were brave you may have even deleted it. The configuration that determines this filename is in settings.py. The `DATABASES` configuration item specifies what database engine to use and what parameters that engine expects. The default Django puts in your settings.py when your project gets created looks like this:

```
# RiffMates/RiffMates/settings.py
...
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Django is capable of interacting with multiple databases at one time. Doing so is outside the scope of this book, but this is why there is a nested dictionary inside the `DATABASES` configuration. You can end up with more than just `default` configured.

To use a different database, you change the `ENGINE` sub-property to point at a different backend. The other key/value pairs in the configuration are database specific. SQLite, being a file-based database, has a simple configuration: just the filename. The other databases interact over the network. To configure for a PostgeSQL database, you provide an `OPTIONS` dictionary instead of the `NAME` argument. The contents of that dictionary specify a service configuration file and a password storage file. The service configuration file contains network information, a username, and the database name to connect with. The password storage file contains your connection password.

More details on interacting with specific databases can be found in the Django documentation: https://docs.djangoproject.com/en/4.1/ref/databases/.

B.7 Caching

For a small project with a few users, your web server will likely be snappy enough that you'll get sufficient performance for your page. As you scale, you may need to introduce caching. A cache records the previous output of a computation and serves the recording up instead of re-running the computation.

In Django's case, the computation is the view. With caching turned on, the first time a user hits a view, the result gets stored in the cache. Subsequent visits to the same view get served out of the cache instead, reducing the amount of code that needs to be run.

Django can use three different types of caches, based on:

- Servers
- Databases
- Files

A cache server is a dedicated piece of software that provides caching functionality. Two popular choices are MemCache and Redis. Your hosting provider may offer these as add-on packages or cloud-based instances.

You can also have Django use your database to cache content. To do this, you need to create an extra table in your database to store the content; this can be accomplished with the `createcachetable` management sub-command.

Alternatively, you can use files to store the cached contents. In this case you point Django at a directory to use for the cache files. This directory should not be inside your project.

In all three cases, you set the `CACHES` value in your settings.py file to turn on caching, indicate your style of caching backend, and provide any additional parameters that your backend requires. The following example is for a MemCache server running on port `11211` on the same machine as the Django instance:

```
# RiffMates/RiffMates/settings.py
...
CACHES = {
```

```
    "default": {
        "BACKEND": "django.core.cache.backends.memcached.PyMemcac
        "LOCATION": "127.0.0.1:11211",
    }
}
```

For more details about caching in Django including the specific settings you need for different backends, see: https://docs.djangoproject.com/en/4.2/topics/cache/.

B.8 Putting it all together

Once you are set up and your infrastructure is all in place, there are a series of actions necessary for each deployment:

- Update your project code on the server
- Install any third-party packages that were newly added to your project
- Optionally upgrade third party packages
- Copy static files to the production location
- Sync the project's ORM with the production database

B.8.1 Builds and releases

In the case of a PaaS, much of this is taken care of for you. For a VPS, you may want to write a script or two ensuring a consistent process with each release. Some PaaS providers split the deployment process into two steps: *build* and *release*. When using containers, the build process is what builds the container object, while the release process pushes it into the infrastructure. Even if you're not using containers, thinking of deployment as a two phase process can be beneficial.

If updating your code or any associated third-party libraries fails, you don't want to run the database migration sub-command. Grouping the code updates, package updates, and static file management into a build script, while putting the migration call in a release script, means you won't be triggering changes in the database unless everything else went smoothly.

How you update the server's copy of your project code depends on your

infrastructure. For simple systems, having a clone of your Git repo on the server means you simply do `git pull` on the branch used for production (typically `main` or `master`). For a PaaS like Heroku, this gets done for you automatically. For more complex systems, with multiple servers being load-balanced, you need to look into configuration management tools such as Chef and Puppet. This doesn't become important until you're scaling, though, and scaling means your server is popular and that's a nice problem to have.

Packages in Python are not a solved problem. There are a variety of toolkits out there that do this and help deal with dependency management. As your project changes over time, you need to ensure that your production environment continues to meet the dependency requirements of your project. Failure to do so might cause bugs in production. In some cases, it could cause the project to crash, resulting in a 500 error. Tools like Pipenv, Poetry, and Hatch help manage package dependencies, and each has its own strengths and weaknesses.

B.8.2 Monitoring

Even when everything goes well and your code got happily deployed, there is a chance of a failure some time in the future. Servers run out of disk space, shared VPS environments have their CPU monopolized by bad neighbors, cosmic particles rain through the ether and blow up processors and disk drives. That last one doesn't happen very frequently, but it has been known to happen.

Once your server is running, you want to make sure it stays running. Or, at least, you want to be notified if it stops running. Monitoring is the process of keeping an eye on your running systems. A simple version of this might be a shell script that once a minute uses `curl` to hit your web server and if it gets no response, it kills and restarts your process.

Web services like Uptime and Pingdom monitor your pages and notify you when a page becomes unreachable. Most monitoring sites even hit your project from a variety of places on the planet, and will log information about the response time. If you end up with a lot of users in a foreign country, you might consider deploying a local server for them to reduce their response

time. These kinds of tools only provide an outside view, a binary "service is up" or "service is down" perspective, but they tend to be inexpensive.

Your hosting provider may offer more detailed services that are integrated with notification tools.

PagerDuty, Dynatrace, and other services like them, allow you to add calls to their API within your code. This is somewhat like logging, but with a view to monitoring your system. Alarms can be set up based on different events, or in the case where an event, like a server heartbeat, is not received in a timely fashion.